מערכת BCI מתקדמת



תוכן עניינים

ספר פרויקט גמר 4

הצעת הפרויקט שאושרה על ידי משרד החינוך: 4

הצהרת מקוריות, יושרה אקדמית ואישור ביצוע פרויקט גמר יד לתואר הנדסאי תוכנה 29

תקציר 30

הרקע לפרויקט 30

תהליך המחקר 31

סקירת ספרות 31

אתגרים מרכזיים 32

הבעיה איתה התמודד התלמיד 32

הסיבות לבחירת הנושא 32

מוטיבציה לעבודה 33

על איזה צורך הפרויקט עונה 33

הצגת הפתרונות לבעיה 33

מטרות ויעדים: 34

אתגרים: 35

מדדי הצלחה וכישלון למערכת: 35

סקירת ספרות: 35

מושגים עיקריים בתחום הדעת של הבעיה: 35

נושאים עיקריים בתחום הדעת של הבעיה: 36

ניתוח חלופות מערכתי 46

הבחירה הסופית ונימוק 47

אפיון המערכת המוצעת 48

ניתוח דרישות המערכת 48

מודול המערכת 49

אפיון פונקציונלי 49

ביצועים עיקריים 50

אילוצים 50

תיאור הארכיטקטורה 50

Top-Down Level Design – הארכיטקטורה של הפתרון המוצע 50

תיאור הרכיבים בפתרון 51

תיאור תהליכים של מערכת ההפעלה שמתבצעים בפרויקט 53

ארכיטקטורת רשת 54

תיאור API בארכיטקטורה 54
תיאור פרוטוקולי התקשורת 55
תיאור ארכיטקטורת שרת לקוח 56
תיאור תהליכי אבטחת מידע בפרויקט 57
תיאור ההגנות 57
הפרויקט עוסק בלמידת מכונה- פירוט והרחבה 58
תהליך איסוף הנתונים 58
טיוב ואיזון נתונים 58
סוג הלמידה הנבחר 59
נוסחאות ורשתות למידה 59
כיוונון ,ייעול וניטור מדדי ביצוע 61
תיאור התוכנה 62
פירוטAPI 62
סביבת עבודה 62
שפות תכנות 63
ניתוח ותרשים UML / UseCase של המערכת המוצעת. 64
תיאור ה UC-העיקריים של המערכת 64
הצגת מקרה עבור כל הפונקציות העיקריות בפרויקט 65
מבני נתונים בהם נעשה שימוש בפרויקט 65
חישוב יעילות האלגוריתם 66
הקשרים בין היחידות השונות 67
עץ מודולים 69
USE CASE DIAGRAM 70
רשימת USE CASES 70
תרשיםUML 73
Design class Diagram 74
תרשים מחלקות 75
תיאור המחלקות המוצעות 76
תרשים מסכים 78
פירוט כל מסך 79
אלמנטים של תצוגה 81
הודעות ומערך התראות למשתמש 81
ממשק משתמש 81

קוד התוכנית 82
תיאור מסד הנתונים 92
מדריך למשתמש 93
בדיקות והערכה 93
מסקנות 94
פיתוחים עתידיים 94
ביבליוגרפיה 95

בס"ד

ספר פרויקט גמר

מגישה: לירז אמסלם

ת.ז.: 216013243
שם המכללה: סמינר גברא בני ברק

סמל מוסד: 595223

מנחה: המורה לירז פרץ

תאריך הגשה: 30/06/2026

הצעת הפרויקט שאושרה על ידי משרד החינוך:

הצעת פרויקט-הנדסת תוכנה

סמל מוסד: 595223

שם מכללה: סמינר בית יעקב גברא בני ברק

שם סטודנט: לירז אמסלם

ת.ז: 216013243

שם פרויקט: שליטה מוטורית באמצעות ממשק מוח מחשב והדגמת התנועה על מסך באמצעות דמות ממוחשבת-NEURO MOTION

פרטי פרויקט-

מבוא

כיום שומעים אנו הרבה מקרים של אנשים שחוו פגיעה בעמוד השידרה וכתוצאה מכך הם לוקים בנכות מסוימת שמונעת מהם את עצמאותם החל מהפרטים הקטנים ביותר כמו להחזיק כוס מים ועד להליכה. מקרים נוספים שהתרבו בעיקר בגלל המלחמה הם המקרים בהם קטעו איבר מסוים כיוון שנפגע או הזדהם ובמקומו מחברים איבר מלאכותי שנקרא פרוטזה. הפרויקט עוסק במציאת פתרון לאנשים בעלי מוגבלות פיזית של תנועה והזזת הגוף כתוצאה מפגיעה בעמוד שידרה או לאנשים קטועי איברים שכיום יש להם איבר מלאכותי-פרוטזה ומתקשים להזיז אותה כרצונם. בעקבות כך חשבתי על רעיון שיכול להביא לפתרון הבעיה הנ"ל ונקרא שליטה מוטורית באמצעות BCI והצגתו כפלט של דמות על מסך-סימולציה של ביצוע כוונת התנועה אותה השגנו מהמשתמש.

מילון מושגים

Brain Computer Interface-BCI תרגום:ממשק מוח מחשב- ערוץ תקשורת ישיר בין מוח לבין התקן חיצוני (למשל מחשב או גפיים רובוטיות). ממשק-מוח מחשב

משמש לעיתים קרובות למחקר, מיפוי, שיפור או תיקון של תפקודים קוגניטיביים, חושיים ומוטורים.

ERD- מונח נוירו-פיזיולוגי המתאר ירידה זמנית ומשמעותית בעוצמת (אמפליטודת) גלי מוח ספציפיים באזור מסוים, בתגובה לאירוע חיצוני או פנימי (כגון חשיבה או רצון).

MI - ראשי תיבות של Motor Imagery, שמשמעותו: דימוי מוטורי הפקודות הרצוניות.

פוטנציאל פעולה- שינוי חשמלי מהיר וזמני במתח החשמלי שבין פנים וחוץ תא עצב או שריר, המשמש להעברת אותות חשמליים; הוא מתחיל ב"דה-פולריזציה" (עליית מתח מהירה) כאשר יוני נתרן נכנסים לתא בעקבות גירוי, ואז חוזר למצב מנוחה דרך "רפולריזציה" (יציאת יוני אשלגן) ו"היפרפולריזציה" קצרה, מה שמאפשר הולכה חד כיוונית של האות, כמו דחף עצבי או התכווצות שריר.

גלי מוח- גלי מוח הם דפוסים של פעילות חשמלית הנובעת מפעולתם של מיליוני נוירונים.

תא עצב/נוירון- תא שנמצא במערכת העצבים ומשמש כיחידת התפקוד הבסיסית שלה. תפקידו לקלוט, לעבד ולהעביר אותות עצביים באמצעות אותות חשמליים וכימיים. המבנה הטיפוסי שלו כולל גוף תא המכיל את הגרעין והאברונים, שלוחה מוארכת (אקסון) להעברת אותות למרחק, ודנדריטים רבים לקליטת אותות מתאים אחרים.

אלקטרודות- אלקטרודה היא מוליך חשמלי המשמש ליצירת מגע עם תווך לא מתכתי של מעגל חשמלי (כגון מוליך למחצה, אלקטרוליט או ואקום). היא משמשת למגוון יישומים, כולל רישום וגירוי חשמלי של גוף, כמו אלקטרודות למוח (ב-EEG) או שרירים (ב-EMG), וכן בתגובות כימיות בתאים אלקטרוכימיים או בתהליכי ריתוך.

אקסון- הוא שלוחה של תא העָצָב שהתמחתה בהעברת פוטנציאל הפעולה ממקום היווצרותו עד לתאי המטרה איתם נוצר המגע הסינפטי.

סינפסה- היא אזור המפגש בין תא עצב לתא המטרה, אשר יכול להיות תא עצב אחר, תא שריר או תאי בלוטות. מקור המילה סינפסה ביוונית ופירושה "לקשר."

מתווכים/מוליכים עצביים- מתווכים עצביים, או מוליכים עצביים (נוירוטרנסמיטרים), הם חומרים כימיים המאפשרים תקשורת בין תאי עצב (נוירונים) לבין עצמם, ובין תאי עצב לתאי שריר או בלוטות. הם מופרשים מקצה של תא עצב אחד אל מרווח סינפטי (מרווח קטן בין תאים), ונקשרים לקולטנים על התא השכן, מה שגורם לתגובה (עירור או עיכוב).

בדיקת EEG- בדיקה המודדת ומסכמת את הפעילות החשמלית של המוח באמצעות אלקטרודות המוצמדות לקרקפת. הפעילות החשמלית המוחית מתועדת בתרשים של גלי מוח.

אלקטרואנצפלוגרם- מכשיר המשמש לרישום הפעילות החשמלית במוח.

הקורטקס המוטורי- זהו אזור במוח הממוקם באונה הקדמית (Frontal Lobe), ממש לפני החריץ המרכזי (Central Sulcus) המפריד בינה לבין האונה הקודקודית (Parietal Lobe).

מעבד קצה קומפקטי- מעבד קטן ובעל יכולות עיבוד מתקדמות, המיועד להרצה של אפליקציות קצה (Edge computing) בסביבות עם מגבלות מקום או משאבים, כמו ברובוטיקה או מכשירים IoT. המונח משלב "מעבד" (Chip), "קצה" (Edge computing)

ו"קומפקטי" (Compact), מה שמרמז על שבב עיבוד קטן הפועל באופן מקומי ליד מקור הנתונים, ולא בענן.

תיאור הפרויקט

כאשר אדם רוצה להזיז איבר בגוף לדוגמא את היד הוא פשוט עושה את זה בלי לחשוב יותר מידי אבל הדרך שבה זה קורה היא ארוכה ומופלאה לחלוטין ואני אסביר את התהליך השלם שקורה בגוף האדם בחמישה חלקים(מצורף מילון מושגים מעל התיאור):

1. השלב הראשוני הוא יצירת פקודה במוח. המוח יוצר את כוונת התנועה ומגייס נוירונים נחוצים בקורטקס המוטורי(חלק במוח). הקורטקס המוטורי מתכנן את התנועה ומפעיל רק את קבוצת תאי העצב הקטנה והיעילה הנחוצה לפעולה הספציפית

   לדוגמה: המוח מזהה את הצורך להרים את היד ומפעיל את הנוירונים המוטוריים העליונים האחראים על שרירי הזרוע.

2. השלב השני הוא הולכה חשמלית כאשר נוצר דחף חשמלי (פוטנציאל פעולה) המועבר במהירות לאורך האקסון הוא מחולל גל חשמלי מהיר. האות נע לאורך האקסון המבודד על ידי שכבת המיאלין. לדוגמה: פוטנציאל הפעולה עוזב את גוף תא העצב המוטורי במוח ומתחיל לרדת לכיוון חוט השדרה.

3. השלב השלישי הוא כאשר האות יורד דרך עמוד השדרה ומגיע לנוירונים מוטוריים תחתונים- הדחף מנווט לאורך המסלולים המוטוריים בחוט השדרה, עובר דקוזציה (חציית קווים) בדרכו, ומגיע אל הנוירונים המוטוריים התחתונים באזור השדרה התואם לגפה (למשל, חוליות צוואריות עבור היד וכו...').

4. השלב הרביעי הוא שחרור מתווכים עצביים בסינפסה- האות החשמלי משתחרר כחומר כימי לתוך הסינפסה. כאשר פוטנציאל הפעולה מגיע לקצה האקסון (הכפתור הסינפטי), הוא גורם לשחרור חומרים כימיים הנקראים מוליכים עצביים הארוזים בתוך בועיות. חומרים אלה משתחררים לתוך המרווח הזעיר שנקרא סינפסה.

5. השלב החמישי הוא שליטה וביצוע של יצירת תגובה בתא המטרה הנוירוטרנסמיטור נקשר לקולטנים בתא המטרה ומעורר כיווץ- המוליכים העצביים נקשרים לקולטנים (receptors) על גבי ממברנת תא המטרה (במקרה זה, תא שריר או בלוטה), ויוצרים בו תגובה. במקרה של שריר, זה גורם ליצירת פוטנציאל פעולה שרירי ולהתחלת מנגנון הכיווץ. לדוגמא היד אוחזת בכוס והיא מכופפת את האצבעות וכך מתרחשת הנעת איבר בגוף האדם.

התהליך בקצרה הוא יצירת פקודה במוח על ידי חלק במוח שאחראי על יצירת תנועה ואז הוא מעביר את הפקודה דרך עמוד השידרה לחלק עיצבי בשריר באותו איבר שבו מתבצעת הפעולה הרצויה.

כל התהליך הזה קורה בגוף של אדם בריא אך אצל אדם שחווה פגיעה בעמוד השידרה ותאי העצב שלו מתים כולם או שנשארו תאי עצב פגועים -אינו יכול להניע את האיברים כרצונו ונתקל בקשיים רבים מאוד בכל דבר שהיה רגיל לעשות. הוא נהפך לתלותי, חסר יכולת ובעל נכות.

פקודה שנוצרה במוח ואמורה לעשות את דרכה לאיבר הרצוי דרך עמוד השידרה בעצם נתקלת במחסום עקב הפגיעה בעמוד השידרה ולכן היא לא יכולה להישלח ולגרום לפעולה המוטורית הרצויה.

על מנת לפתור את הבעיות הללו חשבתי על הרעיון הבא:

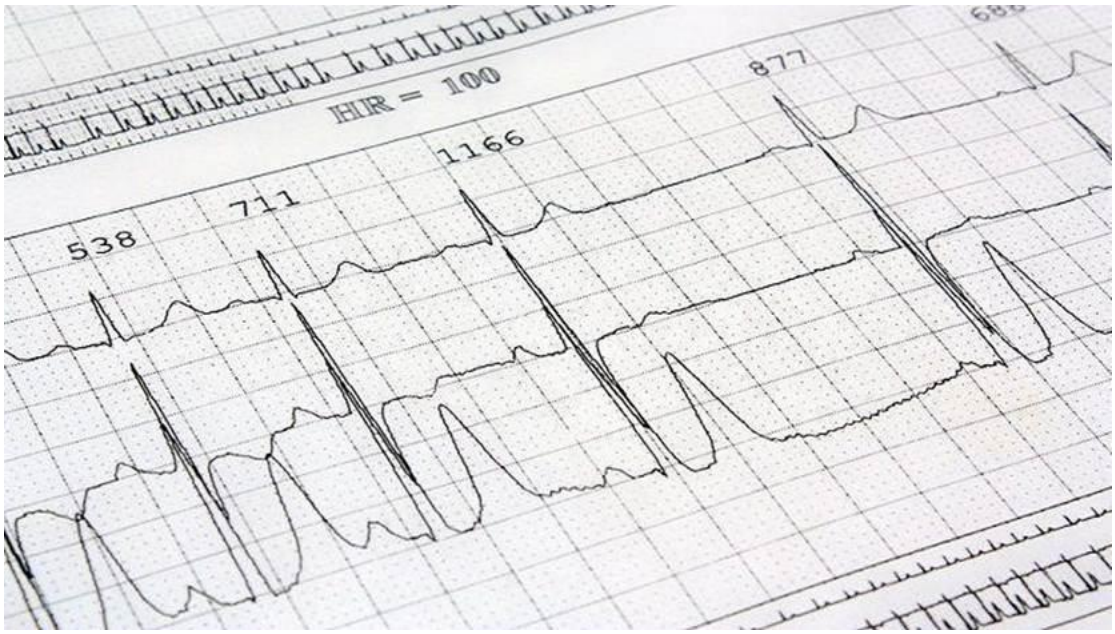
קריאת אותות מוח בצורה לא פולשנית בצורה שמחקה בדיקת EEG. במהלכה מונחות על הקרקפת אלקטרודות מיוחדות שמזהות מטענים חשמליים זעירים שנוצרים מפעילותם של תאי המוח. הפעילות החשמלית תתורגם לצורת גלים על גבי גרף כאשר ציר ה-X מייצג זמן- הגרף מתקדם משמאל לימין, ומציג את הפעילות החשמלית כפונקציה של הזמן. ציר ה-Y מייצג מתח חשמלי-מודד את ההפרש הפוטנציאלי(המתח) בין שתי אלקטרודות או יותר.יש לציין שהאלקטרודות קולטות את גלי המוח העיקריים והמשמעותיים הקשורים לתנועה. הגלים המרכזיים שמעידים על כוונת תנועה ופעילות מוטורית מעובדים באזור הקורטקס המוטורי שהוא המקור העיקרי לתכנון, ייזום ובקרה של תנועות גוף רצוניות.

גלי בטא (Beta Waves)- גלי בטא קשורים למצב של ערנות וריכוז פעיל. בתחום המוטורי, הם קשורים בעיקר לשמירה על מצב נתון (לדוגמה, החזקה של איבר במצב יציב או המתנה).

על מנת לבודד את הגלים הספציפיים הללו צריך להפעיל סינון שיעביר רק את טווח התדרים של גלי מיו ובטא וצריך גם לסנן רעשי חשמל שעלולים להתווסף במלך הקליטה וההעברת הנתונים, לאחר מכן להתחיל לנתח את הטווח של כל גל ובאיזה איזור הוא הודלק וכו...' וכך בסיום התהליך, המערכת יכולה לתרגם את הפעילות החשמלית המוטורית לפקודה.

המערכת תציג על המסך דמות וירטואלית ותדגים עליו את הפעולה המוטורית שפוענחה, המערכת תציג את התנועה שהאדם רוצה לעשות במטרה שבהמשך נוכל לחבר את הקוד ל"פרוטזה חכמה" שתקבל את הפקודות היישר מהמוח ותוכל לזוז באופן אוטומטי על פי פקודות המוח מה שיצור התנהלות חלקה, מדוייקת יותר ומהירה יותר מהאופן בהם פרוטזות עובדות היום.

פרוייקט זה נוצר מתוך הרעיון להגיש עזרה לאנשים נכים מחוסר יכולת לנוע ומגבלות תנועה. לעזור להם להשתקם ולהתגבר על הבעיות וחוסר הנוחות הנגרמות מחוסר יכולת וליקוי בתזוזת האיברים.



הגדרת הבעיה האלגוריתמית:

1. איך המחשב עושה את המעבר משליטה דיסקרטית(בטווח מסויים) לשליטה רציפה?

2. איך מחשב יכול לפענח את גלי מוח ולהבין מהם את כוונת התנועה?

הרחבת הבעיות -

1) סינון רעשים

הבעיה האלגוריתמית הראשונה היא הפיכת אות ה-EEG הנקלט לאות ממוקד ושימושי- להצליח לקרוא את גלי המוח מתוך המוני הרעשים שמגיעים יחד איתם.

בבסיסו, אות EEG הוא אות אנלוגי (מתח חשמלי רציף) המשתנה בהתאם לפעילות הנוירונלית. כדי שמערכת ממוחשבת תוכל לעבד את האות, הוא חייב לעבור דיגיטציה. אות ה-EEG הוא כמו חלש מאוד ומגיע מכמה כיוונים(מרובה ערוצים), ולכן קשה להפריד את המידע החשוב מכל ה"רעשי רקע." מבחינת מחשוב, הבעיה מתחילה ברעש שוטף: רעש Hz/60Hz50 שמגיע ישירות מרשת החשמל הופך לבעיית תדר קבוע שחייבת להיחסם עם מסנן פס-חסימה מדויק. האתגר הגדול ביותר הוא דחיית רעשי הגוף -ארטיפקטים: תנועות העיניים או כיווצי השרירים יוצרים "קפיצות מתח" חזקות שמסוות את אות המוח.

2) אתגר הפענוח והבקרה- מהמרחב ההסתברותי לביצוע דטרמיניסטי.

לאחר עיבוד האות, הבעיה עוברת לאתגר של הפיכת וקטור המאפיינים (גלי Mu ו-Beta) לפקודת קוד אופרטיבית. שלב זה מחולק לשני תתי-אתגרים המגדירים את הלוגיקה של המערכת:

א. ניתוח וסיווג האות (Pattern Classification)

מבחינת מדעי המחשב, האתגר הוא סיווג תבניות (Pattern Recognition) בתוך סביבה רועשת ובעלת ממדים רבים.

- התמודדות עם שונות הנתונים: אות ה-ERD אינו קבוע. הוא מתאפיין ב"סטיית נתונים" (Data Drift) בין משתמשים ובין זמני דגימה שונים. כדי לפתור זאת, אלגוריתם הסיווג (כמו SVM או LDA) מבצע מיפוי למרחב תכונות (Feature Mapping). המטרה היא למצוא את ההיפר-מישור המפריד בצורה אופטימלית בין מצבי "דימוי תנועה לימין" לבין "מצב מנוחה."
- סינון רעשים סטטיסטי (Outlier Detection): במקום "סינון סמנטי," המחשב מבצע סינון סטטיסטי. הוא בוחן את וקטור הקלט אל מול מודל הסתברותי. אם וקטור התכונות נופל מחוץ לטווח הביטחון (Confidence Interval) שהוגדר מראש, המערכת תסווג אותו כ"רעש" (Undefined State) ולא תנפיק פקודה. זהו מנגנון בטיחות קריטי למניעת הפעלות שווא (False Positives).
- זיהוי פרמטרים רציפים: כדי להבחין בין איברים שונים (יד/רגל) או עוצמת תנועה, המערכת משתמשת בסיווג רב-מחלקתי (Multi-class Classification). לכל סוג תנועה מוקצה "תווית" (Label) ייחודית, והמחשב מחשב את ההסתברות של הקלט להשתייך לכל אחת מהתוויות הללו.

ב. עבודה בזמן אמת ובקרת רצף (Real-Time Control)

זהו אתגר של תזמון ורציפות נתונים:

- המרה מבדיד לרציף (Smoothing & Interpolation): תוצאת הסיווג היא ערך בדיד (Binary/Discrete), אך בקרת המנועים דורשת אות רציף. כדי למנוע "גמגום" (Jitter) בתנועת הרובוט, אנו משתמשים באלגוריתם אינטרפולציה או במסנן רקורסיבי (כמו קלמן). האלגוריתם מבצע "החלקה" של ההחלטות הבדידיות והופך אותן לזרם פקודות רציף, המבטיח תנועה חלקה של המנועים.
- דטרמיניזם והשהייה (Latency): המערכת חייבת לפעול תחת אילוצי זמן אמת קשיחים (Hard Real-Time). המשמעות היא שהזמן המקסימלי מקליטת האות ב-ADC ועד ליציאת הפקודה לדמות חייב להיות קבוע וקטן מ-200ms. כל עיכוב מעבר לכך יגרום לקריסת לולאת הבקרה (Control Loop) ולחוסר יציבות של הדמות . השימוש ב-RTOS (מערכת הפעלה בזמן אמת) מבטיח תזמון מדויק של תהליכי העיבוד, כך שתהליך הסיווג לעולם לא יתעכב בגלל משימות רקע של המעבד.

רקע תאורתי

מה זה גלי מוח?

גלי מוח הם אותות חשמליים שנוצרים על ידי תקשורת בין נוירונים במוח, המשקפים את מצב התודעה והפעילות המנטלית, ונמדדים באמצעות מכשיר EEG. ישנם סוגים שונים של גלים (דלתא, תטא, אלפא, בטא, גמא), שכל אחד מהם מתאים לתדר אחר ולמצב מנטלי שונה.

הרחבה על הגלים בהם נעסוק בפרוייקט:

גלי מיו הם הסמן המרכזי לכוונה מוטורית.

גלי המיו, הנעים בטווח תדרים של 8-13Hz הם הכלים החשובים ביותר בזיהוי תנועה במערכות BCI. גלים אלו מתקיימים באופן טבעי מעל הקורטקס הסנסו-מוטורי (האזור שאחראי על הפעלת השרירים) כאשר הגוף נמצא במנוחה. הייחודיות שלהם היא תופעת ה-ERD (די-סינכרוניזציה הקשורה לאירוע): ברגע שאדם רק מדמיין תנועה של יד או רגל, האמפליטודה של גלי המיו באזור המתאים במוח צונחת באופן דרמטי. המחשב מזהה את ה"נפילה" הזו באנרגיה ומתרגם אותה להבנה שהמשתמש מעוניין לבצע פעולה פיזית.

גלי בטא- תפקידם העיקרי הוא עיבוד אקטיבי ומשוב על סיום תנועה.

גלי הבטא פועלים בתדרים גבוהים יותר, לרוב בין 13-30Hz והם מעידים על מצב של עיבוד מידע פעיל וריכוז. בהקשר של תנועה, גלי הבטא עובדים בתיאום עם גלי המיו אך מספקים מידע נוסף: לאחר שהכוונה לתנועה מסתיימת, מופיעה לעיתים תופעת ה-ERS (סינכרוניזציה הקשורה לאירוע), מעין "פרץ" של אנרגיה בתדר בטא המכונה גם " Beta rebound". אות זה קריטי למערכת כדי לדעת שהמשתמש סיים את הפקודה המוטורית שלו, מה שמאפשר שליטה מדויקת יותר ברובוט או בסמן שעל המסך מבלי שהתנועה תימשך מעבר לנדרש.

עיבוד האות במישור המרחבי- ההבדל בין ימין לשמאל:

מעבר לשמות הגלים, זיהוי התנועה מתבסס על הניגודיות המרחבית (Spatial Contrast).
המוח בנוי בצורה קונטרלטרלית – כלומר, צד שמאל של המוח שולט על צד ימין של הגוף, ולהיפך. כאשר המערכת מזהה ERD (דעיכת גל) באלקטרודה C3 (צד שמאל) בלבד, היא מסווגת את הפעולה ככוונה להזיז את יד ימין. ניתוח זה, המשלב את תדרי המיו והבטא יחד עם מיקום האלקטרודה, הוא המפתח להפיכת גלים חשמליים גולמיים לפקודות בקרה של "ימינה," "שמאלה" או "עצור.."

הדרך להשיג את גלי המוח היא באמצעות טכנולוגיית EEG:

קבלת הקלט במערכות BCI מתבססת על **אלקטרואנצפלוגרפיה (EEG)**. זוהי שיטה לא-פולשנית שבה אלקטרודות מוצמדות לקרקפת ומודדות את הפרשי הפוטנציאל החשמלי הנוצרים במוח. האות החשמלי עובר דרך הגולגולת והעור, מוגבר על ידי מכשור אלקטרוני, ומותמר מאות אנלוגי (רציף) לאות דיגיטלי (מספרי) שהמחשב יכול לעבד.

המדידה מתבצעת לרוב לפי תקן ה-10-20, המגדיר מיקומים מדויקים על הראש כדי להבטיח שהאלקטרודות נמצאות מעל האזורים הפונקציונליים הנכונים (כמו 3C ו-4C עבור הקורטקס המוטורי).

דוגמא לשתי מערכות נפוצות המשמשות לקבלת הקלט:

מערכת OpenBCI (לוח Cyton/Ganglion)

מערכת זו היא מסוג מערכת "קוד פתוח" (Open Source) המציעה גמישות מלאה בבחירת סוג האלקטרודות ומיקומן. המערכת מספקת דגימה איכותית ברזולוציה של 24-bit וקצב דגימה של Hz250 ומעלה. המכשיר מורכב מלוח אלקטרוני שאליו מחברים אלקטרודות בודדות (רטובות או יבשות). הוא משתמש במגברי מכשור בעלי עכבת כניסה גבוהה מאוד כדי ללכוד את האותות החלשים. הנתונים מועברים למחשב באמצעות Bluetooth או דונגל ייעודי (Dongle) כדי לבודד את המשתמש מרשת החשמל בשביל הבטיחות ובישביל לצמצם רעשים. מערכת זו פופולרית מאוד במחקר כי היא מאפשרת גישה לנתונים הגולמיים (Raw Data) ללא פילטרים קבועים מראש של היצרן.

מערכת Emotiv (דגמי Epoc+ או Insight)

מכשיר זה הא מכשיר לביש מעוצב ונוח, הכולל זרועות קבועות עם חיישנים מובנים במיקומים אסטרטגיים. המערכת נותנת פתרון מסוג "הכל באחד" הכולל גם חיישני תנועה (גירוסקופ ואקסלרומטר) שעוזרים לזהות ארטיפקטים של תנועת ראש. המכשיר משתמש באלקטרודות חצי-יבשות המשתמשות בתמיסת מלח במקום ג'ל דביק או אלקטרודות יבשות לחלוטין. האלקטרודות מותקנות על קשת פלסטיק גמישה שמתהדקת על הראש ומבטיחה מגע קבוע. המכשיר כולל מעבד פנימי המבצע הצפנה ושידור אלחוטי למחשב. היתרון הגדול שלו הוא המהירות שבה ניתן להתחיל את המדידה (Setup time קצר), מה שהופך אותו לאידיאלי ליישומים צרכניים ולבדיקות מהירות של ממשקי מוח-מחשב.

סינון רעשים

במערכות ממשק מוח-מחשב האתגר הגדול ביותר אינו קליטת האות אלא היכולת להפריד בין הפעילות המוחית הרצויה לבין ה"רעש" שמקיף אותה. אות ה-EEG נמדד בערכים זעירים של מיקרו-וולטים בעוד שהרעשים הסביבתיים והביולוגיים חזקים ממנו בעשרות ומאות מונים. ללא סינון קפדני, המחשב יסווג רעשי שרירים או הפרעות חשמליות במקום את כוונת המשתמש.

מה זה אומר רעש- סוגים ומקומות ההיווצרות שלהם-

את הרעשים באות ה-EEG המכונים "ארטיפקטים" ניתן לחלק לשתי קטגוריות מרכזיות:

א. ארטיפקטים פיזיולוגיים-ממקור פנימי- אלו רעשים הנוצרים מגוף המשתמש עצמו. הגוף הוא מוליך חשמלי, וכל פעילות שרירית מייצרת פוטנציאלים חשמליים ש"נדבקים" לאות המוח:

ב. ארטיפקטים סביבתיים ממקור חיצוני:

רשת החשמלת המכשירים החשמליים סביבנו והתשתית בקירות פולטים שדה אלקטרומגנטי בתדר קבוע של Hz50 (בישראל, בחלק מקומות בעולם תדר אחר). זהו הרעש הנפוץ ביותר, המופיע ב-EEG כ"זמזום" סינוסי קבוע ועוצמתי.

.2אלגוריתמים לניקוי וסינון רעשים

כדי לנקות את האות, אנו משתמשים בשרשרת של פעולות מתמטיות:

א. סינון במישור התדר (Spectral Filtering)

זהו השלב הראשון והבסיסי ביותר, המשתמש במסננים דיגיטליים כדי להסיר תדרים שאינם רלוונטיים למוח:

***מסנן חריץ (Notch Filter):** מסנן זה מתוכנת להסיר תדר ספציפי וצר מאוד. אנו מכוונים אותו ל-Hz50 כדי לבטל את רעשי רשת החשמל.

***מסנן מעביר פס (Band-pass Filter):** בפרויקט BCI המבוסס על תנועה, רוב המידע

נמצא בין Hz8 ל-Hz30 המסנן חותך כל מה שמתחת ל-Hz1 (כדי להסיר תנודות איטיות של הזעת עור או נשימה) וכל מה שמעל Hz40 (כדי להסיר רעשי שרירים גבוהים). השימוש הנפוץ הוא במסנני Butterworth בשל התגובה השטוחה שלהם בטווח התדרים הרצוי.

ב. ניתוח רכיבים בלתי תלויים (ICA - Independent Component Analysis) זהו אלגוריתם "למידה לא מונחית" לניקוי רעשים מורכבים.

ה-ICA מתייחס לאות ה-EEG הנמדד כאל כמה מקורות שונים (מקור מוחי, מקור עין, מקור לב). האלגוריתם מבצע "הפרדת מקורות עיוורת" (Blind Source Separation). הוא מפרק את האותות מכל האלקטרודות למרכיבים בלתי תלויים סטטיסטית. לאחר הפירוק, ניתן לראות בבירור רכיב אחד שנראה בדיוק כמו מצמוץ. האלגוריתם "מכבה" את הרכיב המזוהם ובנה מחדש את האות רק מהרכיבים המוחיים הטהורים. זה מאפשר למשתמש למצמץ בחופשיות במהלך הניסוי מבלי להרוס את הנתונים.

כדי לפתור זאת, המחשב מפעיל את אלגוריתם- שמפרק את הרעש לכמה ערוצים נפרדים, לזהות איזה ערוץ משדר את רעש ה-EMG להשתיק אותו, ורק אז להמשיך עם הערוצים הנקיים. לבסוף, המחשב משתמש בניתוח ספקטרלי כדי לחשב את האנרגיה של גלי המטרה גלי מיו (mu}(8-13{Hz) וגלי בטא (beta}(13-30{Hz}– שיהיו הקלט הסופי למודל למידת מכונה עבור שלב קבלת ההחלטות.

ג. הפחתת רעש מרחבית (Spatial Filtering - CAR)

שיטה פשוטה אך אפקטיבית היא ה-**Common Average Reference (CAR)**. בשיטה הזאת מחשבים את ממוצע המתח בכל האלקטרודות ברגע נתון, ומחסירים את הממוצע הזה מכל אלקטרודה בנפרד.

שיטת ה-CAR מבוססת על ההבחנה ההנדסית בין פעילות מוחית מקומית לבין רעש סביבתי גלובלי. התיאוריה מאחורי השיטה מניחה כי רעשים חיצוניים, כמו הפרעות מרשת החשמל או תנודות הנגרמות מנשימה ודופק, משפיעים על כל שטח הקרקפת באופן אחיד וכמעט זהה. לעומת זאת, האות הנוירולוגי שאנו מחפשים הוא מקומי וספציפי לאזור תפקודי מסוים כמו הקורטקס המוטורי. כאשר אנו מחשבים את ממוצע המתח מכל האלקטרודות ברגע נתון, אנו למעשה יוצרים "חתימה" של הרעש המשותף לכולן. על ידי החסרת הממוצע הזה מכל אלקטרודה בנפרד, אנו מבטלים את המרכיב החשמלי שזהה

לכל הראש ("הרעש הגלובלי") ומבודדים רק את הפעילות הייחודית שהתרחשה מתחת לאלקטרודה הספציפית. תהליך זה מתקן גם הטיות שנובעות מאלקטרודת ייחוס (Reference) שאינה אידיאלית, במקום להסתמך על נקודת ייחוס אחת שעלולה להיות רועשת בעצמה, אנו יוצרים "ייחוס וירטואלי" המבוסס על סך כל הערוצים. התוצאה היא שיפור משמעותי בניגודיות המרחבית של האות, המאפשר למחשב לזהות בדיוק רב יותר שינויים במתח שנובעים אך ורק מהכוונה המוטורית של המשתמש.

ניתוח הגלים וחילוץ המאפיינים

לאחר שלב העיבוד המקדים, האות שנשאר לנו הוא נקי מרעשים חיצוניים אך הוא עדיין מורכב מאוד. אות ה-EEG הגולמי במישור הזמן הוא רצף של ערכי אמפליטודה
(מתח) שמשתנים במהירות. הבעיה היא שהמתח כשלעצמו אינו מעיד על כוונה.
המידע הנצרך הוא זה שטמון ב**דפוסים** של התדר והמרחב. שלב חילוץ המאפיינים הוא התהליך של "צמצום ממדים" (Dimensionality Reduction) – לקיחת אלפי דגימות בשנייה וזיקוקן למספר קטן של "מאפיינים" המייצגים את מהות הכוונה המוטורית.

א. ניתוח ספקטרלי: המעבר למישור התדר

כדי לזהות את תופעות ה-ERD (דעיכת גל) וה-ERS (עליית גל), עלינו להפסיק להסתכל על האות כפונקציה של זמן ולעבור להסתכל עליו כפונקציה של תדר.

1) התמרת פורייה המהירה (FFT - Fast Fourier Transform):
זהו האלגוריתם המרכזי במדעי המחשב לפירוק אותות. הוא מניח שכל אות מורכב מסכום של גלי סינוס ותוסינוס בתדרים שונים. ה-FFT מאפשר לנו לחשב את ה-PSD (Power Spectral Density) – כלומר, כמה "עוצמה" או "אנרגיה" קיימת בכל הרץ (Hz). בפרויקט זה, אנו מחלצים את העוצמה בטווחים הספציפיים של גלי המיו (Hz8-13) והבטא (Hz13-30). ירידה משמעותית ב-PSD בטווח המיו באלקטרודה 3C היא הוכחה מתמטית לכוונה להניע את יד ימין.
2. חלוקה לחלונות זמן :
מכיוון שהכוונה של המשתמש משתנה בזמן אמת, לא ניתן לנתח את כל ההקלטה בבת אחת. אנו מחלקים את האות לחלונות קצרים למשל, חלון של 500 מילי-שנייה שמתעדכן כל 100 מילי-שנייה. על כל חלון כזה מופעלת פונקציית חלון כמו Hamming Window כדי למנוע עיוותים מתמטיים בקצוות, ורק אז מבוצעת התמרת הפורייה.

ב. סינון מרחבי באמצעות אלגוריתם CSP Common Spatial Patterns

זהו האלגוריתם ה"קלאסי" והחזק ביותר בממשקי מוח-מחשב מבוססי תנועה. כיוון שהאלקטרודות שלך ממוקמות מעל הקורטקס המוטורי (C3, C4, Cz), ה-CSP הוא הלב של חילוץ המאפיינים.

קיימת הבעיה הבאה: אותות ב-3C וב-4C דומים מאוד זה לזה בגלל הולכת נפח בגולגולת. קשה להבדיל בין כוונה ליד ימין לכוונה ליד שמאל רק לפי מבט פשוט בגרף.

הפתרון הוא: ה-CSP מוצא מערכת של "מסננים מרחביים" (משקולות לכל אלקטרודה) שממקסמים את השונות (Variance) של האות עבור מחלקה אחת לדוגמא:למשל יד ימין וממזערים אותה עבור המחלקה השנייה לדוגמא:יד שמאל.

התהליך המתמטי שקורה: האלגוריתם מחשב מטריצות של שונות משותפת (Covariance Matrices) עבור כל מצב תנועה. הוא פותר בעיית ערכים עצמיים (Eigenvalue problem) כדי למצוא וקטורים שמפרידים בצורה הטובה ביותר בין המצבים. בסופו של תהליך, האות הגולמי מוכפל במטריצת ה-CSP, והתוצאה היא אות חדש שבו ההבדל בין ימין לשמאל בולט באופן קיצוני.

ג. חילוץ מאפיינים מבוסס זמן-תדר (Wavelet Transform)

שיטה מתקדמת יותר מ-FFT היא **התמרת וובלט ((Wavelet**. בעוד ש-FFT נותן תמונה טובה של תדר אך מאבד את המידע המדויק על ה"מתי" זה קרה בתוך החלון, ה-Wavelet מאפשר רזולוציה גבוהה גם בזמן וגם בתדר. זה מאפשר למערכת לזהות שינויים מהירים מאוד וקצרים בכוונה המוטורית, מה שמשפר את מהירות התגובה של הדמות .

ד. בניית וקטור המאפיינים (Feature Vector)

בסוף שלב זה, כל המידע שחולץ (האנרגיה בתדרי המיו, משקולות ה-CSP, ועוצמת ה-ERS בבטא) נארז בתוך מבנה נתונים אחד הנקרא וקטור מאפיינים.

לדוגמה, וקטור כזה עשוי להיראות כך:

$$|Log(Var_{CSP1}), Log(Var_{CSP2}), PSD_{\mu}, PSD_{\beta}|$$

וקטור זה הוא הקלט היחיד שעובר לשלב הבא – הסיווג. במקום להזין למחשב 1000 דגימות מתח, אנו מזינים לו 4-10 מספרים מזוקקים שמהווים את ה"חתימה הדיגיטלית" של המחשבה.

שלב הסיווג- תרגום המידע לפקודות בקרה

שלב הסיווג הוא המוח של מערכת ה- BCI. לאחר שזוקק וקטור המאפיינים , המערכת נדרשת לבצע הכרעה לוגית: לאיזו קטגוריה שייך האות שנקלט? ללא סיווג, המידע המוחי נותר אוסף של נתונים סטטיסטיים ללא יישום.

א. למה מסווגים? הסיווג נדרש כדי לגשר על הפער שבין העולם הביולוגי הרציף לבין העולם הדיגיטלי הבדיד. המוח מייצר פעילות חשמלית משתנה ללא הרף, אך הדמות או המחשב זקוקים לפקודות ברורות כמו "נוע ימינה," "נוע שמאלה" או "עצור." הסיווג מאפשר לנו שלושה דברים:

1. קבלת החלטות תחת אי-ודאות: המוח הוא מערכת "רועשת" ודינמית. המסווג יודע למצוא את הדפוס העיקרי גם כשהאות אינו מושלם.
2. הפחתת שגיאות: באמצעות הגדרת גבולות החלטה מתמטיים, המערכת מונעת מהדמות להגיב לשינויים מקריים באות המוח שאינם מייצגים כוונה אמיתית. 3 . זמן אמת: סיווג יעיל מאפשר למערכת להגיב בתוך שברירי שנייה, מה שיוצר תחושה של שליטה ישירה וטבעית עבור המשתמש.

ב. איך מסווגים?

פירוט על המכניקה ותהליך הסיווג המבוסס על למידה מונחית והוא מורכב משני שלבים:

- שלב האימון :המשתמש מבצע סדרה של מטלות למשל, דמיון תנועת יד ימין ושמאל מספר פעמים. המערכת לומדת את "חתימת המוח" הספציפית שלו ומגדירה גבול מתמטי שמפריד בין המצבים.
- שלב הבדיקה/זמן אמת : המערכת מקבלת וקטור מאפיינים חדש וקובעת באיזה צד של הגבול המתמטי הוא נמצא.

<u>פירוט על כמה אלגוריתמים מרכזיים שמשתמשים בהם בדרך כלל בפרוייקטים ומערכות מסוג ממשק מוח מחשב (BCI) – הרחבה והעמקה</u> * **ניתוח דיסקרימיננטי ליניארי Linear Discriminant Analysis - LDA** הוא אלגוריתם מפחית ממדים ומבצע סיווג ליניארי שמהווה כלי סטטיסטי קלאסי ביישומי BCI. העיקרון המרכזי שלו הוא למצוא היטל אופטימלי (Optimal Projection) של הנתונים הרב-ממדיים שלנו (וקטור המאפיינים) על ציר אחד או יותר. המטרה היא למקסם את המרחק בין ממוצעי המחלקות (לדוגמה, ממוצע ה-ERD של 'ימין' מול ממוצע ה-ERD של 'שמאל'), תוך מזעור הפיזור או השונות של הנתונים בתוך כל מחלקה. באופן זה, LDA יוצר הפרדה ליניארית ברורה בנתונים מרובי הממדים. רלוונטיות זו קריטית לפרויקט BCI: מכיוון ש-LDA הוא ליניארי, החישוב שלו הוא טריוויאלי ומהיר מאוד בזמן ריצה, והוא דורש משאבי חישוב נמוכים. תכונות אלו הופכות אותו לאידיאלי למערכות זמן אמת (RTOS), בהן כל מילישנייה חשובה. עם זאת, הוא מניח שהנתונים ניתנים להפרדה ליניארית, ולכן הוא משמש לרוב רק לאחר שאלגוריתם CSP מבטיח חילוץ מאפיינים ליניאריים ככל האפשר.

***מכונת וקטורים תומכים (Support Vector Machine - SVM)** SVM הוא אלגוריתם לא-ליניארי עוצמתי המבוסס על מציאת היפר-מישור ההפרדה בעל השוליים (Margin) הרחבים ביותר בין המחלקות. מטרתו היא למקסם את המרחק בין ההיפר-מישור לבין נקודות הנתונים הקרובות ביותר, אשר מכונות וקטורים תומכים. כוחו העיקרי של ה-SVM הוא טריק הגרעין (Kernel Trick) באמצעות פונקציית גרעין כגון Radial Basis Function - RBF, SVM ממפה באופן יעיל את וקטור המאפיינים למרחב ממד גבוה יותר. במרחב החדש הזה, דפוסים מורכבים ובלתי ניתנים להפרדה ליניארית (כמו נתוני ERD/ERS רועשים) הופכים לניתנים להפרדה ליניארית, ושם הוא מוצא את ההיפר-מישור המפריד. SVM מציע דיוק גבוה ויכולת הכללה

Generalization)) טובה לדפוסים משתנים של ERD. למרות ששלב האימון מורכב, שלב הסיווג בזמן אמת הוא מהיר מספיק לשימוש ב-RTOS, מה שהופך אותו לבחירה מומלצת ליישום סופי ואמין.

***רגרסיה לוגיסטית Logistic Regression - LR** הוא מודל סטטיסטי ליניארי המשמש לסיווג בינארי על ידי חיזוי ההסתברות של השתייכות למחלקה ספציפית. הוא עושה זאת באמצעות פונקציית הסיגמואיד:

$$(f(x) = \frac{1}{1+e^{-x}}),$$

שממירה את התוצאה הליניארית לערך שבין 0 ל-1. ב-BCI, הוא משמש לרוב למשימות סיווג בינארי פשוטות, אך חשיבותו העיקרית בפרויקט מורכב יותר כמו שלכם היא ככלי אינטרפרטבילי (ניתן לפירוש). מאחר שהוא מודל שקוף, הוא מאפשר לנו להבין את המשקל היחסי של כל מאפיין (לדוגמה: עוצמת ERD ב-3C מול 4C) בתהליך ההחלטה. בנוסף, ניתן להשתמש בפלט ההסתברותי של LR כשכבת בקרת איכות, המעריכה את מידת הביטחון (Confidence) של הסיווג לפני הפעלת הרובוט הווירטואלי, ובכך משפרת את יציבות המערכת ומפחיתה שגיאות קריטיות (False Positives).

<u>רשת נוירונים קונבולוציונית (CNN) ולמידה היררכית</u>

לעומת הגישה הגיאומטרית של ה-SVM, ה-CNN מייצגת למידה עמוקה המסוגלת לחלץ מאפיינים באופן אוטומטי מבלי שנצטרך להגדיר לה מראש מהו ERD. המכונה מקבלת כקלט מטריצת תדר-זמן (Spectrogram) – מעין "תמונה" דיגיטלית שבה כל פיקסל מייצג את עוצמת האנרגיה במוח בנקודת זמן ותדר ספציפיים. בתוך השכבות החבויות, המכונה משתמשת בפילטרים של קונבולוציה הסורקים את התמונה ומזהים דפוסים היררכיים:
השכבות הראשונות מזהות שינויים בסיסיים במתח, והשכבות העמוקות יותר משלבות אותם לכדי זיהוי "חתימות" מורכבות של כוונה מוטורית.

<u>פונקציית ההפעלה ReLU וניקוי הרעשים</u>

בתוך תהליך העיבוד של ה-CNN, המכונה חייבת רכיב לא-ליניארי כדי להבדיל בין עיקר לטפל. כאן נכנסת פונקציית ההפעלה ReLU (Rectified Linear Unit). מתמטית, הפונקציה מחשבת f(x) = max(0, x). המשמעות הפרקטית היא שהמכונה "משתיקה" אותות שליליים או חלשים מאוד (רעש) על ידי הפיכתם לאפס, ומעבירה הלאה רק אותות חיוביים משמעותיים. פעולה זו מאפשרת לרשת להיות ממוקדת מאוד בדפוסי ה-ERD הרלוונטיים ולמנוע מרעשי רקע אקראיים להשפיע על החלטת הסיווג.

<u>מנגנון האימון והלמידה מטעויות (Backpropagation)</u>

כדי שהמכונה תגיע לרמת דיוק גבוהה, היא עוברת תהליך של אימון (Training) המבוסס על למידה מטעויות. בכל פעם שהמכונה טועה בסיווג במהלך הלמידה, היא מפעילה אלגוריתם שנקרא Backpropagation. בתהליך זה, המחשב מחשב את גודל השגיאה וחוזר אחורה דרך כל שכבות הרשת כדי לעדכן את המשקולות (החשיבות) של כל נוירון. התהליך מתבצע בעזרת "אלגוריתם אופטימיזציה" המנסה למצוא את המינימום של פונקציית ההפסד, ובכך המכונה משפרת את עצמה בכל סבב של נתונים עד שהיא מגיעה ליכולת זיהוי כמעט מושלמת.

הפלט הסופי ופונקציית הסיגמואיד

בשלב האחרון של ה-CNN, המידע המזוקק מגיע לשכבת הפלט, שם נעשה שימוש בפונקציית הסיגמואיד (Sigmoid) או Softmax. פונקציה זו דוחסת את כל החישובים לערך שבין 0 ל-1, המייצג הסתברות. הפלט שהמערכת מוציאה הוא וקטור של הסתברויות (לדוגמה: 95% לימין, 3% לשמאל). המחשב בוחר את הערך הגבוה ביותר ומנפיק את הסיווג הדיגיטלי הסופי. מעבר זה, מהסתברות סטטיסטית להחלטה נחרצת, הוא מה שמאפשר לשלוח פקודה ברורה למנוע הגרפי של הדמות הווירטואלית ולהניע אותו על המסך.

איך מאמנים את המודל?

תהליך אימון המודל בפרויקט שלי מתחיל בבניית סט נתונים שבו אוספים דגימות EEG גולמיות ומתייגים אותן בתוויות (Labels) לפי כוונת המשתמש – למשל, תיוג קטע קוד כ"ימין" או "שמאל." בשלב הבא, מתבצע חילוץ מאפיינים (Feature Extraction) שבו הופכים את המתח החשמלי לוקטורים של מספרים שמשמשים כקלט לאלגוריתם. האימון עצמו הוא תהליך איטרטיבי שבו המודל, לדוגמא SVM- מנסה למצוא פונקציה מתמטית שתבצע הפרדה אופטימלית בין המחלקות השונות. המחשב משווה את התחזית שלו לתווית האמיתית ומחשב את ה"הפסד" (Loss Function) – ככל שהטעות גדולה יותר ה-Loss גבוה יותר. כדי להשתפר המודל מבצע עדכון משקולות (Weights Update) בעזרת אלגוריתם אופטימיזציה במטרה למזער את הטעות בכל סבב למידה (Epoch). כדי לוודא שהמודל לא פשוט "שינן" את הנתונים (Overfitting) משתמשים בטכניקה של Cross-Validation- בה בודקים את המודל על נתונים שהוא מעולם לא ראה. דוגמה לכך היא אימון המודל על 80% מהזמן שהקלטנו ובדיקתו על ה-20% הנותרים כדי לראות אם הוא באמת זיהה חתימת מוח ולא סתם רעש מקרי. בסיום התהליך מקבלים מודל "קפוא" (Frozen Model) שאותו מטמיעים במערכת בזמן האמת. המודל הזה מקבל זרם נתונים חי ומריץ עליהם את הפונקציה שלמד ופולט פקודה שמזיזה את הדמות על המסך.

מהו תהליך האופטימיזציה שעובר המודל?

אופטימיזציה היא התהליך ההנדסי של כוונון המודל בשביל שיגיע לביצועים המקסימליים והיא מתבצעת על ידי שינוי הפרמטרים הפנימיים שלו. לדוגמא: בתוך ה-CNN משתמשים באלגוריתם אופטימיזציה שתפקידו לעדכן את המשקולות בכל שכבה כדי לצמצם את השגיאה של המערכת. רכיב קריטי בתהליך הזה הוא פונקציית ההפעלה ReLU שמתנהגת כפילטר לוגי: זה אומר שהיא מאפסת (Set to 0) כל אות שנמצאת מתחת לסף מסוים ובזה היא משתיקה רעשי רקע חלשים ומאפשרת לרשת להתמקד רק במידע המשמעותי. דוגמה לאופטימיזציה היא בחירת ה-Learning Rate שקובע כמה מהר המודל לומד כי קצב מהיר מדי עלול לגרום למודל לדלג מעל הפתרון האופטימלי ולעומת זאת קצב איטי מדי יגרום להשהייה גבוהה בזמן האימון. בשלב הזה מבצעים גם כוונון ל-Hyperparameters שהוא כמו גודל חלון הזמן (Time Window) של אות ה-EEG שנשלח למערכת. אם החלון קצר מדי לא יהיה מספיק מידע לסיווג ואם הוא ארוך מדי המערכת לא תעמוד בדרישות ה-Real-Time. המטרה הסופית היא למצוא את הנקודה שבה המודל מזהה את כוונת התנועה במינימום זמן עיבוד ומקסימום דיוק ובסוף

התהליך המודל עובר אופטימיזציה של קוד כדי שיוכל לרוץ במהירות על מעבד המחשב או הבקר ללא עיכובים ויזואליים.

מה זה ?OVERFITTING

בעולם התכנות המושג Overfitting מתייחס ללמידת יתר.

קודם כל נרחיב על הבעיה: בעיית ה-Overfitting היא מצב שבו המודל הופך להיות מומחה רק לנתוני האימון שהקלטנו אבל הוא נכשל לחלוטין בזמן אמת מול נתונים חדשים. זה קורה כשהאלגוריתם משנן רעשים ספציפיים שהיו בחדר בזמן ההקלטה (כמו רעש מזגן או תנועה מקרית של המשתמש) וטועה לחשוב שהם חלק מחתימת המוח של "יד ימין" לדוגמא. כדי למנוע זאת משתמשים בטכניקת Regularization- -מוסיפה -כמין קנס מתמטי למודל אם הוא הופך להיות מורכב מדי מה שמחייב אותו להישאר פשוט וכללי. כלי נוסף ב- לדוגמא CNN הוא ה-Dropout שבו מכבים נוירונים באקראי בכל סבב אימון וככה מונעים מהרשת להסתמך על תדר בודד או אלקטרודה בודדת ומכריחים אותה ללמוד תבניות רחבות יותר.
דוגמה מעשית למניעה היא שימוש ב-Early Stopping- שמפסיקים את האימון ברגע שרואים שהדיוק על נתוני הבדיקה מתחיל לרדת וגם אם הדיוק על נתוני האימון ממשיך לעלות. מיישמים גם Data Augmentation על ידי הוספת רעש לבן (White Noise) מכוון לנתוני האימון, כדי לאמן את המודל להיות חסין להפרעות. המטרה היא להשיג יכולת הכללה- גנריות בצורה שהמערכת תדע לזהות את כוונת התנועה של המשתמש גם אם הוא קצת עייף יותר מאשר ביום האימון. בסופו של דבר מניעת Overfitting היא ההבדל בין פרויקט שעובד רק במעבדה לבין מערכת BCI אמינה שבאמת מאפשרת שליטה חלקה בדמות הוירטואלית כדי להשיג למשתמש את תחושת השליטה העצמית.

הליכים עיקריים בפתרון בעיה בטכנולוגיות הנדסה מתקדמות:

טבלת ניתוח אלגוריתמים ו- SWOT לבעיה האלגוריתמית

מספר הבעיה האלגוריתמית	תיאור הבעיה האלגוריתמית	האלגוריתם לפתרון הבעיה	למה הוא מתאים לפרויקט שלי?	SWOT: יתרונות	SWOT: חסרונות

1- מיקוד האות	הפרדת האות המוחי מרעשי רקע	**CNN-** רשת נוירונים קונבולוציונית	מסנן רעשים באופן שבו הרשת בנויה משכבות - הדרך בה הפרויקט בנוי, מכמה שלבים. בנוסף לניקוי המידע היא גם מסווגת וגם מזקקת את המידע.	המכונה מגלה דפוסים לבד. יש לה חסינות לרעש וביצועים מקסימליים.	צורכת נתונים רבים ודורשת חומרה ברמה גבוהה מאד מה שעלול לגרום לשהייה.
		DWT-Wavelet Transform	הוא מתאים כי הוא מפרק את האות גם לפי זמן וגם לפי תדר בו-זמנית. זה מאפשר לזהות רעשים קצרים בזמן בלי לפגוע בתדרים המוחים היציבים.	יכולת מצוינת לניקוי אותות לא קבועים (Non-stationary) הוא שומר על המידע של "מתי" קרה האירוע המוחי בדיוק רב.	דורש בחירה ידנית של "אמא גל" (Mother Wavelet) שמתאימ ה ל-EEG, מה שדורש הרבה ניסוי וטעייה.

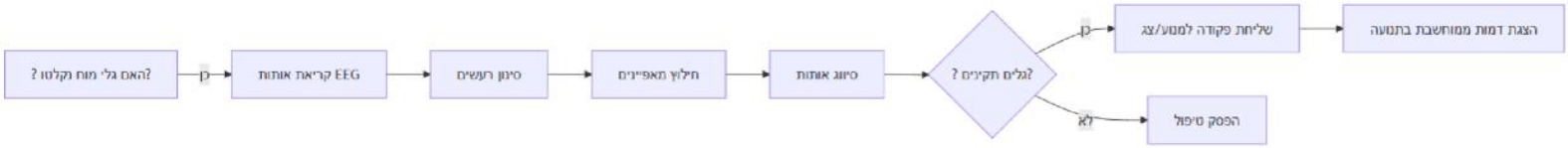
סינון רעשים	הפרדת האות המוחי מרעשי רקע	**ICA- Independent Component Analysis**	הוא מתאים כי הוא יודע לפרק את אות ה-EEG לרכיבים נפרדים. אם המשתמש מצמץ, ה-ICA מזהה את המצמוץ כרכיב נפרד ומאפשר למחוק רק אותו בלי לפגוע באות המוחי שנמצא מתחתיו.	יכולת הפרדה מעולה בין רעש ביולוגי (עיניים/שרירים) לאות מוחי. מאפשר לראות את המוח נקי באמת.	דורש כוח חישוב משמעותי. קשה מאוד לביצוע בזמן אמת (דורש הרבה נתונים כדי להצליח לפרק את הרכיבים).
2 א'- סיווג	מיפוי הקלט לקטגוריה ספציפית	**SVM- מכונת ווקטורים תומכים**	הוא מצליח להפריד בין מחשבות שאלגוריתמים פשוטים יותר יראו כאותו דבר, הוא פחות רגיש לרעשים קטנים בזכות השוליים, הוא עושה את כל החישוב הזה בתוך המרחב הגבוה אבל הפלט שהוא נותן בסוף הוא פקודה פשוטה ומהירה: "ימין" או "שמאל".	מכונה סטנדרטית ביותר בשימוש של מערכות BCI - יש לו איזון מקצועי.	האימון שלו איטי יותר מ-LDA ודורש ניסוי וטעיה כדי למצוא את הגרעין המתאים ביותר.

2 א'- סיווג - המשך	מיפוי הקלט לקטגוריה ספציפית- המשך	**LR- Logistic Regression**	הוא מתאים למשימות סיווג בינאריות פשוטות (כמו "מנוחה" מול "פעולה"). הוא מספק ערך הסתברותי (בין 0 ל-1), מה שמאפשר להחליט שרק אם המחשב בטוח מעל 80%, נזיז את הדמות.	מודל ופשוט מאוד להבנה. מאפשר בקרת איכות על ההחלטה בזכות הפלט ההסתברותי. קל מאוד למימוש.	מוגבל מאוד בהפרדה של דפוסים מורכבים. דומה ל- LDA במובן שהוא ליניארי ועלול להיכשל אם הנתונים מסובכים
		Random Forest - יער אקראי	הוא מתאים כי הוא מורכב מהמון "עצי החלטה" שעובדים יחד. הוא מעולה בלהבין אילו תדרים הם הכי חשובים לסיווג ומספק דירוג חשיבות לכל אלקטרודה במערכת שלך.	עמיד מאוד בפני רעשים ומידע חסר. לא דורש שהנתונים יהיו מסודרים בצורה גיאומטרית (כמו SVM) או ליניארית (כמו LDA).	עלול להיות "כבד" ואיטי בזיכרון אם היער גדול מדי. קשה יותר למימוש על בקר קצה כמו 32ESP בהשוואה ל-SVM.

2. ב'- זמן אמת ושהייה	אתגר להבטיח שהמערכת תגיב מהר מספיק כדי שהמשתמש ירגיש שליטה ישירה	**LDA- ניתוח דיסקרימיננט י לינארי**	הוא בעל חישוב מהיר במיוחד שמונע שהייה בין מחשבת המשתמש לפלט על המסך מה שיתן למשתמש הרגשה של שליטה.	מהירות תגובה מקסימלית. דורשת מעט זיכרון וכוח עיבוד. אינה נוטה ללמידת יתר כשמדובר בכמות קטנה של נתונים.	לא מסוגל להפריד נתונים שאינם ניתנים להפרדה על ידי קו ישר. הוא בעל תלות בעיבוד מוקדם כי הוא חייב מאפיינים נקיים וליניארים ואותות מוח אינם כאלה
		LSL- Lab Streaming Layer	זהו אלגוריתם לניהול זרם נתונים. הוא מתאים כי הוא מבצע "סנכרון שעונים" בין ה-32ESP למחשב, ומבטיח שהמידע יגיע בסדר הנכון גם אם היו עיכובים ברשת ה-Wi-Fi.	פותר את בעיית ה-Jitter (תנודות בזמן ההגעה של המידע). מאפשר הזרמת נתונים חלקה מכמה מקורות במקביל ללא תקלות.	מוסיף שכבת תוכנה נוספת שיכולה להיות מורכבת להגדרה ראשונית ב-C++.

2. ב'- זמן אמת ושהייה - המשך	אתגר להבטיח שהמערכת תגיב מהר מספיק כדי שהמשתמש ירגיש שליטה ישירה- המשך	**Kalman- מסנן קלמן - Filter**	הוא מתאים כי הוא אלגוריתם חיזוי. הוא לוקח את החלטות הסיווג הקופצניות והופך אותן לתנועה רציפה. הוא חוזה איפה הדמות צריכה להיות בפריים הבא ובכך מפצה על ההשהייה של המערכת.	יוצר תחושת שליטה חלקה וטבעית למשתמש. מתקן טעויות סיווג רגעיות (Outliers) בצורה אוטונומית.	דורש הגדרה מתמטית מדויקת של מודל התנועה- איך היד אמורה לזוז. מורכב לתכנות על בקרים קטנים.

תרשים זרימה



- קבלת גלי מוח באמצעות בדיקת EEG .
- ניקוי המידע מרעשים.
- אימון מודל לסיווג והבנת כוונת התנועה.
- איסוף הנתונים ושליחתם להצגה על מסך באמצעות דמות שתראה את כוונת התנועה המוטורית והצגה על המסך.

הליכים עיקריים בתחום למידת מכונה:

איסוף המידע של אותות המוח הוא באמצעות בדיקת EEG .

בחינת החלופות לסיווג במסגרת הפרויקט שלי :

- **SVM -מכונת ווקטורים תומכים: מודל המצטיין בהפרדה ליניארית ולא-ליניארית של מאפייני תדר, ונחשב ליציב ומהיר מאוד לחישוב.**
- **CNN -רשתות נוירונים: גישה של למידה עמוקה המאפשרת למערכת ללמוד באופן אוטונומי את המאפיינים המרחביים והזמניים מתוך האות הגולמי או מתוך ספקטרוגרמות.**
- **LDA : חלופה נוספת הנבחנת בשל יעילותה החישובית הגבוהה המאפשרת תגובה מהירה במערכות זמן-אמת.**

לא הגעתי להחלטה סופית של סוג המודל בו אשתמש בפרוייקט.

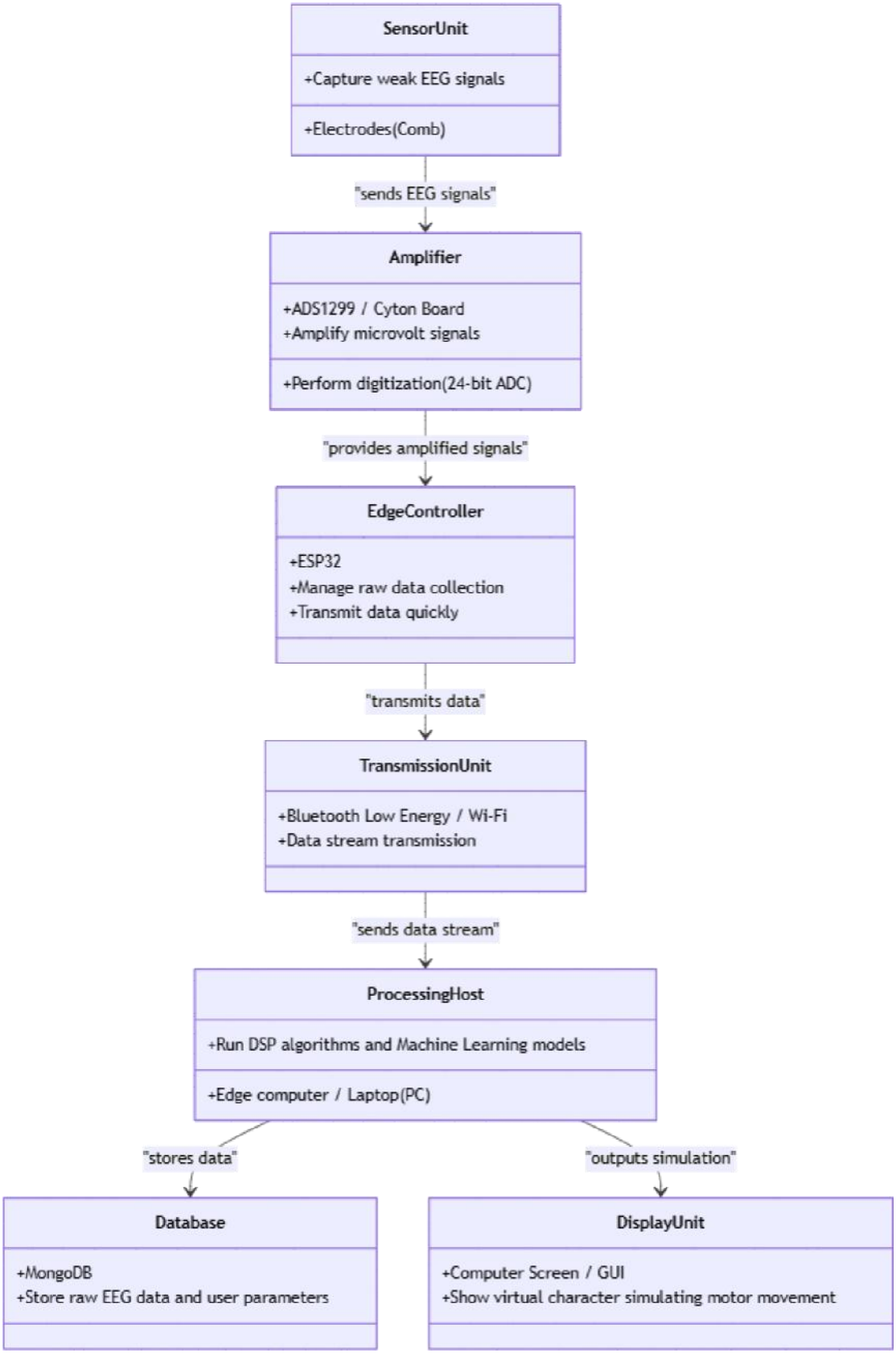
מעקב אחר נתוני הביצוע:

- **נבדוק את אחוזי ההצלחה בזיהוי נכון של כוונת התנועה לעומת מקרים של "זיהוי שווא."**
- **נעשה כוונון פרמטרים כדי למזער את טעויות הסיווג.**
- **נמדוד את הזמן שלוקח למודל להגיע להחלטה בשביל להבטיח תחושת שליטה חלקה ומיידית למשתמש.**

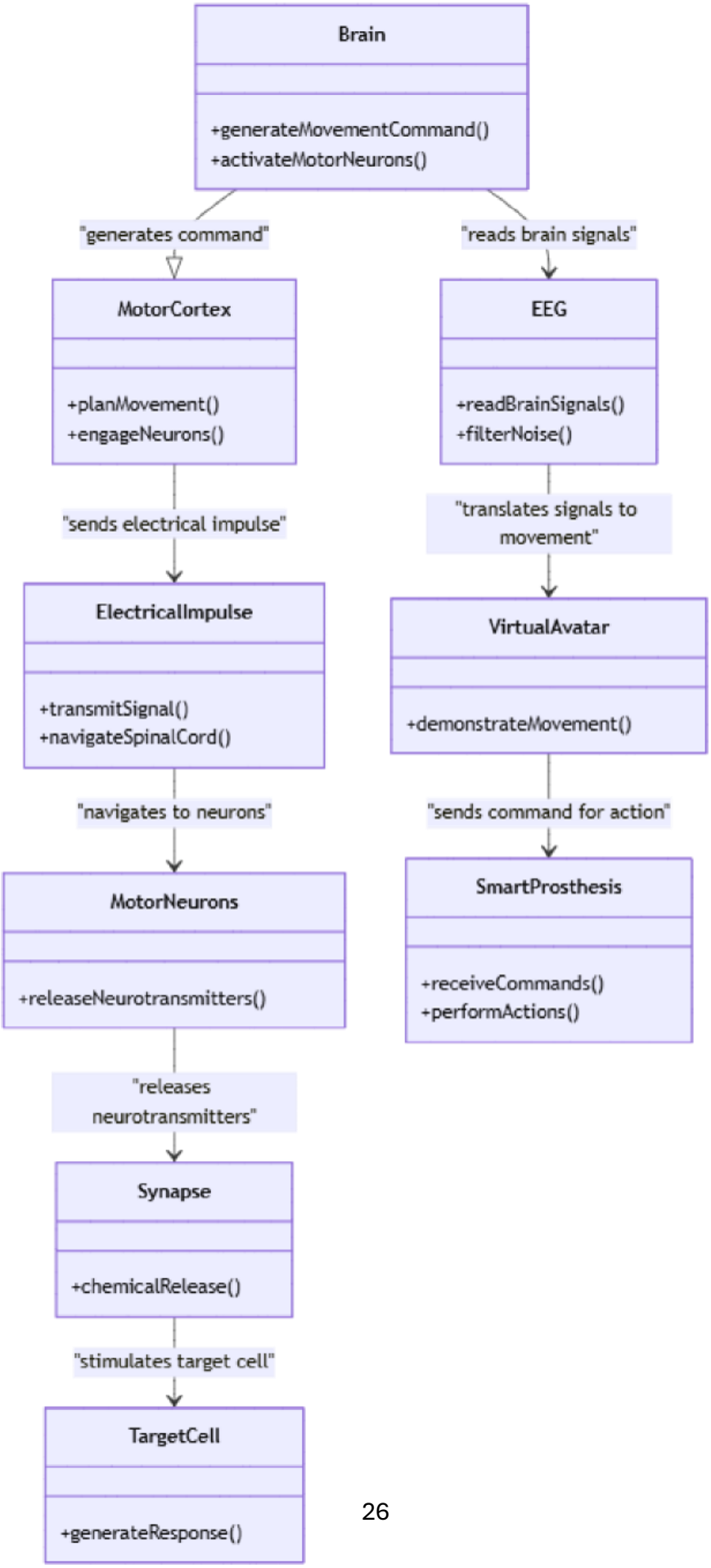
הליכים עיקריים בתחום מערכות הפעלה/רשתות משבים/תקשורת נתונים/אבטחת מידע:

לא קיים התייחסות בפרויקט שלי.

תרשים ארכיטקטורה פיזית



תרשים ארכיטקטורה לוגית



פיתוחים עתידיים:

הרצון הוא שלאחר שהמערכת תפענח את הנתונים לכוונת התנועה היא תשלח פקודת בקרה קצרה ופשוטה. מערכת זו היא מערכת ניידת בצורת מעבד קצה קומפקטי המלווה את המשתמש לאורך היום מה שמעניק ניידות מלאה והשהייה מינימלית, היא חייבת להיות מהירה וזמינה לאדם בכל עת שיחפוץ ולכן היא תעבוד תמיד כדי להבטיח יעילות וחופש תנועה. המערכת תשלח את הפקודה לתוך צמיד או "שעון יד" המכיל בקר וסוללה ומחובר למספר אלקטרודות שיונחו על האיבר לכל אורכו במיקומים מסויימים בהם נדרשת הפעלה של אותו שריר הדרוש לביצוע תנועה כלשהיא וכאשר תתקבל הפקודה אותה פיענחנו מהגלים ששודרו היא תישלח אל הבקר באופן אל חוטי. הבקר יפעיל אלקטרודות מסוימות לגירוי השריר והגירוי החשמלי הוא הגורם לשרירים להתכווץ ולבצע את התנועה המבוקשת.

במקרה של פרוטזה האות ישלח ישירות למנועים החשמליים המובנים בתוך הפרוטזה ותתקבל תוצאה זהה

- לעזור לאנשים הסובלים מאותם נכויות מסיבות אחרות- הרחבת ההתאמה למערכת לרבות פגיעות הנובעות מ: שבץ מוחי, חבלת ראש, שיתוק מוחין, טרשת נפוצה ועוד...
- לעזור לאנשים הסובלים מתנועות בלתי רצוניות כגון: תסמונת טורט, בליסמוס, טיקים, פרקינסון, הנטינגטון ועוד... המטרה היא לעזור להם בצורה ההפוכה בדיוק מצורת העזרה בפרויקט זה. אם פרויקט זה עוסק בגירוי שיוביל לתנועה אז הפיתוח יהיה זיהוי תנועה בלתי רצונית וויסות שלה עד כדי מניעתה לדוגמא: מניעת רעד ופירכוס, מניעת טיקים וכו...'

תיאור טכנולוגית הנדסה:

ממשק מוח מחשב הוא ערוץ תקשורת ישיר בין מוח לבין התקן חיצוני למשל מחשב או גפיים רובוטיות. ממשק-מוח מחשב משמש לעיתים קרובות למחקר, מיפוי, שיפור או תיקון של תפקודים קוגניטיביים, חושיים ומוטורים.

מערכת BCI מזהה אותות עצביים מהמוח, מנתחת אותם ומתרגמת אותם לפעולות או פקודות שמכשירים יכולים להבין ולבצע.

זהו נושא קריטי המהווה את הליבה התיאורטית של כל פרויקט BCI (ממשק מוח-מחשב), והוא מדגים כיצד המדע הביולוגי פוגש את מדעי המחשב וההנדסה.

הפרוייקט הוא בעצם מערכת שמשתמשת בחומרה כדי לקבל את הנתונים על המוח בזמן אמת. החומרה הכוללת כובע מיקומי הנחת אלקטרודות, אלקטרודות מסרק, מגבר אותות ADS 1299 ובקר 32ESP. החומרה הזו בעצם משיגה את הנתונים איתם אעבוד כדי להסיק ולפענח את כוונת התנועה. לאחר שמצאנו את כוונת התנועה נציג אותה ויזואלית בזמן אמת על מסך באמצעות רובוט שידמה את תנועת המישתמש.

במערכת שלי השפות שיעבדו הכי טוב הן C ו-C++ כיוון שהן קרובות למעבד ולכן עובדות מהר מאוד שזהו עיקרון חשוב בפרוייקט שלי- המהירות בה נראה את הפלט

תנועה על המסך ושפה נוספת היא שפת פייתון שמכילה הרבה סיפריות ובנושא שלי BCI היא מכילה גם הרבה ספריות בנושא.

C / ++C-שפות מהירות **יותר ובפרויקט זה המהירות היא קריטית.**

יש להם גישה ישירה לחומרה והידור ישיר לקוד. C מציעה יעילות ופשטות למערכות קריטיות, ו C++ היא שפה מונחית עצמים, גמישה, ויש לה ביצועים גבוהים ויעילות.

פייתון - שפה גמישה, מונחית עצמים ובנוסף יש לה מאגר ענק של ספריות.

מסד נתונים:

כיוון שהזמן הוא הרכיב החשוב אם לא החשוב ביותר בפרוייקט שלי יש להשתמש במסד נתונים שעובד בצורה מהירה- שליפת והכנסת נתונים כמו mongoDB כדי שנוכל לאסוף ולהציג נתונים בזמן אמת.

לוחות זמנים:

ספטמבר: גיבוש רעיון וחקר

אוקטובר: כתיבת הצעה ראשונית

נובמבר: הגשת הצעה ובניית פלטפורמה

דצמבר: אפיון המערכת

ינואר: אפיון בסיס נתונים

פברואר: בניית התוכנה

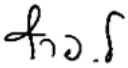
מרץ: עיצוב המערכת

אפריל: בדיקות

מאי: הכנת תיק הפרויקט

יוני: הטמעת המערכת

מנחת הפרויקט: המורה לירז פרץ



חתימת הפרויקט: לירז אמסלם



חתימת רכז המגמה:

חני גרשוני

הצהרת מקוריות, יושרה אקדמית ואישור ביצוע פרויקט גמר יד לתואר הנדסאי תוכנה

שם הפרויקט : neuro motion

תאריך אישור הצעת הפרויקט המקורית:

חלק א': הצהרת הסטודנט/ית (יושרה ומקוריות העבודה)

אני החתום/ה מטה מצהיר/ה ומחייב/ת בזאת כי:

1. **עבודה עצמאית ומקוריות :** ספר הפרויקט, קוד המקור, המערכות והתוצרים הנלווים המוגשים על ידי, נכתבו, פותחו ונערכו על ידי באופן בלעדי ועצמאי. התוצרים אינם מועתקים מכל מקור אחר)לרבות פרויקטים של סטודנטים אחרים, עבודות קודמות, מאמרים, או קוד מקור פתוח/סגור(, למעט חומרים וציטוטים שצוינו כדין ובהתאם לכללי הציטוט האקדמיים המקובלים.

2. **מגבלות וחובת דיווח בשימוש בכלי בינה מלאכותית יוצרת(AI Generative)**

2.1 ככל שנעשה שימוש בכלי בינה מלאכותית ,כגון ChatGPT ,Claude ,Copilot וכדומה, במהלך העבודה, הדבר נעשה **ככלי עזר בלבד**)לצורך ניסוח, עריכה לשונית, איתור שגיאות בקוד או סיעור מוחות(ולא כתחליף לחשיבה, לתכנון, לכתיבה או לפיתוח העצמאיים שלי. 2.2 ידוע לי כי יצירת חלקי טקסט נרחבים, כתיבת פרקים שלמים או הפקת קוד המקור של הפרויקט ישירות על ידי כלי בינה מלאכותית – ללא פיתוח, הבנה ועיבוד עצמי שלי – נחשבים כהפרה של היושרה האקדמית וכאקט של העתקה.

2.3 אני מתחייב/ת כי כל שימוש מהותי שנעשה בכלים אלו במהלך העבודה פורט ודווח באופן שקוף, מפורש ומלא בגוף ספר הפרויקט או בנספח ייעודי לכך, בהתאם להנחיות המוסד הלימודי.

3. **ידיעת הסנקציות :** ידוע לי כי העתקה, הצגת עבודה שאינה שלי, שימוש לא מורשה בחומרים מוגנים או אי- דיווח על שימוש בכלי AI מהווים עבירה משמעתית ואקדמית חמורה. אני מודע/ת לכך שהפרת סעיפים אלו עלולה להוביל לפסילת הפרויקט לאלתר ולנקיטת צעדים משמעתיים נגדי במוסד הלימודים.

שם מלא של הסטודנט/ית : לירז אמסלם

מספר תעודת זהות : 216013243

חתימה : לירז אמסלם

תאריך :

<u>חלק ב': הצהרת מנחה הפרויקט (פיקוח והתאמה להצעה)</u>

אני החתום/ה מטה, מנחה הפרויקט, מצהיר/ה בזאת כי:

1. **פיקוח ויושרה :** למיטב ידיעתי, הבנתי ועל בסיס מהלך הפיקוח והמפגשים השוטפים שלי עם הסטודנט/ית לאורך תקופת העבודה, הסטודנט/ית ביצע/ה את הפרויקט בפועל, שולט/ת בחומר המוצג, ופעל/ה בהתאם לעקרונות היושרה האקדמית המצופים ממנו/ה.
2. **תאימות להצעת הפרויקט :** הפרויקט, על כלל תוצריו וספר הפרויקט המצורף, בוצע, פותח והושלם בהתאם להצעת הפרויקט המקורית כפי שאושרה)או בהתאם לשינויים והתאמות טכנולוגיות/פדגוגיות מאושרות שסוכמו בינינו ותועדו במהלך תקופת הפיתוח(.
3. **אישור להגשה :** העבודה הגיעה לרמה הנדרשת ואני מאשר/ת את הגשתה להערכה ולבחינה סופית.

שם מלא של המנחה:

חתימה :

תאריך: :

תקציר

פרויקט **NeuroMotion** מציג מערכת ממשק מוח-מחשב BCI - Brain-Computer Interface מתקדמת מקצה לקצה, המיועדת לפענח אותות חשמליים מהמוח (EEG) בזמן אמת ולתרגם אותם לפקודות תנועה דיגיטליות עבור אוואטאר רובוטי תלת-ממדי. הליבה הטכנולוגית של הפרויקט משלבת חומרת קצה ייעודית מבוססת מיקרו-בקר 32ESP, צינור עיבוד אותות ספרתי אקטיבי , מודל לניקוי רעשים, מודל למידה עמוקה מבוסס רשת עצבים קונבולוציונית (EEGNet), וממשק משתמש. המערכת פותחה תוך דגש על אילוצי זמן-אמת קשיחים (Real-Time Control), השהיה מינימלית (Ultra-Low Latency), ועלות חומרה נמוכה, במטרה להוות תשתית נגישה לשיקום וסיוע מוטורי.

<u>הרקע לפרויקט</u>

בעשורים האחרונים, עולם הרפואה והטכנולוגיה מתמודד עם אחד האתגרים המורכבים ביותר: השבת יכולת התנועה והתקשורת לאנשים הסובלים מפגיעות מוטוריות קשות, כגון שיתוק מוחין, פגיעות בעמוד השדרה, שבץ מוחי, או מחלות ניווניות סופניות כמו ALS (ניוון שרירים). במצבים אלו, קליפת המוח המוטורית (Motor Cortex) ממשיכה לתפקד, ליזום ולתכנן תנועות בצורה תקינה לחלוטין, אך חוטי המערכת – קרי, מערכת העצבים ההיקפית והשרירים – פגועים ואינם מסוגלים להוציא את הפקודה לפועל.

טכנולוגיית BCI מציעה מעקף טכנולוגי ישיר: קריאת הפעילות החשמלית הנוצרת במוח באמצעות אלקטרודות לא-פולשניות המוצמדות לקרקפת, עיבודה, ופענוחה לכדי פקודות בקרה דיגיטליות המפעילות מכשור חיצוני ללא תלות במערכת השרירים הפיזית. פרויקט זה מתמקד במרחב הנתונים של דמיון מוטורי (Motor Imagery) – מצב שבו המשתמש רק מדמיין שהוא מניע את ידו או רגלו, והמחשב מזהה כוונה זו ומפעיל דמות ממוחשבת בהתאם.

תהליך המחקר

תהליך הפיתוח נפתח במחקר מקדים מקיף ובעבודת שטח שבחנה את פתרונות הנגישות והשיקום הקיימים כיום בשוק ובמוסדות הרפואיים. במסגרת זו, נותחו טכנולוגיות עזר נפוצות כגון מערכות מעקב עיניים (Eye Tracking), מתגים מכניים המופעלים על ידי תנועות פנים, ומקלדות וירטואליות מבוססות מצמוץ וכו… .

מתוך עבודת השטח עלו מספר מסקנות ותובנות קריטיות:

- **עייפות קוגניטיבית ופיזית:** מערכות מבוססות מעקב עיניים או מצמוצים לאורך זמן גורמות למאמץ שרירי חריג בעיניים, ליובש ולעייפות קוגניטיבית מהירה, המגבילה את זמן השימוש הרציף.
- **תלות ביכולת מוטורית שיעורית:** מתגים מכניים דורשים מהמטופל לשמר לפחות יכולת תנועה מינימלית כלשהי כמו הזזת הלחי או הבוהן, פתרון שאינו רלוונטי לחולים בשלבים מתקדמים או משותקים לחלוטין.
- **עלויות חסרות תקדים:** פתרונות BCI קליניים קיימים מבוססים על ציוד רפואי יקר, הדורש מחשבי על, חדרים מבודדים אלקטרומגנטית, ומערך אלקטרודות פולשני או רטוב, מה שהופך אותם ללא-נגישים עבור חולה פרטי או מרכזים קהילתיים קטנים.

בעקבות ממצאים אלו, המחקר המקדים התמקד במציאת ארכיטקטורה חלופית המבוססת על חומרה מסחרית נגישה וזולה כמו שבב ה-32ESP ויצירת שרשרת אלגוריתמית חכמה מבוססת תוכנה המסוגלת לפצות על מגבלות הציוד הגולמי.

סקירת ספרות

הביסוס התיאורטי של פרויקט NeuroMotion נשען על שלושה דברים מדעיים מרכזיים:

א. פיזיולוגיה של המוח ודמיון מוטורי (Motor Imagery)

הניתוח הפיזיולוגי מבוסס על מערכת המיקום הבינלאומית 10-20 להצבת אלקטרודות על גבי הקרקפת. מחקרים מוכיחים כי בעת תכנון או דמיון של תנועה פיזית כגון תנועת יד ימין או יד שמאל, מתרחשת תופעה נוירולוגית המכונה ERD/ERS Event-Related Desynchronization / Synchronization בקליפת המוח המוטורית. תופעה זו מתבטאת בירידה או עלייה באנרגיה של תדרי המטרה הספציפיים: **גלי Mu (בטווח של 8-12 הרץ)** ו**גלי Beta (בטווח של 13-30 הרץ)** באזורים הממוקמים בעיקר מעל אלקטרודות **C3** ו-**C4** (ההמיספרה הימנית והשמאלית).

ב. עיבוד אותות מתקדם והפרדת מקורות (Signal Processing)

אות ה-EEG הנקלט הוא חלש מאוד. הוא בטווח של מיקרו-וולטים בודדים ומעורבב עם רעשים סביבתיים וביולוגיים קשים. הספרות המקצועית מציעה שילוב של מסנני תדר דיגיטליים לדוגמא Band-pass & Notch לסינון רשת החשמל הביתית של 50 הרץ בארץ) יחד עם אלגוריתמים מתקדמים להפרדת מקורות עיוורת (Blind Source Separation):

- **ICA (Independent Component Analysis):** אלגוריתם המפרק את האות הרב-ערוצי לרכיבים עצמאיים סטטיסטית, ובכך מאפשר לזהות ולאפס במדויק רעשי מצמוצי עיניים (EOG) ותנועות לסת (EMG), מבלי לפגוע באות המוחי הבסיסי.
- **CSP (Common Spatial Pattern):** מסנן מרחבי הממקסם את השונות של האות עבור מחלקה אחת (למשל, דמיון יד ימין) וממזער אותה עבור המחלקה השנייה (יד שמאל), מה שמקל על חילוץ המאפיינים הגיאומטריים של האות.

ג. רשתות עצבים עמוקות לנתוני EEG

בעוד שבעבר השתמשו במודלים קלאסיים כמו SVM או Random Forest, הספרות העדכנית מסמנת את רשת ה-**EEGNet** כפורצת דרך בתחום. זוהי רשת עצבים קונבולוציונית (CNN) קומפקטית ומותאמת במיוחד, תוך שמירה על מספר פרמטרים נמוך המאפשר עבודה בזמן אמת.

אתגרים מרכזיים

הבעיה איתה התמודד התלמיד

האתגר ההנדסי המרכזי בפרויקט היה התמודדות עם יחס אות לרעש נמוך במיוחד ותנאי סביבה דינמיים. אות ה-EEG הגולמי מעורבב לחלוטין ברעשי רקע כמו שדות אלקטרומגנטיים של מסכים וכבלי חשמל) וארטיפקטים אנושיים קשים. בנוסף, נדרש פתרון לבעיית ההשהיה ברשת: הזרמת כמות דגימות עצומה (קצב דגימה של 250 הרץ למספר ערוצים במקביל) על גבי תקשורת אלחוטית ללא איבוד מידע, תוך עמידה בחלון זמן נע הדורש עיבוד, ניקוי, וסיווג בתוך מהיר ביותר, כדי שהמשתמש לא יחווה נתק בין המחשבה לפעולת הרובוט.

הסיבות לבחירת הנושא

הבחירה בנושא זה נובעת משילוב של עניין אישי עמוק בעולמות הנוירו-טכנולוגיה (Neurotechnology) ומדעי המוח, יחד עם רצון לאתגר את גבולות היכולת ההנדסית בעולמות הפיתוח. פרויקט מסוג זה אינו פרויקט תוכנה סטנדרטי (כמו פיתוח אתר או מסד נתונים פשוט), אלא מערכת מולטי-דיסציפלינרית (רב-תחומית) המאלצת את המפתח לשלוט בחומרה ומיקרו-בקרים (Embedded C/C++), רשתות ותקשורת רשת נמוכה (UDP Sockets), מתמטיקה ואלגוריתמיקה מורכבת של עיבוד אותות ובינה מלאכותית, ופיתוח ממשקים מודרניים (React).

מוטיבציה לעבודה

המוטיבציה העיקרית מאחורי הפרויקט היא **חברתית ואנושית**. החזון הניצב בבסיס המערכת הוא הענקת עצמאות, ולו חלקית, לאוכלוסיית המשותקים וכלואי הגוף. היכולת להוכיח כי באמצעות חומרה זולה ונגישה, ושורות קוד שנכתבו בצורה חכמה ואופטימלית, ניתן לאפשר לאדם להניע עצמים בעולם האמיתי אך ורק באמצעות כוח המחשבה והתודעה שלו, מהווה מנוע דחיפה אדיר המעניק משמעות עמוקה לכל שלב בפיתוח.

על איזה צורך הפרויקט עונה

- **הצורך:** קיים פער עצום בין פתרונות תקשורת פשוטים ומעייפים (כמו מתגי לחיים) לבין מערכות BCI יקרות ומסורבלות שקיימות רק במעבדות מחקר אקדמיות מבודדות, מה שמשאיר אלפי פגועי מערכת עצבים ללא מענה נגיש בביתם.
- **הפתרון שהפרויקט נותן:** מערכת NeuroMotion מספקת פלטפורמה קלה, ניידת, זולה ולא-פולשנית. המערכת דוגמת את גלי המוח בצורה אלחוטית דרך ה-32ESP, מעבירה אותם בצינור תקשורת מבוסס UDP מהיר לשרת Python מרכזי שמנקה ומסווג את האות באמצעות בינה מלאכותית (EEGNet), ומציגה פלט תנועה מידי בממשק React תלת-ממדי. המערכת מציעה ארכיטקטורה פתוחה וגמישה שיכולה להתחבר בעתיד לכיסאות גלגלים ממונעים, פרוטזות רובוטיות או כלי כתיבה דיגיטליים.

הצגת הפתרונות לבעיה

כדי להבין את מקומה של מערכת **NeuroMotion** בשוק הטכנולוגי והרפואי הנוכחי, יש לבחון את הפתרונות הקיימים כיום בעולם להתמודדות עם פגיעות מוטוריות ושיתוק. פתרונות אלו נחלקים לשלוש קטגוריות מרכזיות: מערכות BCI פולשניות, מערכות BCI לא-פולשניות מסחריות, וטכנולוגיות עזר מסורתיות שאינן מבוססות על אותות מוח.

מערכות ממשק מוח-מחשב פולשניות (Invasive BCI): בחזית המחקר המדעי קיימות מערכות כדוגמת שבב ה-Utah Array (של חברת Blackrock Neurotech) או השבב המדובר של חברת Neuralink. מערכות אלו מבוססות על השתלה כירורגית של מערכי מיקרו-אלקטרודות ישירות לתוך קליפת המוח המוטורית. היתרון העצום של פתרון זה הוא איכות אות חסרת תקדים (SNR גבוה במיוחד) המאפשרת קריאה של ניורונים בודדים ושליטה ברזולוציה גבוהה על זרועות רובוטיות מורכבות או כתיבה מהירה על מסך. עם זאת, החסרונות של פתרון זה מונעים ממנו להיות נגיש לציבור הרחב; הוא דורש ניתוח מוח מורכב ומסוכן, כרוך בסיכונים זיהומיים קשים, דורש מעקב רפואי צמוד, ועלותו מגיעה למאות אלפי ואף מיליוני דולרים, מה שמותיר אותו כפתרון ניסיוני בלבד במעבדות קליניות מובחרות.

מערכות BCI לא-פולשניות רפואיות ומחקריות (Medical-Grade Non-Invasive BCI): חברות מובילות בעולם כגון g.tec הרפואית או מערכות BioSemi מציעות כובעי EEG לא-פולשניים המיועדים למחקר ושיקום. מערכות אלו משתמשות באלקטרודות "רטובות" הדורשות מריחת ג'ל מוליך על קרקפת המטופל ומספקות קריאה איכותית ויציבה של גלי המוח מבלי לסכן את המשתמש בניתוח. למרות בטיחותן, מערכות אלו סובלות מסרבול תפעולי משמעותי; תהליך הכנת המטופל, מריחת הג'ל וניקוי הכובע לאחר השימוש דורשים טכנאי מוסמך ואורכים זמן רב. בנוסף, עלות הציוד (הנע בין עשרות למאות אלפי

שקלים) ורישיונות התוכנה הקשיחים שלהן הופכים אותן לבלתי אפשריות לרכישה או לשימוש ביתי עצמאי עבור חולה פרטי או מרכזים קהילתיים קטנים.

מערכות BCI לא-פולשניות צרכניות (Consumer-Grade BCI): בשנים האחרונות צצו פתרונות נגישים וזולים יותר מחברות כמו Emotiv או OpenBCI, המציעות כובעים מבוססי אלקטרודות יבשות או חצי-יבשות במחירים נמוכים משמעותית מהציוד הרפואי. מערכות אלו פתחו את הפתח לפיתוח עצמאי, אך הן עדיין מציגות אתגרים הנדסיים מורכבים; רובן דורשות תשלום חודשי קבוע על תוכנות הפענוח, מוגבלות מבחינת רוחב הפס ותדירות הדגימה האלחוטית, ואינן כוללות פתרון מקצה לקצה (End-to-End) המשלב סביבת סימולציה שיקומית, נגישה וחלקה בדפדפן המותאמת ספציפית למשימות של דמיון מוטורי בזמן אמת.

טכנולוגיות עזר מסורתיות שאינן BCI (Traditional Assistive Tech): הפתרון הנפוץ ביותר כיום בבתי חולים ובבתים פרטיים מבוסס על מערכות מיקוד מבט (Eye Tracking) כדוגמת Tobii, או מתגים פיזיים המופעלים על ידי תנועת שריר שיעורית (כמו תנועת הלחי או הבוהן). מערכות אלו הן זולות ויציבות, אך חסרונן הגדול טמון בעייפות הפיזית והקוגניטיבית הקיצונית שהן מייצרות; שימוש ממושך במערכת מיקוד מבט גורם ליובש, כאבי ראש ומאמץ שרירי חריג בעיניים. יתרה מכך, חולים בשלבים מתקדמים ביותר של ניוון שרירים או פגועי ראש קשים מאבדים לעיתים גם את השליטה בשרירי העיניים ובשרירי הפנים, מה שמציב את הטכנולוגיות הללו כלא-רלוונטיות עבורם.

מטרות ויעדים:

המטרה הראשית:
להצליח לסווג את כוונת תנועת המשתמש ולהציג אותה.
יעדים:

-להצליח לבנות מודל מאומן שיעמוד על רמת דיוק גבוהה.

- לבנות מודל שאחראי על ניקוי רעשי רקע.
- להצליח ליצור ממשק משתמש גרפי יפה למשתמש.
- לאפשר תיקשורת בין הרכיבים השונים של המערכת.
- להצליח לחבר ולקרוא את הנתונים מהשבב.

אתגרים:

- להצליח לבנות את החומרה הנדרשת לפרויקט- כובע EEG לקליטת אותות מוח.
- לנסות להפעיל את שבב ה-ADS1299 שהוא זה אחראי על קליטת הנתונים מהאלקטרודות.
- למצוא מאגר נתונים סריקות מוח קודמות בכמות דגימות ובקצב תקין.
- למצוא אלגוריתם מתאים לניקוי רעשים - לקבלת אותות מוח נקיים ומוכנים לעיבוד.
- למצוא מודל מתאים לעיבוד הנתונים.
- להצליח לאמן את המודל ולהצליח להגיע אתו לרמת דיוק גבוהה ביותר ככל האפשר.

-להציג את הנתונים בצורה מדויקת על המסך .

מדדי הצלחה וכישלון למערכת:

- להצליח לקרוא נתונים מהשבב.

- לקבל אחוזי דיוק גבוהים לסיווג כוונת התנועה.

- מערכת יציבה, מהירה ואמינה מבחינת מהירות ונתונים.

-ממשק משתמש גרפי ברור ונוח לשימוש.

מדדי כישלון למערכת:

- אחוזי דיוק נמוכים.
- מערכת לא מהירה ועם שהיות וניתוקים.

סקירת ספרות:

מערכת BCI מבוססת EEG לזיהוי כוונת תנועה והצגתה בזמן אמת.

הפרויקט שלי עוסק בתחום ממשק מוח מחשב בשילוב חומרה- כובע EEG לקליטת אותות מוחיים בזמן אמת, המרת האותות לגלי מוח, שליחתם למחשב לעיבוד וניקוי מרעשים, הסקת כוונת התנועה ושליחתה למערכת המציגה למשתמש את כוונת התנועה על דמות.

מושגים עיקריים בתחום הדעת של הבעיה :

- אפידמיולוגיה (Epidemiology): חקר השכיחות, התפוצה והגורמים למצבים בריאותיים ופציעות באוכלוסיות שונות.

- פתופיזיולוגיה (Pathophysiology): חקר השיבושים בתפקוד המכני והביולוגי של הגוף כתוצאה ממחלה או פציעה.
- פנומנולוגיה (Phenomenology): חקר החוויה הסובייקטיבית; כיצד המטופל חווה את נכותו ואת אובדן השליטה ברמה הרגשית והאישית.
- הקורטקס המוטורי הראשוני (M1): האזור במוח האחראי על תכנון והוצאה לפועל של פקודות תנועה רצוניות.
- דמיון מוטורי (Motor Imagery - MI): תהליך קוגניטיבי שבו אדם מדמיין ביצוע תנועה פיזית מבלי להניע את השרירים, תוך הפעלת אזורים מוטוריים במוח.
- ERD/ERS: תופעות של ירידה (Desynchronization) או עלייה (Synchronization) בעוצמת תדרי מוח ספציפיים בעת דמיון תנועה.
- גירוי חשמלי פונקציונלי (FES): שימוש בזרמים חשמליים להפעלת שרירים משותקים לצורך שחזור תנועה.

מושגים בתחום מדעי המחשב ועיבוד אותות

- יחס אות לרעש (SNR): מדד לעוצמת האות המבוקש מול עוצמת הרעשים ברקע.
- ארטיפקטים (Artifacts): רעשים חיצוניים (רשת חשמל) או פנימיים (מצמוץ, שרירים) המזהמים את האות המוחי.
- חילוץ מאפיינים (Feature Extraction): תהליך מתמטי לבידוד המשתנים המובהקים ביותר מהאות הגולמי.
- זמן תגובה (Latency): העיכוב מרגע יצירת האות במוח ועד להצגת התוצאה הסופית.סקירה מקצועית של הבעיה בעולם

נושאים עיקריים בתחום הדעת של הבעיה :

כדי להבין כיצד המוח פועל וכיצד נוצרת תנועה, עלינו להבין שהכל מתחיל ברמה המיקרוסקופית ביותר – התא הבודד – ומסתיים בתנועה הפיזית של היד. המוח האנושי הוא למעשה הרשת התקשורתית והאלקטרוכימית המורכבת ביותר בטבע, המורכבת מכ-86 מיליארד תאי עצב (נוירונים).

להלן הפירוט המלא של התהליך, שלב אחר שלב:

מהו "החשמל" במוח וכיצד הוא נוצר?

המוח אינו מחובר למקור מתח חיצוני ואין בו אלקטרונים שזורמים כמו בחוטי נחושת של רשת החשמל הביתית. החשמל במוח הוא למעשה חשמל כימי. כל נוירון מוקף בקרום (ממברנה), ובתוכו ומחוצה לו שטים יונים חיוביים ושליליים (בעיקר נתרן, אשלגן, סידן וכלור). במצב מנוחה,

קיים מתח חשמלי זעיר בין פנים התא לחוץ התא. כאשר נוירון מקבל גירוי מספיק מנוירונים שכנים, נפתחות תעלות זעירות בקרום שלו, ויונים חיוביים (נתרן) שועטים פנימה. חדירה זו יוצרת היפוך מתח פתאומי ומהיר הנקרא פוטנציאל פעולה (Action Potential). זהו בעצם "הניצוץ" החשמלי. הפוטנציאל הזה מתקדם לאורך שלוחה ארוכה של התא שנקראת- אקסון, עד שהוא מגיע לקצהו (הסינפסה), שם הוא משחרר חומרים כימיים (נוירוטרנסמיטורים) שמעבירים את המסר לנוירון הבא.

גלי מוח ותדרים: השפה של הרשת העצבית

כאשר נוירון בודד שולח פוטנציאל פעולה, זהו אות חלש מאוד. אולם, כאשר מיליוני נוירונים שולחים יחד בסנכרון קצבי, הם מייצרים שדה חשמלי חזק מספיק שניתן למדוד אותו אפילו מחוץ לגולגולת (באמצעות מכשיר EEG). התנודות המחזוריות הללו של פעילות חשמלית מסונכרנת הן מה שאנו מכנים גלי מוח.

תדר (Frequency) נמדד בהרץ (Hz) ומתאר כמה פעמים בשנייה הגל הזה עולה ויורד.

למה המוח צריך את התדרים האלו? התדרים הם למעשה כמין פרוטוקולי התקשורת של המוח. אזורים שונים במוח צריכים לדבר זה עם זה ולתאם פעולות. בדיוק כפי שתחנות רדיו משדרות בתדרים שונים כדי לא להפריע זו לזו, המוח משתמש בתדרים נמוכים ואיטיים (כמו גלי דלתא בשינה עמוקה) לצורך סנכרון רחב ופעולות תחזוקה, ובתדרים מהירים (כמו גלי בטא וגמא) כשהוא נדרש לריכוז עצום, עיבוד מידע מורכב או ביצוע משימות קוגניטיביות מהירות.

יצירת הפקודה המוטורית: מהמחשבה אל התכנון

כעת, נבחן את הכרונולוגיה של תנועה – כיצד אדם מפעיל את ידו:

הכל מתחיל באזור הקדמי של המוח, הקורטקס הקדם-מצחי. זהו האזור האחראי על רצון, קבלת החלטות ואישיות. ברגע שנוצרת אצלך ההחלטה המודעת "אני רוצה להזיז את היד כדי להרים את הכוס", האזור הזה שולח אות לאזור סמוך הנקרא הקורטקס הקדם-מוטורי (Premotor Cortex). תפקידו של אזור זה הוא להיות האדריכל. הוא לא מזיז את השריר בעצמו, אלא בונה את תוכנית הפעולה: אילו שרירים צריכים להתכווץ, באיזה סדר, ובאיזו עוצמה, תוך התחשבות במידע שמגיע מהעיניים (איפה הכוס נמצאת) ומהגוף (באיזו זווית היד נמצאת כרגע).

הוצאה לפועל- מהמוח אל עמוד השדרה

ברגע שתוכנית הפעולה מוכנה, היא מועברת אל הקורטקס המוטורי הראשוני (M1). זהו המפעיל של התנועה. אזור זה מסודר כמפה של הגוף (ההומונקולוס המוטורי), כך שיש בו קבוצת נוירונים ספציפית שאחראית אך ורק על כף היד והאצבעות.

ברגע שהפקודה ניתנת, הנוירונים באזור היד ב-M1 יורים שרשרת אדירה של פוטנציאלי פעולה. האות החשמלי מתחיל לטוס במהירות של עד 120 מטר בשנייה לאורך האקסונים

שלהם, היורדים מטה אל תוך גזע המוח. שם, באופן מרתק, רוב הסיבים מצטלבים ועוברים לצד השני של הגוף (לכן צד שמאל של המוח שולט ביד ימין, ולהפך). משם, האות החשמלי ממשיך לרדת לאורך חוט השדרה (Spinal Cord), שהוא למעשה הכבל הראשי של הגוף דרכו הכל עובר!.

התחנה הסופית: הפעלת השריר

האות החשמלי מגיע לגובה המתאים בחוט השדרה (באזור הצוואר, כאשר מדובר ביד), ושם הוא עובר סינפסה – הוא מוסר את המידע לנוירון חדש הנקרא נוירון מוטורי תחתון. נוירון זה יוצא מחוט השדרה ומרכיב את העצב הפריפרי (ההיקפי) שנוסע כל הדרך לאורך הזרוע עד שהוא מגיע לשרירי האצבעות.

הקצה של העצב מתחבר ישירות לסיבי השריר באזור שנקרא צומת עצב-שריר. כאשר האות החשמלי מגיע לשם, הוא גורם לשחרור של נוירוטרנסמיטר בשם אצטילכולין. החומר הכימי הזה נקשר לקרום של תאי השריר, גורם להם לייצר פוטנציאל פעולה משלהם, ומתניע מנגנון מכני של החלקת חלבונים זה על גבי זה. התוצאה של כל התהליך הזה – סיבי השריר מתקצרים ומתכווצים, היד זזה, והכוס מורמת.

כל השרשרת הכרונולוגית הזו – מהחלטה כימית-חשמלית בקידמת המוח, דרך גלי המוח המסונכרנים, הירידה בחוט השדרה וכיווץ השריר – מתרחשת בשבריר שנייה ומתבצעת בסנכרון מושלם.

אבל עם זה כל כך מושלם מה הבעיה?

הבעיה בארץ ובעולם

1. האפידמיולוגיה (השכיחות) הגלובלית וההקשר הלאומי של אובדן התנועה-

אובדן זמני או קבוע של יכולת התנועה עקב קטיעות גפיים ופגיעות בעמוד השדרה מהווה את אחד האתגרים הבריאותיים, החברתיים והכלכליים הקשים ביותר הניצבים בפני הרפואה המודרנית. ארגון הבריאות העולמי מעריך כי ברחבי העולם חיים מיליוני בני אדם המתמודדים עם מגבלות תנועה חמורות כתוצאה מטראומה פיזית.

ברמה הגלובלית, הגורמים נחלקים בצורה ברורה בין מדינות מערביות מפותחות לבין אזורי מתפתחים או אזורי עימות:

- **בעולם המערבי והמזדקן:** רוב קטיעות הגפיים נובעות ממחלות כרוניות ומחלות כלי דם פריפריאליות, ובראשן סיבוכי סוכרת. פגיעות עמוד שדרה נובעות בעיקר מתאונות דרכים ונפילות מגובה באוכלוסייה מבוגרת.
- **באזורי עימות וקונפליקטים גאופוליטיים:** המציאות משתנה לחלוטין. השימוש המסיבי בטכנולוגיות חבלה מודרניות – מטעני הדף, ארטילריה, רחפנים מתאבדים וטילים – יוצר גל נרחב של פגיעות טראומטיות קשות באוכלוסייה צעירה, יצרנית ובריאה בבסיסה.

בישראל, השנים האחרונות והעלייה החדה בהיקף המלחמות והפעילויות הביטחוניות הביאו להחמרה בהיקף הפצועים. חיילים ואזרחים רבים מוצאים את עצמם בעקבות פגיעות הדף, רסיסים וירי חודר עם פגיעות עמוד שדרה מורכבות וקטיעות גפיים (עליונות ותחתונות כאחד). מציאות זו מייצרת דרישה דחופה, בקנה מידה לאומי, למערכות שיקום טכנולוגיות שיוכלו להחזיר לנפגעים את האוטונומיה הבסיסית שלהם.

הפתופיזיולוגיה של הנתק המוטורי

הבעיה העמוקה ביותר בפגיעות אלו אינה טמונה במוח עצמו. אצל רובם המכריע של הפצועים, קליפת המוח המוטורית הראשונית (1M) מתפקדת בצורה מושלמת. המוח מסוגל להגות את רעיון התנועה, לתכנן אותה במדויק ולשלוח את פוטנציאלי הפעולה החשמליים הנדרשים. הכשל הוא כשל תחבורתי-מכני:

- **בפגיעות עמוד שדרה:** האות החשמלי שנוצר במוח נע מטה לאורך המסלול הקורטיקו-ספינאלי, אך נתקל בנתק פיזי- חלל או רקמה צלקתית חסימ) בעמוד השדרה. הפקודה אינה יכולה להגיע אל מערכת העצבים ההיקפית ואל השרירים, וכתוצאה מכך האיברים נותרים משותקים ומתנוונים, למרות שהפקודה המקורית נשלחה.
- **בקטיעות איברים:** האות העצבי מצליח לעבור את עמוד השדרה ומגיע עד לקצה הגדם, אך הוא אינו מוצא את איבר הקצה (היד או הרגל שהוסרו). האות מתפזר ברקמה, יוצר לעיתים קרובות צמיחה פתולוגית של סיבי עצב ומחולל כאבים קשים.

התוצאה היא מצב קיומי מייאש שבו המטופל חווה דיסוננס חריף בין מוח חי ויוצר לבין גוף שאינו מגיב עוד, דבר המוביל באחוזים גבוהים מאוד לדיכאון קליני, חרדה קיומית ותסמונת פוסט-טראומטית, לצד נטל כלכלי עצום המגיע למיליוני דולרים לכל ימי חייו של המטופל.

מגבלות הפתרונות הקיימים-

1. **פרוטזות מיו-אלקטריות קונבנציונליות:** נסמכות על קריאת אותות חשמליים משרירי הגדם שנותרו בריאים-EMG. פתרון זה אינו רלוונטי לפגועי עמוד שדרה שחווים שיתוק מוחלט. עבור קטועי גפיים, הוא דורש מאמץ קוגניטיבי מתיש ולא טבעי למשל, כיווץ שריר בזרוע כדי לסגור כף יד, והאותות משתבשים בקלות מזיעה או מתזוזת המכשיר.
2. **ממשקי BCI פולשניים:** השתלת שבבים ישירות לתוך רקמת המוח בניתוחים כירורגיים מורכבים. גישה זו כרוכה בסכנות חיים (זיהומים, דימומים מוחיים), עלויות עתק, ותופעה שבה הגוף מזהה את השבב כגוף זר, מייצר סביבו צלקת ומבודד אותו עד שהאות דועך לחלוטין תוך שנים ספורות.
3. **תהליכי השיקום המסורתיים:** פיזיותרפיה וריפוי בעיסוק דורשים חודשים ושנים של עבודה מייגעת. עבור פצועים רבים, חוסר היכולת לראות תוצאות מידיות והצורך להתרגל למצב החדש במקום לשחזר את התנועה הטבעית גורמים לתסכול עמוק ואחוזי נטישה גבוהים במיוחד של עזרי השיקום.

פער מחקרי נוסף בתחום ה-BCI הלא פולשני הוא בעיית הניידות-
פער מחקרי נוסף בתחום ה-BCI הלא-פולשני הוא התלות במעבדות מבוקרות. מרבית המערכות המחקריות דורשות סביבה מבודדת מרעשים אלקטרומגנטיים, ושימוש

באלקטרודות רטובות הדורשות מריחת ג'ל מוליך המגבילה את זמן השימוש (בשל התייבשות הג'ל) ומחייבת חפיפת ראש בתום התהליך.

המערכת המפותחת בפרויקט זה פותרת פער זה על ידי מעבר לארכיטקטורה ניידת לחלוטין. השילוב של אלקטרודות מסרק יבשות (המבטלות את הצורך בג'ל) יחד עם סינון דיגיטלי מתקדם בזמן אמת ומערכת שידור אלחוטית מבוססת סוללה, מאפשר להוציא את הטכנולוגיה מכותלי המעבדה הסטרילית ולהנגיש אותה ככלי שיקומי ביתי ויום-יומי עבור המטופל.

הרקע הקליני והצורך הטכנולוגי

אובדן יכולת תנועה, בין אם בשל מחלות כרוניות, פגיעות עמוד שדרה או קטיעות איברים, מציב אתגרים כלכליים וחברתיים עצומים. הבעיה המרכזית אינה במוח – האזור המוטורי (M1)מתפקד כראוי ומייצר פקודות חשמליות – אלא בנתק מכני במסלול העצבי. הפתרונות הקיימים כיום (פרוטזות מיו-אלקטריות או שתלים פולשניים) מוגבלים על ידי פולשנות רפואית, חוסר נוחות, או דעיכת אותות עם הזמן. קיים צורך אקוטי במערכות ניידות ובלתי פולשניות שיאפשרו שיקום עצמאי.

הבסיס הנוירולוגי: כיצד המוח מייצר אותות

הפעילות החשמלית במוח נוצרת כתוצאה משינויים במתח החשמלי על פני קרומי תאי העצב (נוירונים). כאשר תאי עצב פועלים יחד בסנכרון, הם מייצרים פוטנציאליים פוסט-סינפטיים מעוררים (EPSP) ומעכבים .(IPSP) סכום הפוטנציאלים הללו יוצר דיפולים חשמליים שניתן למדוד על פני הקרקפת באמצעות אלקטרודות.(EEG)

- **שיטת 10-20 :**שיטה סטנדרטית למיקום אלקטרודות המבטיחה כיסוי האזורים המוטוריים.(C3, C4, Cz)
- **מקצבים מוחיים :**הפעילות נמדדת בתדרים שונים. גלי מיו 8-12 Hz וגלי בטא 13-30 Hzמשנים את עוצמתם באופן מובהק בזמן דמיון תנועה. תופעת ה ERD-מתרחשת כאשר יש ירידה באמפליטודת הגלים (סימן לפעילות מוגברת באזור המוחי), בעוד ה-ERSמסמן את חזרת האזור לרגיעה.

מבוא לממשקי מוח מחשב- BCI - Brain-Computer Interfaces

ממשק מוח-מחשב הוא ערוץ תקשורת ישיר בין המוח האנושי לבין מכשיר חיצוני, העוקף את מסלולי העצבים והשרירים הרגילים של הגוף. המטרה המרכזית של מערכות אלו היא לתרגם פעילות חשמלית של תאי עצב לפקודות דיגיטליות.

מערכות BCI מתחלקות לשלוש קטגוריות עיקריות:

- **מערכות פולשניות-** מציעות איכות אות גבוהה אך כרוכות בסיכון רפואי. לדוגמא: אלקטרודות המושתלות בניתוח ישירות לתוך קליפת המוח.
- **מערכות פולשניות חלקית-** איכות האות גבוהה יחסית אבל גם כאן יש סיכון-מופחת. לדוגמא: אלקטרודות המונחות על קרום המוח.
- **מערכות לא פולשניות:** שיטה זו, ובראשה ה- **EEG**שבמונחים רפואיים נקרא גם אלקטרואנצפלוגרפיה, היא הנפוצה והבטוחה ביותר לשימוש מסחרי ומחקרי, והיא הבסיס לפרויקט זה בשל זמינותה, בטיחותה וקלות התפעול שלה. לדוגמא: חיישנים המוצבים על הקרקפת.

הבסיס הנוירולוגי: כיצד המוח מתכנן ומדמיין תנועה-

כדי לזהות כוונת תנועה ללא ביצוע פיזי שלה, נשענים על תופעה קוגניטיבית הנקראת **דמיון מוטורי Motor Imagery - MI**. כאשר אדם מדמיין שהוא מניע איבר בגופו (למשל, יד ימין), המוח מפעיל את אותם האזורים שהיו פועלים בעת ביצוע התנועה בפועל.

האזורים הרלוונטיים במוח:

- **הקורטקס המוטורי הראשוני-** ממוקם באונה הפרונטלית (רצועה מרכזית במוח) ואחראי על תכנון והוצאה לפועל של תנועות.
- **ההומונקולוס המוטורי:** מיפוי נוירולוגי שבו אזורים שונים בקורטקס המוטורי שולטים על איברים שונים בגוף (ידיים, רגליים, פנים). אזור הידיים, למשל, הוא גדול ובעל ייצוג נרחב, מה שמקל על זיהוי האות.

תופעות פיזיולוגיות באות ה-EEG:

כאשר המוח נכנס למצב של דמיון מוטורי, חלים שינויים במקצבים החשמליים באזורים המוטוריים, במיוחד באלקטרודות במיקומים של: C3, C4 ו-Cz לפי שיטת ה-10-20 הבינלאומית:

- **ERD- Event Related Desynchronization -הסבר:** ירידה בעוצמת התדרים של גלי מיו (Mu 8-12 Hz) וגלי בטא (Beta 13-30 Hz) בצד הנגדי לאיבר המדומיין. לדוגמה: דמיון תנועת יד ימין יגרום ל-ERD בחצי הכדור השמאלי של המוח - אלקטרודה במיקום C3.
- **ERS- EventRelated Synchronization -הסבר:** עליה בעוצמת הגלים (לרוב גלי בטא) מיד לאחר סיום הדמיון או באזורים שאינם פעילים.

נושאים רלוונטים במדעי המחשב בהקשר לבעיה

קליטת אותות, סינון ועיבוד מידע.

אות ה-EEG הנקלט מהקרקפת הוא חלש מאוד- בטווח של מיקרו-וולטים ורועש במיוחד. לכן, המערכת נדרשת לעבור מספר שלבים קריטיים:

א. קדם עיבוד וסינון רעשים-

האותות מושפעים מרעשים סביבתיים כגון: רשת החשמל הביתית - Hz50(בארץ) ומרעשים ביולוגיים לדוגמא: מצמוצי עיניים, תנועות לסת, דופק... .

דוגמאות לסינון:

- שימוש בסינונים אקטיביים כמו **Band-pass filter-** חיתוך בין 8 ל-30 הרץ כדי להתמקד בגלי Mu ו-Beta.
- מסנני Notch להסרת תדר רשת החשמל ידוע שהוא 50 Hz בארץ.
- אלגוריתמים מתקדמים כמו ICA להפרדת רעשי מצמוצים מאות המוח הנקי.

כדי להתמודד עם המורכבות של אותות ה-EEG והיחס הנמוך בין האות לרעש (SNR), תורת עיבוד האותות בעולמות ה-BCI נשענת על שני אלגוריתמים מרכזיים המשלימים את שלב סינון התדרים הסטנדרטי:

1. אלגוריתם ICA Independent Component Analysis – ניתוח רכיבים עצמאיים

אלגוריתם ICA מבוסס על מתודולוגיה של הפרדת מקורות עיוורת (Blind Source Separation - BSS), ומטרתו העיקרית היא לסדר את האותות מכיוון שכל אלקטרודת EEG קולטת תערובת ליניארית של פעימות חשמליות ממקורות קוגניטיביים שונים במקביל לארטיפקטים פיזיולוגיים (כגון מצמוצי עיניים או פעילות שרירי הלסת), ה-ICA מפרק את האות הרב-ערוצי לרכיבים זמניים שהם בלתי תלויים סטטיסטית זה בזה.

האלגוריתם מחשב מטריצת הפרדה (Unmixing Matrix) המבודדת את זרם הנתונים לרכיבים עצמאיים (ICs). רכיבים המייצגים רעש ביולוגי מזוהים בקלות יחסית בשל המאפיינים הייחודיים שלהם (לדוגמה, אמפליטודה חריגה באלקטרודות הקדמיות בעת מצמוץ). באמצעות איפוס מתמטי של רכיבי הרעש הללו וביצוע התמרה הפוכה (Inverse ICA), המערכת מסוגלת לשחזר את אות ה-EEG המקורי כשהוא נקי ומטוהר מהארטיפקטים שנבחרו, מבלי לפגוע או לעוות את תדרי המטרה הקוגניטיביים הנמצאים באותן נקודות זמן.

2. אלגוריתם CSP Common Spatial Pattern – תבנית מרחבית משותפת

בשונה ממסננים אנלוגיים או דיגיטליים הפועלים במישור התדר או הזמן, אלגוריתם CSP הוא מסנן מרחבי (Spatial Filter) מתקדם, הנחשב לאחד הכלים החזקים ביותר בעיבוד אותות עבור משימות דמיון מוטורי (Motor Imagery). מתודולוגיה זו מבוססת על אלגברה ליניארית ומיועדת לעבודה עם מערכות דו-מחלקתיות (כגון הבחנה בין דמיון תנועת יד ימין לעומת יד שמאל). המטרה המרכזית של CSP היא למקסם את השונות (Variance) של האות המסונן עבור מחלקה אחת, ובמקביל למזער ככל הניתן את השונות עבור המחלקה השנייה.
האלגוריתם מנתח את מטריצות השונות המשותפת (Covariance Matrices) של ערוצי ה-EEG השונים שנמדדו בשלב הכיול, ופותר בעיית ערכים עצמיים משותפת (Generalized Eigenvalue Problem). כתוצאה מכך, מתקבלים וקטורי משקל מרחביים המשמשים כמסננים. כאשר מפעילים את המסננים הללו על אות ה-EEG הגולמי, המערכת מציגה הפרדה גאומטרית מיטבית בין תתי-האזורים המוחיים הרלוונטיים (כמו קליפת המוח המוטורית באזור

ה-SMC). תהליך זה מנקה אקטיבית פעילות מוחית שאינה קשורה למשימה, ומחלץ מאפיינים מרחביים מזוקקים (Features) המאפשרים לשלבי הלמידה החישובית והסיווג להגיע לאחוזי דיוק גבוהים בהרבה.

סיווג ולמידה חישובית

לאחר חילוץ המאפיינים, אלגוריתם של למידה חישובית ML נדרש להכריע למה התכוון המשתמש:

- מודלים קלאסיים כמו **LDA** או **SVM** נפוצים מאוד בזכות מהירותם ויעילותם בזמן אמת.
- בשנים האחרונות ישנה מגמה של מעבר לרשתות עצביות עמוקות כמו **EEGNet** שהוא רשת קונבולוציה - CNN המותאמת לאותות EEG המשלבת חילוץ מאפיינים וסיווג בשלב אחד.

הרחבה על ארכיטקטורת רשת ה-EEGNet

ארכיטקטורת ה-EEGNet מהווה פריצת דרך בספרות המקצועית של ה-BCI, שכן היא גושרת על הפער שבין רשתות CNN גנריות לבין האופי הייחודי של אותות EEG – שהם אותות סדרתיים בזמן בעלי ממד מרחבי.

עקרונות התכנון: קומפקטיות ופתרון בעיית ה-Overfitting

בניגוד לרשתות עמוקות כבדות (כמו ResNet או VGG), ה-EEGNet תוכננה להיות קומפקטית. האתגר המרכזי ב-EEG הוא מחסור בנתונים (Few-Shot) ביחס לכמות הפרמטרים במודל. רשת גדולה מדי תבצע שינון-Overfitting של הרעשים במקום ללמוד את התבנית המוטורית. ה-EEGNet פותרת זאת באמצעות מספר מינימלי של שכבות, תוך שימוש בטכניקות קונבולוציה חכמות.

פירוק השכבות וההיגיון המתמטי

הרשת פועלת ברצף של שלוש פעולות קונבולוציה המדמות את תהליך עיבוד האותות הקלאסי:

Temporal Convolution (קונבולוציה בזמן): השכבה הראשונה מבצעת סינון (Filtering) לאורך ציר הזמן. פעולה זו למעשה לומדת את פונקציות ה-Band-pass filter באופן עצמאי. במקום להגדיר ידנית תדרים, הרשת לומדת אילו תדרים (כמו Hz8-12 או Hz13-30) הם הכי אינפורמטיביים לסיווג התנועה.

Depthwise Convolution (קונבולוציה עמוקה): כאן הרשת מבצעת פילטר מרחבי (Spatial Filtering). היא מחשבת שילובים של אלקטרודות שונות כדי להדגיש את האזור במוח שממנו מגיע האות הכי חזק. זוהי למעשה הגרסה הדינמית והלומדת של אלגוריתם ה-CSP.

Separable Convolution (קונבולוציה נפרדת - Pointwise): זו היחודיות של הרשת. במקום לבצע קונבולוציה רגילה שהיא יקרה מבחינה חישובית היא מחלקת אותה לשניים:

Depthwise: עיבוד ערוצים בנפרד.

Pointwise: שילוב המידע מהערוצים השונים.

התוצאה: הפחתה דרמטית (של עד 90%) בכמות הפרמטרים, מה שמאפשר למודל להתכנס לתוצאה מדויקת גם עם מעט דגימות אימון, ומהירות אינפרנס (Inference) שמתאימה לזמן אמת.

כיוון שהרשת עושה שימוש בקונבולוציות עומק נפרדות, כמות הפרמטרים שלה קטנה ב-90% מרשתות CNN רגילות, מה שמאפשר לתוכנת האינפרנס להריץ את המודל בזמן קבוע ומהיר ללא יצירת השהיות במערכת.

מערכות משובצות מחשב, פרוטוקולים ותקשורת

- **ניהול פרוטוקולי תקשורת מהירים:** מימוש תקשורת **SPI**- Serial Peripheral Interface בין מעבד ה-32ESP לשבב ה-1299ADS, כולל שליחת פקודות בקרה קריטיות.
- **ניהול פסיקות חומרה:** הגדרת פין ה-Data Ready-DRDY של השבב כפסיקת חומרה בבקר, המבטיחה שליפת חבילות מידע בדיוק ברגע סיום ההמרה, ללא איבוד דגימות.
- **תקשורת אלחוטית ופרוטוקולי שידור:** שימוש ברכיבי ה-**Wi-Fi** או ה-**Bluetooth** המובנים בבקר לצורך הזרמת נתונים גולמיים (Streaming) באופן אלחוטי ומאובטח למחשב המארח.

אתגר השהות

כדי שהמשתמש ירגיש שהוא שולט על התנועה של הדמות במסך, העיבוד וההצגה על המסך חייבים להתבצע בשהות מינימלית שלא תורגש על ידי המשתמש. במיוחד שהמערכת עובדת בכמה שלבים: קודם כל קליטת האותות מהמוח באמצעות אלקטרודות, דרך השבב לבקר, מהבקר לעיבוד האותות ושליחתם לתצוגת משתמש גרפית.

פער מחקרי וחיבור לפרויקט הנוכחי

מרבית המחקרים בתחום ה-BCI מתמקדים בסיווג אופליין- ניתוח האותות לאחר מעשה או במתן משוב דו ממדי פשוט מאוד. הפער הקיים הוא ביצירת מערכת יציבה, חסינת רעשים, שמסוגלת לבצע זיהוי בזמן אמת ולתרגם אותו לתנועה חלקה וטבעית של דמות ממוחשבת, תוך שמירה על כיול מהיר שמתאים למשתמשים שונים.

הפרויקט שלי בא לענות על פער זה על ידי פיתוח ארכיטקטורה מקצה לקצה המשלבת חומרה נגישה (כובע EEG), עיבוד אותות מהיר, וממשק משתמש אינטראקטיבי.

תיאור פתרונות קיימים לבעיה:

הפתרונות והטכנולוגיות המרכזיים הקיימים כיום בשוק ובתחום המחקר להתמודדות עם קטיעות גפיים ופגיעות מוטוריות:

1. פרוטזות מיו-אלקטריות:

מערכות אלו הן תותבות אקטיביות לגפיים עליונות או תחתונות, המופעלות באמצעות אותות חשמליים הנוצרים בשרירים הפריפריאליים של המשתמש.

- **מנגנון הפעלה:** חיישני EMG (אלקטרומיוגרפיה) שטחיים מוצמדים לעור בתוך תושבת הפרוטזה . כאשר המשתמש מכווץ שריר מסוים שנותר בגדם, החיישן קולט את המתח החשמלי, ומנועים זעירים בפרוטזה מתרגמים זאת לתנועה של פתיחה, סגירה או סיבוב.

2. שלדים חיצוניים רובוטיים:

מדובר במבנים מכניים לבישים, המורכבים על גופו של המטופל-לרוב על פלג הגוף התחתון והרגליים, ומספקים כוח מניע חיצוני למפרקים.

- **מנגנון הפעלה:** המערכות כוללות מנועים חשמליים, סוללות וחיישני תנועה (כמו חיישני הטיה או מנח גוף). התנועה מופעלת ומנוהלת על ידי זיהוי שינויים במרכז הכובד של המשתמש, או באמצעות שלט רחוק המורכב על קביים או על ידי המטפל.

3. ממשקי מוח מחשב BCI פולשניים:

טכנולוגיות הנמצאות בשלבי פיתוח מתקדמים וניסויים קליניים, המבוססות על יצירת קשר ישיר בין הרקמה המוחית לבין אמצעי מחשוב חיצוניים באמצעות הליך כירורגי.

- **מנגנון הפעלה:** מערכי אלקטרודות זעירים מושתלים ישירות בתוך קליפת המוח המוטורית (או בתוך כלי דם מוחיים קרובים). האלקטרודות קולטות את פוטנציאלי הפעולה של נוירונים בודדים בעת תכנון תנועה, והנתונים נשלחים דרך חיווט או משדר אלחוטי אל מחשב שמפעיל זרוע רובוטית או סמן על מסך.

4. ממשקי מוח מחשב BCI לא פולשניים:

מערכות לקריאת פעילות מוחית הפועלות לחלוטין מחוץ לגוף, ללא צורך בהתערבות כירורגית, ומשמשות לצרכי מחקר, שיקום או שליטה בסיסית במחשב.

- **מנגנון הפעלה:** המשתמש לובש כובע או קשת המכילים חיישני EEG (אלקטרואנצפלוגרפיה) יבשים או רטובים. החיישנים קולטים את גלי המוח החלשים העוברים דרך הגולגולת, ומערכת ממוחשבת מנתחת את שינויי התדרים כדי לזהות דפוסי קשב, הרפיה או דמיון מוטורי.

5. מערכות גירוי חשמלי פונקציונלי FES - Functional Electrical Stimulation:

מכשירים טכנולוגיים המיועדים להפעיל באופן ישיר את שרירי הגוף באמצעות זרם חשמלי חיצוני, ללא מעבר האות דרך מערכת העצבים המרכזית.

- **מנגנון הפעלה:** אלקטרודות מדבקה מוצמדות לעור מעל קבוצות שרירים משותקות או חלשות (כמו שרירי השוק או השרוול המסובב). המכשיר שולח פולסים חשמליים בתדר ובמתח מבוקרים, הגורמים לסיבי השריר להתכווץ ולבצע פעולה פונקציונלית קבועה מראש, כגון הרמת כף הרגל בעת צעידה.

ניתוח חלופות מערכתי

חלופת מבחינת הקוד- מבנה הנתונים לשידור המידע:

תת-הבעיה הראשונה עוסקת בארכיטקטורת מבנה הנתונים המשמש לשידור וסנכרון האות החשמלי ברשת. המערכת נדרשת להזרים קצב נתונים קבוע וגבוה של 250 דגימות בשנייה מרכיב הקצה (32ESP) אל שרת העיבוד, ומאחר ופרוטוקול התקשורת שנבחר הוא אסינכרוני, קיים סיכון ממשי לאובדן חבילות, הגעה של חבילות מחוץ לסדר, או עיוות ברצף האות החשמלי, ולכן נדרש פתרון שיבטיח בקרה ושיחזור של האות בצד המקבל. במסגרת זו נבחנו שתי חלופות מרכזיות.

החלופה הראשונה מציעה שליחה של מידע כמערך נתונים פשוט המכיל רק את ערכי המתח הגולמיים של ה-EEG, פתרון שהוא קל מאוד למימוש ובעל תקורה אפסית, אך הוא חסר כל יכולת בקרה, סנכרון או זיהוי אובדן מידע ברשת ועלול להוביל לחוסר סנכרון מוחלט.

החלופה השנייה מציעה בניית מבנה נתונים ייעודי ומותאם אישית מסוג Struct האורז בתוכו כל חבילת מידע בצורה חכמה, הכולל תווי סנכרון ייעודיים בתחילת החבילה למניעת הסטת בתים בבאפר, חותמת זמן מדויקת לניטור זמן השהות, ומונה חבילות רציף המאפשר לעקוב אחר סדר ההגעה.

בחינת החלופות לפרוטוקול תקשורת הרשת:

תת-הבעיה השנייה מתרכזת בבחירת פרוטוקול תקשורת הרשת המתאים להזרמת הנתונים מהחומרה לשרת. העברת אותות במערכות ממשק מוח-מחשב דורשת זמן תגובה מינימלי ככל הניתן בין רגע המחשבה במוח לבין תנועת הדמות על המסך, שכן כל עיכוב ברשת פועל לרעה ומקלקל באופן ישיר את לולאת המשוב הניורו-קוגניטיבית של המשתמש.

החלופה הראשונה שנבחנה היא פרוטוקול TCP, אשר מציע אמינות של מאה אחוזים ומבטיח הגעה סדרתית ותקינה של כל חבילת מידע באמצעות מנגנון אישורים מובנה, אך מנגנון השילוח החוזר שלו ולחיצת היד המשולשת מייצרים השהיות כבדות ומשתנות בכל פעם שחבילה אחת הולכת לאיבוד, דבר הגורם לתקיעות רשת קשות.

החלופה השנייה שנבחנה היא פרוטוקול UDP, המהווה פרוטוקול מהיר במיוחד ללא יצירת קשר קבוע וללא תקורה רשתית כבדה של כותרות ארוכות, המאפשר להזרים את חבילות המידע במהירות מקסימלית אך ללא התחייבות מובנית להגעתן או לסדרן בצד המקבל.

בחינת חלופות לתשתית וספריית למידה עמוקה

תת-הבעיה השלישית עוסקת בבחירת תשתית וספריית הלמידה העמוקה לצורך פיתוח, אימון והרצת רשת העצבים הקונבולוציונית הייעודית EEGNet. סביבת העבודה נדרשת לתמוך במניפולציות מורכבות של מטריצות רב-ממדיות ולאפשר הרצה מהירה ויציבה של המודל בזמן אמת ללא יצירת עומס חישובי או השהיות שיעכבו את מערכת המשוב הויזואלית.

החלופה הראשונה שנבחנה היא ספריית TensorFlow ו-Keras מבית גוגל, המהווה פתרון תעשייתי יציב, קשיח ונפוץ מאוד בשוק, אך היא מבוססת על גרף חישוב סטטי שהופך את פיתוח וניקוי השגיאות של ארכיטקטורות רשת לא קלאסיות למסורבל וכבד יותר למחקרי BCI בהיקף כזה.

החלופה השנייה שנבחנה היא ספריית PyTorch מבית Meta, המציעה סביבה גמישה להפליא המבוססת על קוד פייתונאי טבעי וגרף חישוב דינמי, המאפשרים בנייה, אימון ושינוי מהיר של ארכיטקטורות מורכבות במהלך שלבי המחקר.

הבחירה הסופית ונימוק

לאחר שקלול היתרונות והחסרונות של שתי הגישות, החלופה שנבחרה ליישום בפרויקט היא חלופה ב', המבוססת על מבנה הנתונים המותאם (Struct). הבחירה נבעה מהצורך הקשיח באמינות ובקרה בעיבוד אותות רפואיים בזמן אמת, אשר אינו מאפשר הסתמכות על מערך פשוט. השימוש במבנה הנתונים הייעודי, שנקרא EEG_Raw_Frame, מאפשר לשרת ה-Python לזהות מייד באמצעות המונה ורכיבי הסנכרון הפנימיים אם חבילה אבדה בדרך או הגיעה מחוץ לסדר. פתרון תוכנתי זה מפצה לחלוטין על חסרונות הרשת האסינכרונית ומאפשר לבצע השלמה או סידור מחדש של המידע, דבר המבטיח שרטוט גרף אמין ורציף עבור רשת העצבים ללא עיוותי תדר.

החלופה שנבחרה לפרוטוקול תקשורת הרשת

לאחר בחינת אילוצי המערכת, הוחלט לבחור בחלופה ב', שהיא פרוטוקול UDP. הבחירה ההנדסית הזו מבוססת על ההבנה שבמערכות זמן אמת מוטוריות, מהירות התגובה והקטנת זמן השהות הן הקריטריונים הקריטיים ביותר להצלחת המשוב הויזואלי. מאחר שפרוטוקול TCP עלול לייצר השהיות של מאות מילי-שניות שאינן קבועות ופוסלות אותו לחלוטין מהפעלה תנועתית, העדפנו את המהירות המקסימלית של UDP. אובדן החבילות הזניח של חלקיקי אחוזים ברשת מקומית מנוהל ומטופל בצורה חכמה וממוקדת ברמת התוכנה,

כפי שהוגדר באמצעות מבנה הנתונים הייעודי שנבחר בסעיף הקודם, ובכך אנו מרוויחים גם אפס השהייה וגם שמירה על יציבות האות.

בחינת חלופות לתשתית וספריית למידה עמוקה

לאחר ניתוח הצרכים המחקריים והפיתוחיים של הפרויקט, החלופה שנבחרה לבסוף היא חלופה ב', ספריית PyTorch. הבחירה ב-PyTorch מעניקה יתרון הנדסי מכריע מכיוון שרשת ה-EEGNet עושה שימוש בקונבולוציות עומק נפרדות שאינן סטנדרטיות ברשתות ראייה ממוחשבת רגילות, והגרף הדינמי של הספרייה מקל על ביצוע התאמות ושינויים מהירים במבנה הרשת. בנוסף, הספרייה מאפשרת גישה ישירה וידידותית למטריצות ה-Softmax הפנימיות בריצה בזמן אמת, מה שמאפשר לנטר את רמות ההסתברות של ארבע מחלקות הפלט הכוללות מנוחה, יד ימין, יד שמאל ורגליים, ולבצע אופטימיזציה מיידית לקוד הניבוי ללא יצירת תקורה חישובית המעכבת את המערכת.

אפיון המערכת המוצעת

ניתוח דרישות המערכת

דרישות פונקציונליות (Functional Requirements - FR)

- **קליטת אותות ודיגיטציה:** המערכת תקרא אותות חשמליים מוחיים (EEG) בצורה לא-פולשנית מקרקפת המשתמש. המערכת תבצע המרה מאות אנלוגי רציף לאות דיגיטלי ברמת דיוק של 24 סיביות ובקצב דגימה של 250Hz.
- **עיבוד מקדים וסינון רעשים:** המערכת תסנן באופן אוטומטי רעשי רשת חשמל (תדר 50Hz) וארטיפקטים ביולוגיים (כגון מצמוצי עיניים וכיווצי שרירי ה-EMG). המערכת תפעיל סינון מרחבי כדי לבודד את הפעילות הנוירולוגית המקומית.
- **חילוץ מאפיינים:** המערכת תעביר את האות לחלונות זמן ותחלץ את צפיפות ההספק הספקטרלי בטווחי התדרים של גלי מיו (8-13Hz) וגלי בטא (13-30Hz).
- **סיווג כוונת תנועה (למידת מכונה):** אלגוריתם הסיווג EEGNET יזהה תבניות של דעיכת גל (ERD) בקורטקס המוטורי , ויסווג את קלט המשתמש בצורה דטרמיניסטית למצבים: "מנוחה", "דימוי תנועה יד ימין" או "דימוי תנועה יד שמאל" או "דימוי תנועה רגלים".
- **בקרת החלקה ושידור:** המערכת תמיר את פלטי הסיווג הבדידיים לזרם פקודות רציף וחלק, תארוז את הנתונים ותשדר אותם בפרוטוקול UDP אל מודול התצוגה.
- **סימולציה גרפית (פלט):** המערכת תציג על גבי המסך דמות וירטואלית ממוחשבת המדגימה ומבצעת את התנועה הפיזית שפוענחה בזמן אמת.

דרישות לא-פונקציונליות (Non-Functional Requirements - NFR)

- **זמן אמת והשהיה:** זמן העיבוד הכולל – מרגע קליטת האות החשמלי בחומרה ועד לביצוע התנועה על ידי הדמות במסך – לא יעלה על 200 מילי-שנייה , כדי למנוע קריסה של לולאת הבקרה שזהו הזמן בו המשתמש שירגיש שיש לו שליטה והכל קורה עכשיו.

מודול המערכת

המערכת מחולקת לשישה מודולים עיקריים הפועלים בסנכרון:

1. **מודול קליטת אותות וחומרה (:Data Acquisition Module)** אחראי על המגע הפיזי עם הקרקפת באמצעות מערך אלקטרודות בתקן 10-20 (בדגש על נקודות 3C ו-4C), הגברת האותות הזעירים וביצוע המרת ADC דיגיטלית בשבב הייעודי 1299ADS. הנתונים מועברים לבקר הקצה בפרוטוקול SPI.
2. **מודול עיבוד מקדים (:Signal Preprocessing Module)** מודול תוכנתי המקבל את זרם הדגימות הגולמי ומפעיל שרשרת מסננים: מסנן חריץ-Notch ב-Hz50 , מסנן מעביר פס של Hz1-40 , מסנן Butterworth , ואלגוריתמים מתקדמים להפרדת מקורות וניקוי ארטיפקטים כגון CAR ו-ICA.
3. **מודול חילוץ מאפיינים (:Feature Extraction Module)** אחראי על חלוקת האות הרציף לחלונות זמן מוגדרים (ms500 עם חפיפה) והפעלת פונקציות חלון. המודול מפעיל אלגוריתם לחילוץ ערכי האנרגיה בתוך תדרי המטרה- מיו ובטא, ומשלב סינון מרחבי.
4. **מודול סיווג ולמידת מכונה:** מיישם מודל מתמטי מסווג הממפה את וקטור המאפיינים המובילים ונותן תווית המייצגת את כוונת התנועה המדויקת.
5. **מודול בקרת רצף ותקשורת:** לוקח את החלטות הסיווג ואורז את הפקודה ומשדר אותה ברשת המקומית באמצעות פרוטוקול UDP.
6. **מודול סימולציה גרפית:** מודול הפלט הגרפי. מאזין לפורט ה-UDP, קולט את זרם הפקודות הרציף, ומתרגם אותו בזמן אמת להנעה חלקה וטבעית של דמות ממוחשבת על גבי המסך.

אפיון פונקציונלי

המערכת מספקת מענה פונקציונלי מלא לשרשרת הפעולות הבאה:

- **פענוח דימוי מוטורי:** המערכת מאפשרת למשתמש להניע את הדמות הממוחשבת באמצעות חשיבה של התנועה, ללא צורך בהפעלה פיזית של שריר כלשהו.
- **בלימת רעשים וארטיפקטים ביולוגיים:** בזמן שהמשתמש חושב על תנועה, המערכת מזהה ומנטרלת באופן דינמי הפרעות הנגרמות ממצמוץ עיניים או כיווץ שרירי פנים, כך שהמשתמש יכול להתנהל באופן חופשי מבלי שהדבר ישבש את זיהוי התנועה.

- **ייזום תנועה וזיהוי כיוון:** עם זיהוי נפילת אנרגיה באלקטרודה השמאלית 3C, המערכת תפעיל באופן פונקציונלי פקודת תנועה ימינה בדמות, ולהיפך.
- **עצירה אוטומטית ומדויקת:** המערכת תזהה את ה"פרץ" האנרגטי של גלי הבטא (Beta rebound) המתרחש עם סיום הדימוי המוטורי, ותפסיק מיד ובאופן אוטומטי את תנועת הדמות, כדי למנוע תנועה עודפת או גלישה מהתחום הרצוי.

ביצועים עיקריים

המערכת מתוכננת לעמוד במדדי הביצוע ההנדסיים הבאים:

- **קצב דגימת חומרה:** דגימה רציפה בתדר של Hz250 לקבלת רזולוציית אות איכותית.
- **רזולוציית אות:** המרת ADC ברמת דיוק גבוהה של 24 סיביות (bit-24) לשמירה על מתחים זעירים ברמת המיקרו-וולט.
- **קצב עדכון חלונות עיבוד:** עיבוד דינמי של חלונות זמן קצרים באורך של 2 שניות -ms500, כאשר החלון מתעדכן ומחליק קדימה כל ms100 (קצב עדכון תוכנתי של 10 פעמים בשנייה).
- **השהיה מינימלית:** זמן תגובה מירבי קבוע של פחות מ-200 מילי-שנייה.
- **חסימת תדר רשת:** הנחתה של לפחות dB40 בתדר הצר של Hz50 באמצעות מסנן חריץ (.Notch Filter)

אילוצים

אילוץ זמן אמת: המערכת מחויבת לפעול כמערכת הפעלה בזמן אמת. תהליך הסיווג ועיבוד האות חייב לקבל קדימות עליונה ואסור לו להתעכב, שכן כל חריגה מזמן העיבוד תגרום למערכת שהיה וחוסר אמון.

אילוצי מיקום האלקטרודות (תקן 10-20): מיקום האלקטרודות אינו גמיש ומחויב לעמוד באופן קשיח במדידות של תקן 10-20 הבינלאומי ישירות מעל הקורטקס הסנסו-מוטורי (נקודות 3C ו-4C), אחרת האותות הנקלטים לא ישקפו את גלי המיו והבטא הרלוונטיים.

אילוץ תדר גיאוגרפי: מסנן התדר החשמלי מותאם לתדר רשת החשמל בארץ שהוא Hz50. לכן המערכת תתן תוצאות מדויקות רק ברשת חשמל בעלת Hz50.

תיאור הארכיטקטורה

Top-Down Level Design – הארכיטקטורה של הפתרון המוצע

ארכיטקטורת המערכת מבוססת על מודל מבוזר של שרת לקוח המשולב בצינור עיבוד נתונים רציף בזמן אמת.

הגישה המובילה בעיצוב המערכת היא **Top-Down**– הגדרת זרם המידע הכללי משלב הפקת האות במוח המשתמש ועד להצגת התנועה על המסך, ופירוקו לארבע שכבות ארכיטקטורה פונקציונליות:

1. שכבת הקלט והקצה (Hardware Cap & Sensors)

   אלקטרודות קורטקס מוטורי -> שבב (24-bit ADC) ADS1299 -> בקר 32ESP

2. שכבת התעבורה והרשת (Network & Transport Layer)

   אריזת הנתונים ב-(Struct (EEG_Raw_Frame -> שידור אלחוטי ב-UDP Wi-Fi

3. שכבת הליבה ועיבוד האות (Processing & Machine Learning Backend)

   שרת Python: סינון (Notch, Bandpass, CAR -> FFT) -> מסווג EEGNET/SVM/LDA

4. שכבת היישום והתצוגה (Presentation & Simulation Client)

   מנוע גרפי: קבלת פקודות -> הנעת האווטאר

תיאור זרימת הנתונים בארכיטקטורה

שלב הדגימה: המשתמש מפעיל דימוי מוטורי (Motor Imagery). אותות ה-EEG נקלטים באלקטרודות, מוגברים ומדוגמים ברזולוציה של 24 סיביות על ידי שבב ה-1299ADS בקצב של 250Hz.

1. **שלב הניהול בקצה:** בקר ה-32ESP קורא את הדגימות הדיגיטליות דרך ערוץ SPI בעקבות פסיקת חומרה (פין DRDY). הבקר אורז את הנתונים ומזרים אותם באופן אלחוטי.
2. **שלב הניתוח והסיווג:** שרת ה-Backend קולט את זרם החבילות, מפעיל חלונות זמן מחליקים של 500ms, מנקה רעשים, מחלץ את הספקטרום של גלי המיו והבטא, ומכריע על כוונת התנועה.
3. **שלב התצוגה:** המחשב קורא את פקודת התנועה המפוענחת ומפעיל את התנועה המתאימה של הדמות הוירטואלית על גבי המסך.

תיאור הרכיבים בפתרון

הפתרון ההנדסי מורכב משלושה חלקים טכנולוגיים המשתלבים יחד למערכת דטרמיניסטית אחת, המבטיחה השהיה הנמוכה מ-200 מילי-שנייה מקריאת האות ועד לתגובת האוואטאר:

1. ארכיטקטורת עיבוד מבוזרת

הארכיטקטורה מפרידה בצורה מוחלטת בין רכיב הקצה המשובץ לבין תחנת המחשוב המקומית, ללא תלות ברשת האינטרנט החיצונית:

- **יחידת הקצה המשובצת :** מבוססת על בקר ה-32ESP ומערך רכיבי ה-1299ADS. יחידה זו פועלת בצורה דטרמיניסטית ומיועדת אך ורק לקריאת אותות המתח החשמלי, אריזתם ושידורם המיידי, ללא שמירת היסטוריית נתונים מקומית על הבקר.
- **תחנת העיבוד וההצגה המקומית :** מחשב ה-PC המריץ שתי שכבות לוגיות במקביל:
  - **שרת העיבוד האנליטי (:Core Analytical Backend)** ממומש בשפת Python (תוך שימוש בספריות SciPy, NumPy ו-PyTorch). הוא מאזין לנתוני החומרה, מריץ את שרשרת הסינון הדיגיטלי, מסנן חריץ Notch של **50Hz** ומסנן Butterworth מעביר-פס בטווח **1-40Hz**, מחלק את האות לחלונות זמן באמצעות Hamming Window, מנקה רעשים ב-ICA ומבצע את סיווג כוונות התנועה ברשת ה-EEGNet.
  - **ממשק הסימולציה:** מנוע גרפי תלת-ממדי האחראי על ניהול ממשק המשתמש (GUI) והזזת האוואטאר הווירטואלי בזמן אמת על בסיס הפקודות המפוענחות.

2. פרוטוקולי תקשורת ומנגנוני זרימת מידע בזמן אמת

התקשורת במערכת מחולקת לשני מקטעים נפרדים המותאמים לעבודה בזמן אמת, ללא שימוש בפרוטוקול HTTP כיוון אינו מתאים להזרמת אותות מהירה:

- **תקשורת החומרה (:SPI)** תקשורת טורית סינכרונית קצרת-טווח הממומשת בין שני שבבי ה-1299ADS (בתצורת Slave) לבין בקר ה-32ESP (בתצורת Master). ערוץ זה פועל בתדר שעון גבוה ומאפשר העברה מהירה של מילים בנות **24 סיביות** ישירות מאוגרי השבב אל זיכרון הבקר, מיד עם קבלת פסיקת החומרה בפין ה-DRDY.
- **תקשורת הרשת (:UDP Sockets over Wi-Fi)** הזרמת נתונים ישירה באמצעות UDP Socket מעל רשת ה-Wi-Fi המקומית. הבחירה ב-UDP על פני TCP/HTTP היא קריטית, מאחר שפרוטוקולים אלו כוללים מנגנוני לחיצת יד (Handshake) ואישורים חוזרים המייצרים השהיות דינמיות המונעות עבודה בזמן אמת.
- **מנגנון הבקרה ברמת האפליקציה (:Custom Struct)** כדי להתגבר על חסרונות ה-UDP כמו חבילות המגיעות מחוץ לסדר או אובדן חבילות מקרי, בקר ה-32ESP אורז את הדגימות בתוך מבנה נתונים קשיח בשם `EEG_Raw_Frame`. ה-Struct מכיל תווי סנכרון , מונה חבילות רציף , חותמת זמן במיקרו-שניותואת מערך דגימות ה-EEG של 16 הערוצים. בצד המקבל, שרת ה-Socket המוטמע ב-Python Backend פורק את החבילות ומסדר או פוסל אותן על בסיס המונה וחותמת הזמן לשמירה על סנכרון מושלם.

3. ניהול מודלים וטעינת משקולות מקומית (Local Model & Weight Management)

- **קובצי משקולות קפואים (:Frozen Model Weights)** לאחר שלב האימון האופליין (Offline Training) של רשת ה-EEGNet ומודל ה-ICA, קובצי המשקולות, מטריצות ה-Embedding והמטריצות המפרידות של ה-ICA מיוצאים ונשמרים כקבצים בינאריים מקומיים קבועים בדיסק הקשיח (באמצעות ספריות PyTorch ו-Pickle).

- **טעינה מיידית לזיכרון:** בעת הפעלת המערכת בזמן אמת, שרת ה-Python המקומי טועה באופן חד-פעמי את קובצי המשקולות הייעודיים של המשתמש ישירות לזיכרון ה-RAM. מנגנון זה מאפשר לרשת לבצע את שלב ההסקהשל כוונת התנועה באופן מיידי ורציף בתוך מילי-שניות בודדות לכל חלון זמן נכנס, ללא צורך בלחיצה על משאבי מערכת או ביצוע אימון מחדש בזמן הריצה.

תיאור תהליכים של מערכת ההפעלה שמתבצעים בפרויקט

1. ניהול ותזמון משימות במערכת ההפעלה (RTOS Tasks)

השימוש ב-RTOS נועד להבטיח דטרמיניזם ותזמון מדויק של תהליכי העיבוד, כך שמשימות קריטיות (כמו קליטת האות) לא יתעכבו בשל משימות רקע. המערכת מחלקת את הפעילות למספר משימות מרכזיות הרצות במקביל (Multi-tasking תחת אילוצי קדימויות):

- **משימת קליטת אותות ודגימה (:Data Acquisition Task)**
  - **תפקיד:** משימה זו פועלת ישירות מול שבב ה-1299ADS. היא ממתינה לפסיקת חומרה המגיעה מפין ה-Data Ready DRDY שמאותת כשדגימת EEG חדשה מוכנה.
  - **מנגנון OS:** המשימה מתעוררת מיידית עם קבלת פסיקה (Interrupt Service Routine - ISR), קוראת את הנתונים ברוחב פס מקסימלי דרך פרוטוקול התקשורת הטורי הסינכרוני (SPI) ומעבירה אותם לזיכרון הבקר.
- **משימת אריזה והזרמת נתונים (:Data Streaming Task)**
  - **תפקיד:** לקיחת נתוני ה-EEG הגולמיים, אריזתם בתוך מבנה נתונים קשיח המוגדר באפליקציה כ-EEG_Raw_Frame, והזרמתם למחשב המארח (Host) בזמן אמת.
  - **מנגנון OS:** שימוש בפרוטוקול UDP מעל שכבת ה-Wi-Fi כדי למנוע השהיות דינמיות (כמו אלו הקיימות ב-TCP עקב מנגנוני לחיצת יד ואישורים חוזרים).

- **משימת סיווג ובקרת הדמות (:Classification & Animation Task)**
  - **תפקיד:** הזנת המאפיינים (גלי המיו והבטא) לתוך אלגוריתם סיווג על מנת לקבוע את כוונת התנועה המדויקת.
  - **מנגנון לוגי תוכנתי:** המערכת מבצעת סינון סטטיסטי (Outlier Detection) – אם הקלט נופל מחוץ לטווח הביטחון (Confidence Interval), המצב מוגדר כרעש- Undefined State ולא מונפקת פקודה, כדי למנוע הפעלות שווא. אם האות מסווג בהצלחה, נשלחת פקודה להצגה גרפית במסך המשתמש.

2. אילוצי זמן אמת קשיחים (Hard Real-Time Latency)

מערכת ההפעלה מנהלת את התהליכים תחת אילוץ זמן אמת קשיח וקריטי:

- **חלון ההשהיה המקסימלי (Control Loop Latency):** הזמן הכולל מקליטת האות הפיזיולוגי ב-ADC (בקרב החומרה) ועד להפעלת התנועה והצגת הפלט על המסך באמצעות הדמות הוירטואלית **חייב להיות קבוע וקטן מ-200ms.**
- **המשמעות ההנדסית:** כל עיכוב או חריגה (Jitter) בזמני הריצה של המשימות מעבר ל-200ms יגרמו לקריסה של לולאת הבקרה (Control Loop) ולחוסר יציבות של הסימולציה והדמות. לכן, ה-RTOS מבטיח עבודה דטרמיניסטית שבה תהליכי הסיווג וקליטת הנתונים מקבלים עדיפות עליונה על פני משימות מערכת אחרות.

ארכיטקטורת רשת

הלקוח- Client: רכיב החומרה:

כובע ה-EEG המבוסס על שבב ה-1299ADS ובקר ה-32ESP, שתפקידו לדגום את האות הנירלי (החשמלי) מהקרקפת, לארוז אותו לחבילות מידע ולשלוח אותו.

רשת התקשורת: המידע משודר באופן אלחוטי על גבי רשת ה-**Wi-Fi**, תוך שימוש בפרוטוקול התקשורת **UDP**- פרוטוקול רשת מהיר במיוחד המאפשר הזרמת נתונים בזמן אמת ללא השהיות, מה שקריטי למערכות ממשק מוח-מחשב.

השרת- Server: שרת ה-Python שרץ על המחשב Backend, שמקשיב לרשת, קולט את חבילות הנתונים, מריץ עליהן את אלגוריתם ה-ICA לניקוי הרעשים, מזין אותן לארכיטקטורת הרשת העצבית EEGNet כדי לקבל את הסיווג, ומעביר את הפקודה הסופית להזזת האוואטאר הוירטואלי.

תיאור API בארכיטקטורה

11.5.1 Hardware-to-Backend API UDP Signal Stream

Description: ממשק האחראי על הזרמת נתוני ה-EEG הגולמיים מיחידת הקצה 32ESP אל שרת ה-Python בזמן אמת.

Protocol: UDP User Datagram Protocol לטובת השהיה מינימלית.

Data Format: פירוט השדות שנשלחים בכל דגימה חשמלית מהחומרה:

`EEG_Raw_Frame Struct`

- `Sync_Byte` (בייט אחד): לסימון תחילת חבילה.
- `Packet_Counter` (ארבע בייטים): מזהה רץ כדי לוודא שלא הלכו לאיבוד חבילות מידע ברשת האלחוטית.
- `Channels_Data` (עשרים וארבע בייטים): ערכי המתח הדיגיטליים שנדגמו מתוך 8 ערוצי ה-1299ADS.
- `Timestamp` (שמונה בייטים): חותמת זמן מדויקת במיקרו-שניות לצורך סנכרון האות.

11.5.2 Backend-to-Avatar API -Socket Command Stream

ממשק המעביר את החלטת הסיווג הסופית מרשת ה-EEGNet אל מודול האוואטאר הוירטואלי כדי להניע אותו.

Protocol: UDP / WebSockets

כיוון שמוכרחת הגעה מהירה של הנתונים.

- **API Message Types (סוגי ההודעות בממשק):**
  - **ההודעה:**`MOVE_RIGHT_HAND`

(שולח)Python Backend **->** Virtual Avatar Manager(מקבל)

קלט: אובייקט עם זיהוי הפעולה
`{"action":"right_hand"}`

תגובה: מודול האוואטאר מפעיל באופן מיידי את האנימציה של הרמת יד ימין.

  - **ההודעה:**`MOVE_LEFT_HAND`

קלט: `{"action": "left_hand"}`

תגובה: מודול האוואטאר מפעיל אנימציה של הרמת יד שמאל.

  - **ההודעה:**`SET_REST_STATE`

קלט: `{"action": "rest"}`

תגובה: האוואטאר חוזר או נשאר במצב עמידה ניטרלי ופסיבי.

תיאור פרוטוקולי התקשורת

SPI - Serial Peripheral Interface

פרוטוקול SPI הוא תקן תקשורת טורית סינכרונית, מהירה ובעלת רוחב פס גבוה, המשמשת להעברת נתונים לטווחים קצרים בין רכיבי חומרה על גבי לוח האלקטרוניקה. במערכת ה-NeuroMotion, פרוטוקול זה מהווה את ערוץ התקשורת החיוני והישיר בין בקר ה-32ESP לבין שבב המרת האותות האנלוגיים 1299ADS. התקשורת מבוססת על ארכיטקטורת ארבעה קווים: MISO, MOSI, SCK, CS ומאפשרת העברת מידע במצב של Full-Duplex- דו-כיווני סימולטני. השימוש ב-SPI חיוני להצלחת הפרויקט שכן הוא מבטיח קצב דגימה מהיר והשהיה אפסית לחלוטין בעת קריאת אותות ה-EEG הגולמיים, דבר המאפשר הזרמה אמינה, נקייה מרעשי השהיה ורציפה בזמן אמת של המידע הנירלי(האות החשמלי) אל הבקר.

פרוטוקול Wi-Fi

פרוטוקול Wi-Fi (המבוסס על משפחת תקני IEEE 802.11) הוא פרוטוקול רשת אלחוטי המאפשר העברת נתונים בקצב גבוה וברוחב פס רחב על גבי תדרי רדיו במרחב המקומי (WLAN). במערכת ה-BCI, פרוטוקול זה משמש כגשר התקשורת האלחוטי המרכזי המקשר בין יחידת הדגימה הניידת (ה-32ESP) לבין שרת ה-Python (ה-Backend) המריץ את אלגוריתמי ה-ICA וה-EEGNet. הפרוטוקול תומך בארכיטקטורת שרת-לקוח (Client-Server) ומאפשר אריזה והזרמה של אותות ה-EEG הרב-ערוציים ללא תלות בכבלים פיזיים. הבחירה בפרוטוקול ה-Wi-Fi, בשילוב עם תצורת UDP, היא אבן יסוד בפרויקט, שכן היא מעניקה חופש תנועה מוחלט למשתמש, מונעת רעשים וארטיפיקטים חשמליים הנובעים מתזוזת כבלים, ומבטיחה ארכיטקטורה מבוזרת, בטוחה וניידת.

UDP- User Datagram Protocol

פרוטוקול UDP הוא פרוטוקול תקשורת מרכזי בשכבת התעבורה (Transport Layer) של מודל ה-TCP/IP, המאפשר העברת חבילות נתונים במהירות מקסימלית וללא צורך בהקמת חיבור לוגי מקדים ומורכב. במערכת ה-NeuroMotion, פרוטוקול זה משמש להזרמת חבילות אותות ה-EEG הגולמיים מה-32ESP ישירות אל שרת ה-Python ב-Backend בזמן אמת. בניגוד לפרוטוקול TCP, ה-UDP אינו כולל מנגנוני לחיצת יד ארוכים, אימות הגעה (ACK) או שידור חוזר של חבילות מידע שאבדו, ובכך הוא מונע לחלוטין השהיות או הצטברות של נתונים בתור התקשורת. במערכות ממשק מוח-מחשב (BCI), רציפות זרם המידע הנירלי והשהיה אפסית הן קריטיות להצלחת הפרויקט. אובדן מקרי של חבילת מידע בודדת הוא זניח, בעוד שעיכוב קטן בנתונים עלול לפגוע ברצף סדרת הזמן הנדרש עבור מודל ה-EEGNet. הבחירה ב-UDP מעניקה למערכת את היכולת להזרים נתונים בקצב גבוה וקבוע, ובכך מבטיחה תגובה מיידית וחלקה לחלוטין של דמות האוואטאר הווירטואלי למחשבות המשתמש.

תיאור ארכיטקטורת שרת לקוח

השרת- The Server: שרת ה-Python הרץ ב-Backend על גבי המחשב.

השרת מאזין באופן קבוע לפורט תקשורת מסוים, קולט את זרם המידע, מריץ את אלגוריתם ה-ICA לניקוי הרעשים, ומפעיל את רשת העצבים EEGNet לצורך סיווג התנועה והפעלת האוואטאר.

הלקוח- The Client: יחידת קצה החומרה- הבקר 32ESP יחד עם שבב ה-1299ADS.

תפקידו של הלקוח בארכיטקטורה זו הוא פונקציונלי בלבד: דגימת האותות החשמליים (הנירליים) מהקרקפת, אריזתם לחבילות מידע קטנות, ושידורן המיידי באופן אלחוטי.

תיאור תהליכי אבטחת מידע בפרויקט

תיאור ההגנות

שלמות הנתונים ואימות חבילות המידע

פרוטוקול UDP אינו מבטיח הגעה תקינה או מסודרת של חבילות המידע, מה שעלול לאפשר לתוקף להזריק חבילות זדוניות או משובשות. לכן תהליך שמירה על שלמות המידע מיושם בתוך מבנה חבילת המידע (`EEG_Raw_Frame Struct`). השרת מבצע אימות על כל חבילה שנקלטת:

- **Sync Byte Validation:** בדיקה שחבילת המידע נפתחת בבית הסינכרון המוגדר בחומרה.
- **Packet Counter Verification:** המודל עוקב אחרי מונה חבילות רץ (Sequence Number). אם חל שינוי לא צפוי ברצף המונים למשל, קפיצה פתאומית, המערכת מזהה זאת כשיבוש תקשורת או כניסיון הזרקת מידע (Injection Attack) ופוסלת את רצף הדגימות כדי להגן על מודל ה-EEGNet.

חסינות השרת והגנה מפני הצפות

חלק בלתי נפרד מאבטחת המערכת ושמירה על יציבותה בזמן אמת מתרכז בחסינות תוכנת השרת המקומית (Local Backend Server) הרצה על גבי המחשב המארח. מאחר שתקשורת הנתונים בין בקר ה-32ESP לתוכנת הפייתון מבוססת על פרוטוקול UDP, המערכת פותחת שער האזנה (Port) לקבלת חבילות המידע בצורה רציפה. כדי למנוע מצבים שבהם תקלה בחומרה או ניסיון זדוני של גורם שלישי ברשת המקומית יובילו להצפת הפורט בכמות חבילות חריגה שתגרום לקריסת האפליקציה או להאטת המחשב, הטמענו במערכת מנגנון הגבלת קצב חכם מסוג Rate Limiting ברמת קוד השרת.

מנגנון ה-Rate Limiting מגדיר חסם עליון קשיח לכמות חבילות ה-UDP שהשרת מוכן לקבל ולעבד בכל שנייה, בהתאם למגבלות הפיזיקליות של הפרויקט. מכיוון שתדר הדגימה המקסימלי של שבב ה-1299ADS נקבע ל-250 הרץ, השרת מצפה לקבל בדיוק 250 חבילות בשנייה. הקוד מתוכנת כך שכל חריגה קיצונית מקצב זה, המגיעה עד לרף מקסימלי של 500 חבילות בשנייה, מוגדרת מיד כהצפה לא חוקית. במקרה כזה, השרת חוסם זמנית את הפורט הספציפי ומסנן את החבילות העודפות בצורה אוטומטית מבלי לעבד אותן, ובכך הוא מגן על משאבי המעבד, מונע זליגת זיכרון ומבטיח את רציפות הניבוי של המודל גם תחת תנאי עומס קיצוניים ברשת.

הפרויקט עוסק בלמידת מכונה- פירוט והרחבה

תהליך איסוף הנתונים

1. קליטה (אלקטרודות): קליטת אותות החשמל הזעירים (ברמת המיקרו-וולט) מהקרקפת בזמן דמיון התנועה החופשית.

2. הגברה והמרה (1299ADS): השבב מגביר את האותות החלשים וממיר אותם מאנלוגיים לדיגיטליים ברזולוציה גבוהה של 24-bit.

3. העברה לבקר (SPI): בקר ה-32ESP מזהה שהמידע מוכן ומושך אותו מהשבב במהירות מקסימלית דרך פרוטוקול SPI.

4. אריזה (Struct): ה-32ESP מאגד את הנתונים מכל 8 הערוצים, יחד עם מונה חבילות רץ וחותמת זמן, לתוך מבנה נתונים בינארי קבוע.

5. שידור אלחוטי (Wi-Fi & UDP): הבקר משדר את החבילה בזמן אמת לשרת ה-Python באמצעות פרוטוקול UDP המונע השהיות (Latency).

בשלבי אימון המודל נעזרתי במאגרי מידע גלובליים המונגשים דרך פלטפורמות Kaggle וספריית MOABB-Mother of All BCI Benchmarks, המכילים הקלטות אותות מרובות של נבדקים מגוונים המבצעים משימות דמיון מוטורי (Motor Imagery). נתונים אלו הותאמו במדויק לארבע מחלקות הסיווג שהוגדרו במערכת, הכוללות מצב מנוחה, דמיון תנועת יד ימין, דמיון תנועת יד שמאל ודמיון תנועת רגליים, והם אפשרו לרשת ללמוד את מאפייני התדר והמרחב הכלליים של הפעילות המוחית החשמלית.

טיוב ואיזון נתונים

שלב טיוב ואיזון הנתונים במערכת ה-NeuroMotion הוא קריטי כי הוא מכין את המידע הנירלי הגולמי להזנה אופטימלית ומדויקת לתוך רשת העצבים EEGNet. תהליך הטיוב מתמקד בניקוי האות וסינון רעשים פיזיקליים וביולוגיים. האותות שעוברים מהחומרה עוברים סינון דיגיטלי ראשוני להסרת תדר רשת החשמל ומסנן לחיתוך תדרים לא רלוונטיים, ולאחר מכן מופעל אלגוריתם ה-ICA. אלגוריתם זה מפרק את האותות הרב-ערוציים לרכיבים עצמאיים, מזהה ומבודד ארטיפקטים (רעשים שמקורם אינו מוחי) כגון מצמוצי עיניים ותזוזות שרירי פנים, ומנקה אותם מבלי לפגוע במידע המוחי הטהור.

בפועל- קלטתי את האותות הגולמיים מה-32ESP ישירות בקוד ה-Backend שכתבתי ב-Python. בשלב הראשון, הפעלתי על המידע הדיגיטלי מסנן חוסם תדר (Notch Filter) המכויל בדיוק ל-50Hz כדי להסיר את ההפרעות של רשת החשמל המקומית. מיד לאחר מכן, העברתי את האות במסנן מעביר פס (Bandpass Filter) מסוג Butterworth, וחתכתי את התדרים כך שיבודדו במדויק רק את הטווח שבין 8Hz ל-30Hz. החיתוך הזה ממוקד ברצועות התדר של מקצבי המיו והבטא במוח, שבהם מתרחשים שינויי האנרגיה החשמלית בזמן דמיון תנועה, שזהו בדיוק המאפיין הפיזיולוגי המרכזי שרשת ה-EEGNet שלי נדרשת לזהות ולסווג.

בשלב השני, הרצתי בקוד אלגוריתם ICA (ניתוח רכיבים עצמאיים) כדי לנקות ארטיפקטים ביולוגיים מורכבים. בעזרת האלגוריתם פירקתי את האותות שהגיעו משמונת הערוצים של ה-1299ADS למקורות עצמאיים, זיהיתי ובודדתי רכיבים שאינם ממקור מוחי – כמו רעשים שנוצרו ממצמוצי עיניים (EOG) או כיווצי שרירי פנים ולסת (EMG) – ושחזרתי את האות מחדש כשהוא נקי. לבסוף, כדי לפתור את בעיית איזון הנתונים (Data Balancing) לפני שלב האימון ב-PyTorch, מימשתי בקוד ה-Dataset מנגנון דינמי המבצע דגימת חסר ודגימת יתר (Undersampling ו-Oversampling), לצד חלוקה מרובדת (Stratified Split) של המידע. המנגנון שכתבתי דאג לכך שלכל ארבע המחלקות במערכת-מנוחה, יד ימין, יד שמאל ורגליים יהיה מספר דגימות שווה ומאוזן לחלוטין, מה שמונע מרשת העצבים לפתח הטיה בניבוי והבטיח אחוזי דיוק גבוהים ויציבים בזמן אמת.

סוג הלמידה הנבחר

למידה הלא מונחית- ICA שמנקה ומסדרת את המידע, ואז הלמידה המונחית- EEGNet לוקחת את המידע הנקי ומסווגת אותו כדי להבין מה הנבדק באמת עשה.
עיבוד אותות וסינון באמצעות ICA- Independent Component Analysis

נוסחאות ורשתות למידה

כדי לבצע את עיבוד אותות ה-EEG וסיווג כוונות התנועה בזמן אמת, המערכת נשענת על שני מודלים מתמטיים מרכזיים. על מנת לעמוד בדרישות ה-Latency הנמוכות, הארכיטקטורה מחולקת בצורה מבוזרת בין **שתי מכונות מחשוב שונות**:

1. **מכונת האימון האופליין (Offline Training Environment):** סביבת מחשוב בעלת כוח עיבוד גבוה (Google Colab / GPU Rig) המשמשת לניתוח סטטיסטי כבד, אימון רשת העצבים וחילוף משקולות המודל מראש.
2. **מכונת הריצה בזמן אמת (Online Real-time Server):** שרת ה-Python המקומי (Localhost) המריץ את המודלים הקפואים (Inference) בצורה אופטימלית ומהירה על חלון הזמן הנכנס.

אלגוריתם הפרדת מקורות וניקוי רעשים: ICA- Independent Component Analysis

מודל ה-ICA הוא מודל מסוג למידה לא מונחית (Unsupervised Learning) המבוסס על עקרון הפרדת מקורות עיוורת (Blind Source Separation). האלגוריתם מניח כי אות ה-EEG הרב-ערוצי הנקלט במערכת (X) מהווה שילוב ליניארי של מקורות מידע עצמאיים לחלוטין (S), כגון פעילות קורטיקלית טהורה לצד רעשים פיזיולוגיים (מצמוצי עיניים, תנועות שרירי לסת).

הקשר המתימטי מוגדר באופן הבא:

$$X = A \cdot S$$

כאשר A היא מטריצת הערבוב (Mixing Matrix). המטרה היא למצוא מטריצה מפרידה W (ההופכית ל-A) אשר תמקסם את הסטייה של הרכיבים מהתפלגות גאוסיאנית (Non-

Gaussianity) באמצעות מדדי Kurtosis או Negentropy, ובכך תחלץ את הרכיבים הבלתי תלויים (Independent Components - ICs).

חלוקת העבודה בין המכונות:

- **במכונת האימון (Offline):** האלגוריתם מריץ חישובים סטטיסטיים איטרטיביים כבדים על גבי נתוני העבר של המשתמש כדי לחשב במדויק את המטריצה המפרידה W. בשלב זה, מזהים ומסמנים באופן ידני או אוטומטי את מספרי הרכיבים (Components) המשויכים לרעשי מצמוץ (המאופיינים בהספק גבוה בתדרים נמוכים) או רעשי שרירים, ומנטרלים אותם (איפוס ערכי הרכיב). המטריצה המטוהרת נשמרת כקובץ קשיח.
- **במכונת הריצה בזמן אמת (Online):** שרת ה-Python המקומי טוען את מטריצת W הקבועה מראש. עבור כל חלון זמן נכנס, המכונה אינה מבצעת אימון מחדש, אלא מפעילה כפל מטריצות מיידי ומהיר ליצירת האות הנקי (Reconstruction), תהליך הרץ בזמן ליניארי לחלוטין.

רשת העצבים העמוקה לסיווג: EEGNet

לאחר ניקוי האות, החלון המטוהר מוזרם אל רשת **EEGNet** – ארכיטקטורת רשת עצבים קונבולוציונית (CNN) קומפקטית שפותחה במיוחד עבור אותות EEG. הרשת שואבת השראה מאלגוריתם העיבוד הקלאסי **FBCSP**, אשר מחלק את האות לפסי תדר שונים ומחלץ מאפיינים מרחביים (Spatial Features) כדי למקסם את השונות בין סוגי דמיונות התנועה (Motor Imagery).

במקום לבצע את שלבי ה-FBCSP בצורה ידנית ומורכבת, שכבות הקונבולוציה ב-EEGNet לומדות לבצע את הסינון התדרי והמרחבי בצורה משולבת ואוטומטית דרך הארכיטקטורה הבאה:

- **Temporal Convolution:** שכבה ראשונה המבצעת סינון זמני על פני כל ערוץ בנפרד (Temporal Kernels) כדי לחלץ מאפיינים תדריים מתוך האות.
- **Depthwise Convolution**: שכבה המבצעת למידה וסינון מרחבי (Spatial Filtering) על ידי שילוב אותות מכל 16 האלקטרודות במקביל. שלב זה מקביל מתמטית לפעולת ה-CSP הקלאסית.
- **Separable Convolution**: שכבה המפרידה בין קונבולוציה עומקית לנקודתית. מבנה זה מפחית דרסטית את מספר הפרמטרים של הרשת ומגן עליה מפני התאמת-יתר (Overfitting).
- **פונקציית אקטיבציה ELU:** שימוש ב-Exponential Linear Unit המאפשר לטפל בצורה חלקה בערכים השליליים הקיימים באות ה-EEG, ומונע את בעיית ה- Dying ReLU.

שכבת התאמה אישית (User ID Embedding Layer)

כדי להתמודד עם השונות הגבוהה בין משתמשים שונים (הנובעת משוני פיזיולוגי כמו עובי הגולגולת או מיקום פיזי של האלקטרודות), שילבתי בראש הרשת (לפני שכבות ה- Fully Connected) שכבת **Embedding**. השכבה מקבלת את מזהה המשתמש וממירה אותו לווקטור רציף במימד D. וקטור זה מוצמד (Concatenation) למאפיינים שחולצו מהאות, ומתפקד מתמטית כהסטת ה-Bias של הרשת כפונקציה של זהות המשתמש:

$$y = f(x, u)$$

(כאשר X הוא קלט ה-EEG המטוהר ו-U הוא וקטור ה-Embedding של המשתמש).

חלוקת העבודה בין המכונות:

- **במכונת האימון (Offline):** הרשת מתאמנת על מערך נתונים גדול באמצעות אלגוריתם Backpropagation. בשלב זה המכונה מעדכנת את כל המשקולות של שכבות הקונבולוציה ולומדת את ערכי וקטור ה-Embedding האופטימליים עבור כל משתמש. המודל המאומן מיוצא כקובץ משקולות קפוא (כדוגמת קובץ .pt או .h5).
- **במכונת הריצה בזמן אמת (Online):** שרת ה-Python המקומי טוען את הרשת הקפואה לזיכרון. כאשר מגיע חלון דגימות חדש, השרת מריץ אך ורק את שלב ההסקה (Forward Pass) – חישוב מתמטי מהיר של שברירי מילי-שנייה המניב את קוד פקודת התנועה וה-Confidence Score עבור האוואטאר.

כיוונון, ייעול וניטור מדדי ביצוע

בשלב אימון הרשת, התרכזתי באופטימיזציה של המודל כדי להגיע לביצועים הטובים ביותר ולמנוע תופעות של התאמת-יתר. ביצעתי סדרה של ניסויים ובדיקות על גבי ארכיטקטורת ה-EEGNet, ובמסגרתם בחנתי שילובים שונים של היפר-פרמטרים, כולל שינוי קצבי הלמידה, התאמת גודל ה-Batch ועדכון שיעורי ה-Dropout, עד לקבלת כושר ההכללה המיטבי על דגימות המוח. כדי לנטר את התנהגות הרשת לאורך תהליך הריצה, הפקתי בקוד מדדי ביצוע מפורטים כגון גרפי פונקציית ההפסד, אחוזי הדיוק ומטריצת בלבול, אשר אפשרו לי לעקוב בזמן אמת אחר קצב ההתכנסות של המודל ולוודא שהוא מבחין בצורה נכונה בין מצבי התנועה והמנוחה.

מבחינת ייעול הקוד והתאמתו לעבודה בזמן אמת, התרכזתי בצמצום השהיית ההסקה ב-Backend כדי לאפשר חוויית משתמש רציפה. כדי שהרצת האלגוריתמים ומודל הלמידה העמוקה לא ייצרו עומס על המעבד ולא יעכבו את לולאת המשוב, מימשתי בקוד ארכיטקטורה המפרידה בין קליטת הנתונים לבין העיבוד שלהם, תוך שימוש במבני נתונים מתאימים לניהול זרם המידע ברקע. בנוסף, ניצלתי את יכולות החישוב המקביליות והוקטוריות של ספריות הקוד בפייתון כדי להאיץ את המניפולציות על המטריצות של האות. בעזרת תכנון זה, הקטנתי למינימום את זמן התגובה הכולל מקליטת חבילת המידע ועד להפקת פקודת הניבוי, מה שמבטיח תנועה חלקה ורציפה של הדמות ללא השהיות מורגשות.

תיאור התוכנה

פירוטAPI

לצורך העברת נתונים מהירה בזמן אמת ומניעת השהיות (Latency), מימשתי שני ממשקי API ייעודיים המבוססים על פרוטוקול UDP:

1. ממשק חומרה-תוכנה: מה-32ESP אל ה-Python Backend

ממשק זה מזרים את אותות ה-EEG הגולמיים בקצב של 250Hz. הנתונים נארזים בחומרה בתוך מבנה נתונים קבוע שנשלח לפורט UDP בשרת המקומי:

- **Packet ID** (גודל: 4 בתים - Unsigned Long): מונה חבילות רץ לזיהוי איבוד מידע (Packet Loss).
- **Timestamp** (גודל: 4 בתים - Unsigned Long): חותמת זמן במילי-שניות לצורך סנכרון זמנים.
- **EEG Channels Data** (גודל: 64 בתים): מערך של 16 ערוצי EEG (משני שבבי 1299ADS), כאשר כל ערוץ מיוצג כערך נקודה צפה (Float של 4 בתים) במיקרו וולט.

2. ממשק תוכנה-גרפיקה: מה-Python Backend אל ה-D Avatar Dashboard3

ממשק זה מעביר את פקודת התנועה המפוענחת מרשת ה-EEGNet אל מערכת הרינדור הגרפי בפורט UDP נפרד:

- **Command Code** (גודל: בית 1 - Char): תו מייצג עבור פקודת התנועה האופרטיבית:
  - 'W' – מצב מנוחה ועצירה (Rest / Idle)
  - 'R' – תנועה ימינה (Right Hand Imagery)
  - 'L' – תנועה שמאלה (Left Hand Imagery)
  - 'F' – תנועה קדימה (Legs Imagery)
- **Confidence Score** (גודל: 4 בתים - Float): רמת הוודאות הסטטיסטית של הניבוי (ערך בין 0.0 ל-1.0), המשמשת כמנגנון סינון (Threshold) למניעת תנועות שווא.

סביבת עבודה

ממשק הפיתוח המרכזי- VS Code

סביבת העבודה הראשית במחשב המקומי. היא משמשת כעורך קוד המנהל את כלל קבצי הפרויקט תחת תיקיות מסודרות, ומאפשרת כתיבה, ניטור ואינטגרציה מהירה בין קוד ה-Backend לקוד החומרה.

סביבת קצה וחומרה- Arduino (שפת C++)

הסביבה המשמשת לכתיבה, הידור וצריבת הקוד על גבי מיקרו-בקר ה-32ESP. אחראית על ניהול חומרת ה-1299ADS, תזמון פסיקות חומרה (Interrupts), אריזת נתוני ה-EEG ושידורם האלחוטי.

סביבת עיבוד והסקה – Python

סביבת הריצה המקומית המפעילה את שרת ה-Sockets במחשב. היא קולטת את זרם המידע מהחומרה, מריצה ספריות ייעודיות (כגון NumPy ו-SciPy) לטובת אלגוריתם ה-ICA (ניקוי רעשים), ומריצה את מודל ה-EEGNet לביצוע הסקה (Inference) ותרגום פקודות בזמן אמת.

סביבת אימון מודל ה-AI – פלטפורמת Google Colab: סביבה מבוססת ענן המשמשת באופן בלעדי לשלב המחקר, האופטימיזציה והאימון (Training) של רשת העצבים הקונבולוציונית (**CNN**). היא מנצלת מעבדים גרפיים חזקים (**GPU**) להאצת הלמידה העמוקה, וממנה מיוצא קובץ המשקולות הסופי אל השרת המקומי.

שפות תכנות

שפת C++

תפקיד בפרויקט: כתיבת קוד המקור עבור מיקרו-בקר ה-32ESP והרכיב הייעודי 1299ADS.

C++היא שפה קרובה לחומרה ומצטיינת במהירות ריצה מקסימלית וניהול זיכרון קפדני. היא חיונית לטיפול מהיר בפסיקות חומרה, ניהול תקשורת טורית בפרוטוקול SPI, ואריזה בינארית של אותות ה-EEG לצורך שידור אלחוטי מיידי בהשהיה אפסית.

שפת Python

תפקיד בפרויקט: ניהול שרת ה-Sockets המקומי, הרצת אלגוריתמיקה (ICA) ואימון/הסקה של מודל ה-EEGNet.

Pythonשפה עילית המהווה את הסטנדרט המוביל בעולם למדעי הנתונים ובינה מלאכותית. פייתון נבחרה בזכות המערכת האקולוגית העשירה שלה (ספריות כמו NumPy, SciPy ו-TensorFlow) המאפשרת לקלוט את זרם ה-UDP, לבצע חישובים מטריציאליים מורכבים על אותות זמן, ולתרגם אותם פקודות בזמן אמת.

ניתוח ותרשים UML / UseCase של המערכת המוצעת.

תיאור ה UC-העיקריים של המערכת

UC-1: הזרמת אותות, ניקוי וסיווג כוונת תנועה בזמן אמת Real-Time EEG Processing Pipeline

המשתמש חובש את כובע ה-EEG ומבצע דמיון מוטורי (Motor Imagery). המערכת קולטת את זרם האותות החשמליים מהקרקפת, מפרקת ומנקה את המידע מרעשי רקע וארטיפקטים ביולוגיים במקביל, ומפעילה מודל למידה עמוקה המסווג את כוונת התנועה ומפעיל את האוואטאר הווירטואלי בהתאם.

- **תנאי קדם:** המערכת תוכל להתחיל בעיבוד רק לאחר קיום קשר אלחוטי יציב בין בקר ה-32ESP לשרת ה-Python.

זרימה ראשית:
המפתח או המטפל מפעיל את סשן האימון דרך ממשק הניהול. המערכת מאתחלת את ה-Socket ומנקה את חוצצי הזיכרון (Buffers) של השרת לטובת סביבה סטרילית ונקייה מרעשי ריצה קודמים. לאחר מכן, שבב ה-1299ADS מתחיל לדגום את הפוטנציאלים החשמליים מהקרקפת, ובקר ה-32ESP מושך את הנתונים ומשדר אותם אלחוטית ב-Wi-Fi.

במקביל, שרת ה-Python קולט את חבילות המידע הבינאריות UDP Sockets, מבצע להן דקומפרסיה ומזרים אותן אל מערך האנליזה בזמן אמת. המידע הגולמי מ-8 הערוצים עובר סינון דיגיטלי ראשוני להסרת תדרי רשת החשמל ורעשי סביבה.

האותות המסוננים מוזנים לאלגוריתם ה- **ICA** שמפרק את זרם המידע הרב ערוצי לרכיבים עצמאיים. עבור כל רכיב נבדוק האם הוא חורג מהנורמה הקוגניטיבית ומייצג ארטיפקט ביולוגי (כגון מצמוצי עיניים או כיווצי שרירי פנים). הרכיבים הרועשים מאופסים והאות משוחזר בצורה נקייה.

המערכת לוקחת את חלון הזמן הנקי ומריצה עליו את רשת העצבים המאומנת **EEGNet** במטרה לחלץ מאפיינים מרחביים (Spatial) ותדריים (Temporal) של דמיון מוטורי ולזהות לאיזה איבר המשתמש התכוון.

עבור כל חלון זמן מעובד, המערכת מנתחת את ההסתברויות שפלט מודל ה-EEGNet. המערכת בוחנת האם רמת הוודאות חרגה מסף ה-Threshold שהוגדר מראש. במידה וזוהה זיהוי מובהק, המערכת מייצרת אובייקט פקודה MovementCommand ושולחת אותו בפרוטוקול UDP מהיר אל ממשק האוואטאר הווירטואלי (React/Unity) לצורך הפעלת האנימציה המתאימה בזמן אמת.

לבסוף, הנתונים הנירליים הגולמיים יחד עם תוצאות הסיווג ומדדי איכות האות נרשמים בצורה אנונימית לחלוטין (Privacy by Design) בקובצי לוג ייעודיים לצורך ניטור מדדי ביצוע (מצב מפתח/מטפל).

- **תנאי קצה:** האוואטאר הוירטואלי מגיב בצורה חלקה וזזה לאורך ציר הזמן בהתאם למחשבות המשתמש.
- **מקרה קצה:** במידה ויש נפילת חבילות מידע קריטית באוויר (Packet Loss חריג) או ניתוק מהחומרה, המערכת מזהה שהזרם אינו תקין, עוצרת את הניתוח ומציגה הודעת שגיאה/התרעת ניתוק מתאימה בממשק המפתח.

הצגת מקרה עבור כל הפונקציות העיקריות בפרויקט

1. אתחול וחיבור החומרה.

בקר ה-32ESP מתחבר לרשת ה-Wi-Fi המקומית, פותח Socket מול שרת ה-Python ומבצע כיול פנימי לשבב ה-1299ADS, ולאחר מכן המערכת מאשרת מעבר למסך העבודה הראשי.

2. קליטת אותות ושידור אלחוטי במהירות גבוהה (Data Acquisition & UDP Streaming)

המערכת דוגמת באופן רציף את 16 ערוצי ה-EEG מהקרקפת. שבב ה-1299ADS מבצע הגברה והמרה דיגיטלית ברזולוציה גבוהה של **24-bit**, ומוריד את פין ה- DRDY- Data Ready. בקר ה-32ESP מזהה את הפסיקה (Interrupt), מושך את המידע בפרוטוקול **SPI**, אורז אותו בתוך מבנה נתונים בינארי קשיח, ומשדר אותו מיידית ב-**UDP** לשרת ה-Python למניעת השהיות.

3. סינון דיגיטלי והסרת ארטיפקטים

שרת ה-Python קולט את החבילות ומריץ שרשרת עיבוד לטיוב האות. המערכת מפעילה מסנני תדר דיגיטליים ולאחר מכן מריצה את אלגוריתם ה-**ICA** המבודד ומאפס רכיבי רעש של מצמוצים ותזוזות שרירים. האותות הנקיים נשמרים זמנית בזיכרון בתוך חלון זמן נע.

4. פענוח וסיווג כוונות תנועה

המערכת לוקחת את מטריצת חלון הזמן הנקייה ומזינה אותה לתוך רשת העצבים העמוקה **EEGNet** שהיא CNN מותאמת לבדיקות EEG שאימנו מראש שיוצאה מפלטפורמת Google Colab. הרשת מחלצת מאפיינים זמניים ומרחביים מהאות הנירלי ומנפקת וקטור הסתברויות המזהה את כוונת המשתמש (יד ימין, יד שמאל, רגליים או מנוחה) בתוך מילי שניות בודדות.

מבני נתונים בהם נעשה שימוש בפרויקט

1. מבנה נתונים בינארי קשיח (C++ Struct) לניהול חבילות מידע (Data Packetization)

בצד החומרה (32ESP), נעשה שימוש במבנה נתונים קשיח מסוג Struct הנקרא EEG_Raw_Frame . מבנה זה מאגד בצורה רציפה בזיכרון את הנתונים מכל 16 הערוצים, מונה חבילות רץ וחותמת זמן במיקרו-שניות.

ה-Struct מאפשר לבצע סריאליזציה מיידית של המידע למערך בתים (Byte Array) ולשלוח אותו כחבילה אחת אחידה ברשת. הדבר קריטי לשמירה על סדר כרונולוגי מדויק ומאפשר

לשרת ה-Python לבצע דה-סריאליזציה מהירה בזמן קבוע של O(1) לפי היסטים (Offsets) קבועים בזיכרון.

2. מטריצות ומערכים רב-ממדיים (Multidimensional NumPy Arrays)

כל המידע הנירלי המצטבר ב-Backend מנוהל באמצעות מערכים רב-ממדיים של ספריית NumPy בפייתון, המייצגים את מימד הזמן מול מימד המרחב. פירוק האותות לרכיבים עצמאיים ב-ICA והזנתם לשכבות הקונבולוציה של ה-EEGNet דורשים אלגברה ליניארית מתקדמת. מערכי NumPy ממומשים ב-C מאחורי הקלעים ומאפשרים לבצע פעולות אלו על פני כל 16 הערוצים בו-זמנית במהירות קיצונית ללא לולאות פייתון איטיות, דבר המבטיח עמידה בזמן אמת.

חישוב יעילות האלגוריתם

על מנת להבטיח את פעולת המערכת בזמן אמת ובלולאת משוב מהירה, ביצעתי ניתוח סיבוכיות זמן ומקום עבור כל אחד משלבי עיבוד הנתונים בפרויקט:

1. דגימה, אריזה ושידור בחומרה (1299ESP32 & ADS)

- **סיבוכיות זמן:** O(1) – שליפת הנתונים מ-16 ערוצי ה-EEG דרך ערוץ ה-SPI ואריזתם במבנה הנתונים (Struct) אורכת זמן קבוע לחלוטין. היא אינה תלויה בגורמים דינמיים, מאחר שמספר הערוצים ורזולוציית הביטים קבועים קשיחים בחומרה.
- **סיבוכיות מקום:** O(1) – בקר ה-32ESP אינו שומר היסטוריית דגימות בזיכרון. בכל פעימת שעון משתמשים באותו נפח זיכרון קבוע של ה-Struct, והחבילה נשלחת מייד ברשת ומשוחררת.

2. קליטת המידע ובניית חלון הזמן (UDP Socket Receiver)

- **סיבוכיות זמן:** O(n) – עבור n חבילות מידע שמגיעות מהחומרה, קוד ה-Backend פורק את הנתונים ומעדכן את המערך המעגלי (Circular Buffer) בצורה ליניארית ביחס ישיר למספר החבילות הנכנסות.
- **סיבוכיות מקום:** O(w) – המערכת מנהלת בזיכרון ה-RAM חלון זמן נע בגודל קבוע של W דגימות (לדוגמה, חלון של 500 דגימות המייצגות 2 שניות עבודה בקצב של 250Hz), ולכן צריכת המקום קבועה ותלויה אך ורק בהגדרת גודל החלון הנדרש לניתוח.

3. אלגוריתם ניקוי האות והסרת רעשים (ICA Processing)

- **סיבוכיות זמן:** O(w) – בזמן הריצה הדינמי, הקוד מפעיל מטריצת פילטר קבועה שחושבה מראש על חלון המידע הנוכחי. פעולה זו מוגדרת ככפל מטריצות שסיבוכיותו הרשמית היא $O(C^2 \cdot W)$, כאשר C הוא מספר ערוצי ה-EEG. מאחר ובפרויקט שלי מספר הערוצים קבוע קשיח (C = 16), הביטוי C^2 הופך לקבוע ולכן החישוב רץ בזמן ליניארי לחלוטין המושפע רק מאורך החלון W
- **סיבוכיות מקום:** O(w) – האלגוריתם מחזיק בזיכרון את חלון המידע הרב-ערוצי הנוכחי לצורך ביצוע פעולת הכפל המתמטית. מכיוון שמספר הערוצים C קבוע, דרישת הזיכרון הדינמי תלויה ישירות באורך החלון המעובד בלבד.

4. אלגוריתם סיווג כוונות התנועה (EEGNet Inference)

- **סיבוכיות זמן:**O(w)– שלב ההסקה (Forward Pass) של רשת העצבים EEGNet מתבצע על חלון זמן באורך W. כיוון שארכיטקטורת הרשת קפואה ומבוססת על שכבות קונבולוציה ייעודיות (Depthwise & Separable Convolutions) בעלות מספר פרמטרים קבוע מראש, זמן הריצה הוא ליניארי ומהיר במיוחד, ומושפע אך ורק מציר אורך חלון הזמן W המוזן אליה.
- **סיבוכיות מקום:**O(w) – צריכת הזיכרון של המודל בזמן הניבוי פרופורציונלית לגודל של טנסורי האקטיבציה (Activation Tensors) בשכבות השונות, אשר משתנים ומתכווצים ליניארית בהתאם לממד הזמן W של הקלט.

5. שידור פקודות וניטור מדדים (Execution & Sensor Fusion)

- **סיבוכיות זמן:**O(1) – בדיקת סף הוודאות הסטטיסטי של הניבוי, אריזת פקודת התנועה הסופית ושליחתה בפרוטוקול UDP אל עבר ה-Avatar התלת-ממדי מתבצעות בזמן קבוע ומיידי.
- **סיבוכיות מקום:**O(1) – הפעולה דורשת הקצאת זיכרון קבועה ומינימלית לצורך בניית אובייקט הפקודה ושילוחו ברשת, ללא שמירת היסטוריית פקודות ברכיב זה.

הקשרים בין היחידות השונות

1. הקשר בין שכבת האלקטרודות (1299ADS) ליחידת המיקרו-בקר (32ESP)

- **סוג הממשק:** תקשורת חומרה טורית מבוססת פסיקות חומרה ופרוטוקול SPI.
- **תיאור זרימה:** שבב ה-1299ADS דוגם את האותות החשמליים האנלוגיים מהקרקפת. ברגע שחבילת נתונים (מכל 16 הערוצים) מוכנה, השבב מוריד את קו ה-DRDY הפיזי. פין זה מחובר ל-GPIO של ה-32ESP ומשמש כפסיקה חומרתית. עם קבלת הפסיקה, ה-32ESP מפעיל פונקציית ISR (Interrupt Service Routine) ומושך את נתוני ה-24 ביט הדיגיטליים במהירות גבוהה דרך ערוץ ה-SPI, ומכין אותם לאריזה.

2. הקשר בין יחידת החומרה (32ESP) לשרת העיבוד הראשי (Python Backend)

- **סוג הממשק:** תקשורת רשת אלחוטית אסינכרונית (Wireless Sockets) מבוססת פרוטוקול **UDP**.
- **תיאור זרימה:** ה-32ESP אורז את מערך הבתים של ה-Struct ומשדר אותו באוויר באמצעות רכיב ה-Wi-Fi המובנה שלו אל כתובת ה-IP והפורט הייעודי של שרת ה-Python. השרת מריץ Thread נפרד המאזין ל-UDP Socket בסטרימינג רציף. הבחירה ב-UDP מונעת השהיות חומרה ומבטלת את הצורך בלחיצות יד מתישות עבור כל דגימה, דבר המבטיח זרימת נתונים חלקה בזמן אמת.

3. הקשר בין צינור עיבוד האותות למנוע הלמידה העמוקה- EEGNet Model

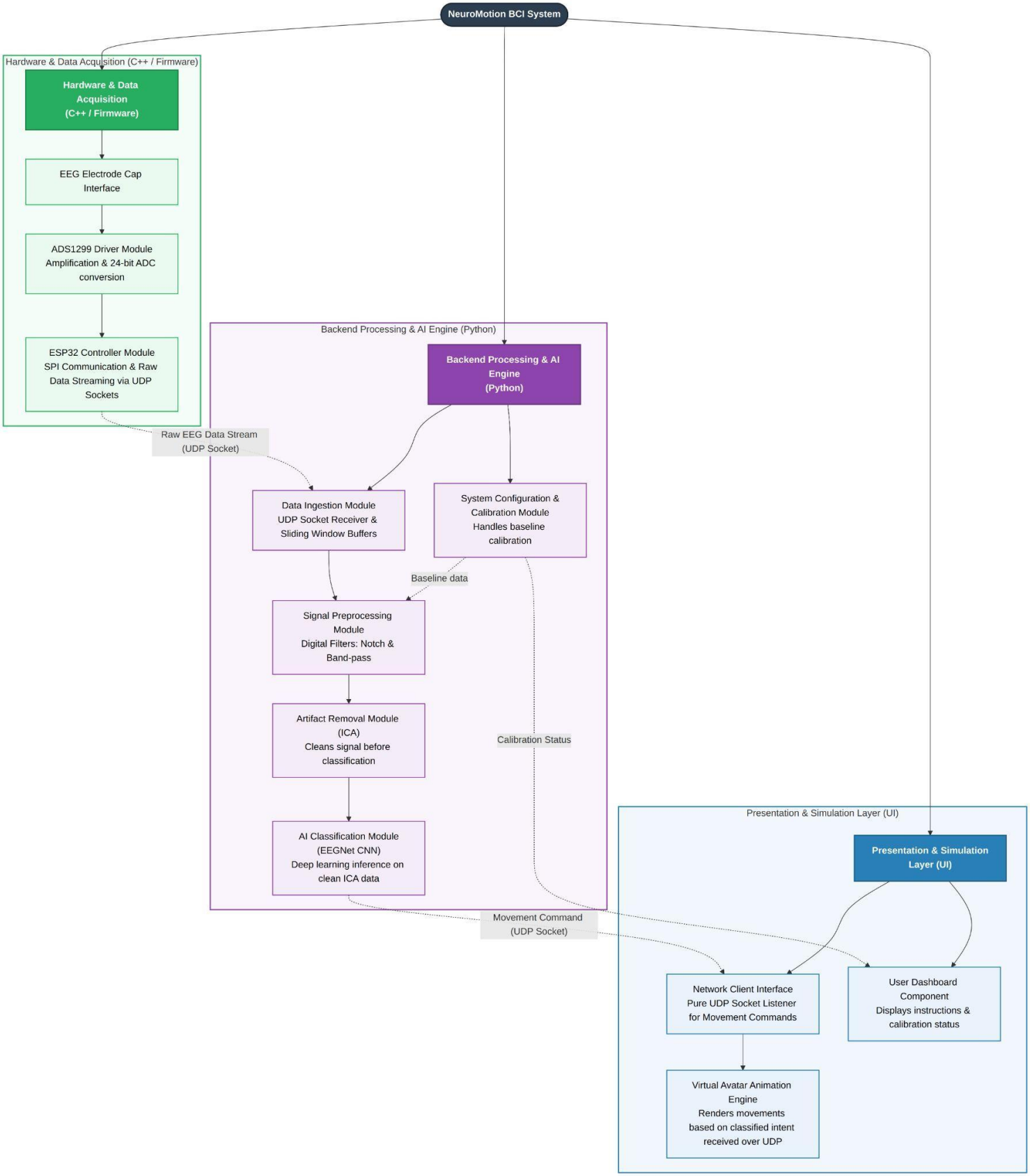
- **סוג הממשק:** ממשק תוכנה פנימי (Internal Memory API) והזרמת טנסורים (Tensor Streaming).
- **תיאור זרימה:** לאחר שרכיב עיבוד האות בפייתון מסיים את הסינון הדיגיטלי ואת אלגוריתם ה-ICA, הוא מייצר מטריצת NumPy נקייה המייצגת את חלון הזמן הנע.

רכיב זה מזרים באופן מיידי את המטריצה לפונקציית ההסקה של מודל ה-EEGNet. המודל מקבל את הטנסור, מבצע את פעולות הקונבולוציה המרחביות והזמניות בזיכרון ה-RAM, ומחזיר למערכת וקטור נורמלי המכיל את הסתברויות הסיווג עבור המחלקות השונות.

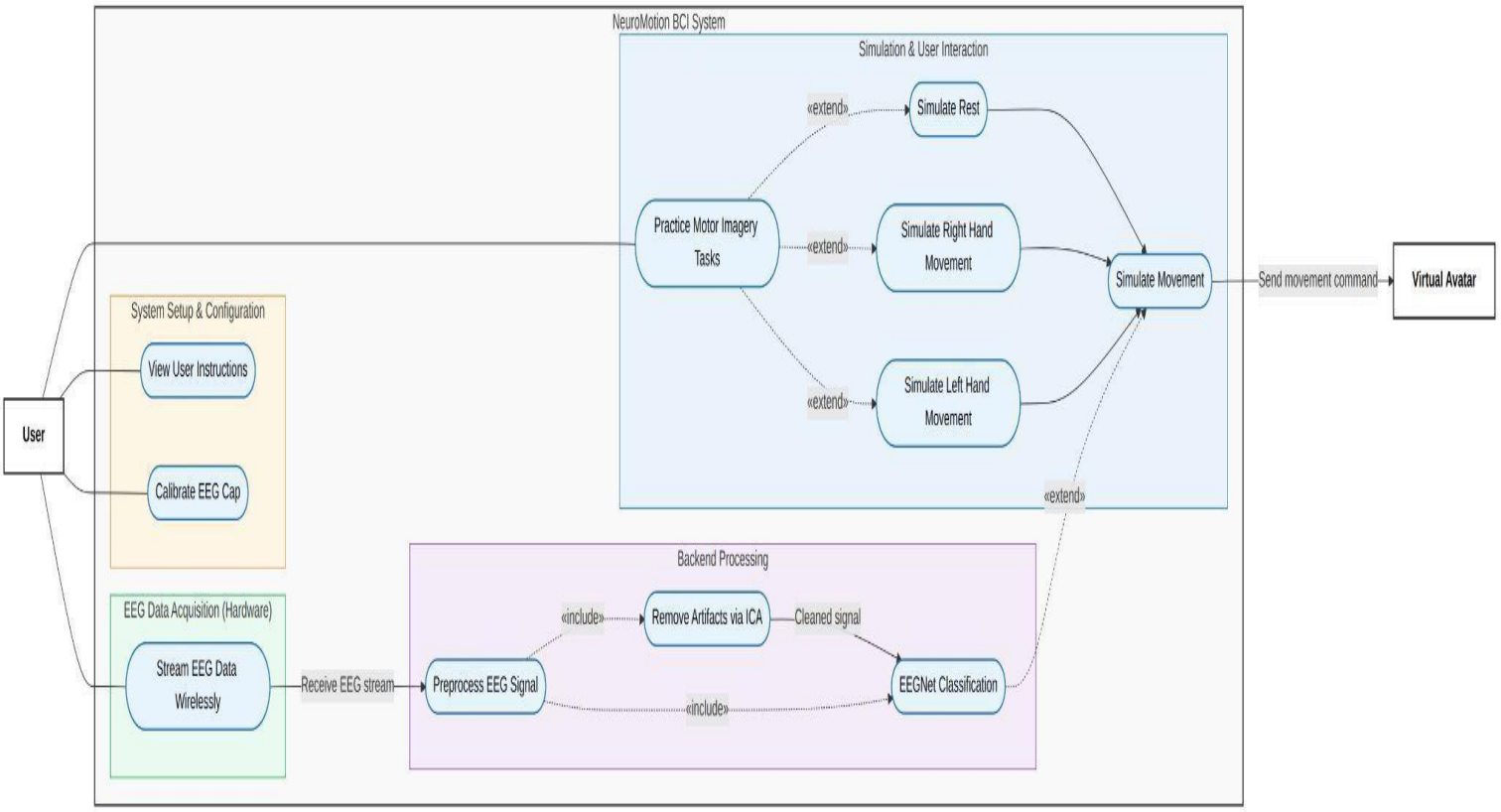
4. הקשר בין שרת העיבוד הראשי (Python Backend) לממשק האוואטאר הווירטואלי (Frontend UI)

- **סוג הממשק:** ממשק תקשורת רשת מבוסס פרוטוקול **UDP Sockets / WebSockets** להבטחת הגעת פקודות.
- **תיאור זרימה:** כאשר שרת ה-Python מזהה שווקטור ההסתברויות של ה-EEGNet עבר את סף המובהקות, הוא מייצר פקודת תנועה רשמית. מאחר שחשוב מאוד שהפקודות ישלחו בזמן ברשת, השרת משתמש בפרוטוקול UDP

עץ מודולים


NeuroMotion BCI System
Hardware & Data Acquisition (C++ / Firmware)
Hardware & Data Acquisition (C++ / Firmware)
EEG Electrode Cap Interface
ADS1299 Driver Module Amplification & 24-bit ADC conversion
ESP32 Controller Module SPI Communication & Raw Data Streaming via UDP Sockets
Raw EEG Data Stream (UDP Socket)
Backend Processing & AI Engine (Python)
Backend Processing & AI Engine (Python)
Data Ingestion Module UDP Socket Receiver & Sliding Window Buffers
System Configuration & Calibration Module Handles baseline calibration
Baseline data
Signal Preprocessing Module Digital Filters: Notch & Band-pass
Artifact Removal Module (ICA) Cleans signal before classification
AI Classification Module (EEGNet CNN) Deep learning inference on clean ICA data
Calibration Status
Movement Command (UDP Socket)
Presentation & Simulation Layer (UI)
Presentation & Simulation Layer (UI)
Network Client Interface Pure UDP Socket Listener for Movement Commands
User Dashboard Component Displays instructions & calibration status
Virtual Avatar Animation Engine Renders movements based on classified intent received over UDP

USE CASE DIAGRAM



USE CASES רשימת

1. שכבת הגדרות ותצורת מערכת

UC -1: צפייה בהוראות שימוש למשתמש

- **שחקן ראשי:** המשתמש (User).
- **תיאור קצר:** המשתמש מפעיל את ממשק הניהול וצופה בהנחיות ובסרטוני ההסבר הויזואליים המנחים אותו כיצד לבצע את משימות הדמיון המוטורי (Motor Imagery) בצורה נכונה כדי להפיק אותות מוח מיטביים.
- **תנאי קדם:** מסך הממשק הגרפי (Frontend UI) פתוח ונגיש.

2. שכבת איסוף הנתונים והחומרה (EEG Data Acquisition)

2-UC: הזרמת אותות EEG אלחוטית בזמן אמת

- **שחקן ראשי:** המשתמש (User) / חומרת הקצה.
- **תיאור קצר:** שבב ה-1299ADS דוגם את הפוטנציאלים החשמליים הזעירים מהאלקטרודות בקצב קבוע, ובקר ה-32ESP מושך את המידע, אורז אותו במבנה בינארי קשיח, ומשדר אותו מיידית באוויר בסטרימינג רציף באמצעות פרוטוקול **UDP** ישירות אל שרת ה-Python Backend.
- **תנאי קדם:** ערוץ ה-Wi-Fi של המיקרו-בקר מחובר לרשת המקומית.

3. שכבת עיבוד ה-Backend והבינה המלאכותית (Backend Processing & AI)

3- UC: עיבוד מקדים של אות ה-EEG (Preprocess EEG Signal)

- **שחקן ראשי:** מערכת ה-Backend (שרת ה-Python).
- **תיאור קצר:** השרת מאזין ל-UDP Socket, קולט את חבילות המידע הבינאריות, מבצע להן דה-סריאליזציה ומפעיל מסנני תדר דיגיטליים (כגון Band-pass) על מנת לחתוך תדרים לא רלוונטיים ולבודד את גלי ה-Mu וה-Beta הנדרשים לניתוח הדמיון המוטורי.
- **תנאי קדם:** זרם הנתונים מהחומרה מגיע בצורה רציפה ותקינה.

4-UC: הסרת ארטיפקטים וניקוי אותות באמצעות ICA

- **שחקן ראשי:** מערכת ה-Backend (שרת ה-Python).
- **תיאור קצר: רכיב קריטי הרץ לפני שלב הסיווג.** אלגוריתם ה-ICA מפרק את האותות הרב-ערוציים שנקלטו לרכיבים עצמאיים, מזהה אוטומטית רעשים ביולוגיים שאינם ממקור מוחי (כמו מצמוצי עיניים או תנועות שרירי פנים), מאפס אותם, ומשחזר אות EEG נקי לחלוטין.
- **תנאי קדם:** חלון זמן של אותות עבר עיבוד מקדים (4-UC) בהצלחה.

5-UC: סיווג כוונת התנועה ברשת EEGNet Classification

- **שחקן ראשי:** מערכת ה-Backend.
- **תיאור קצר:** חלון הזמן של האות הנירלי המטוהר (לאחר ה-ICA) מוזרם אל רשת העצבים הקונבולוציונית **EEGNet CNN**. הרשת מחלצת מאפיינים מרחביים ותדריים מהאות ומפיקה וקטור הסתברויות המגדיר לאיזו תנועה התכוון המשתמש.
- **תנאי קדם:** האות עבר ניקוי ארטיפקטים מלא (5-UC).

4. שכבת הסימולציה והאינטראקציה

6-UC: תרגול משימות דמיון מוטורי

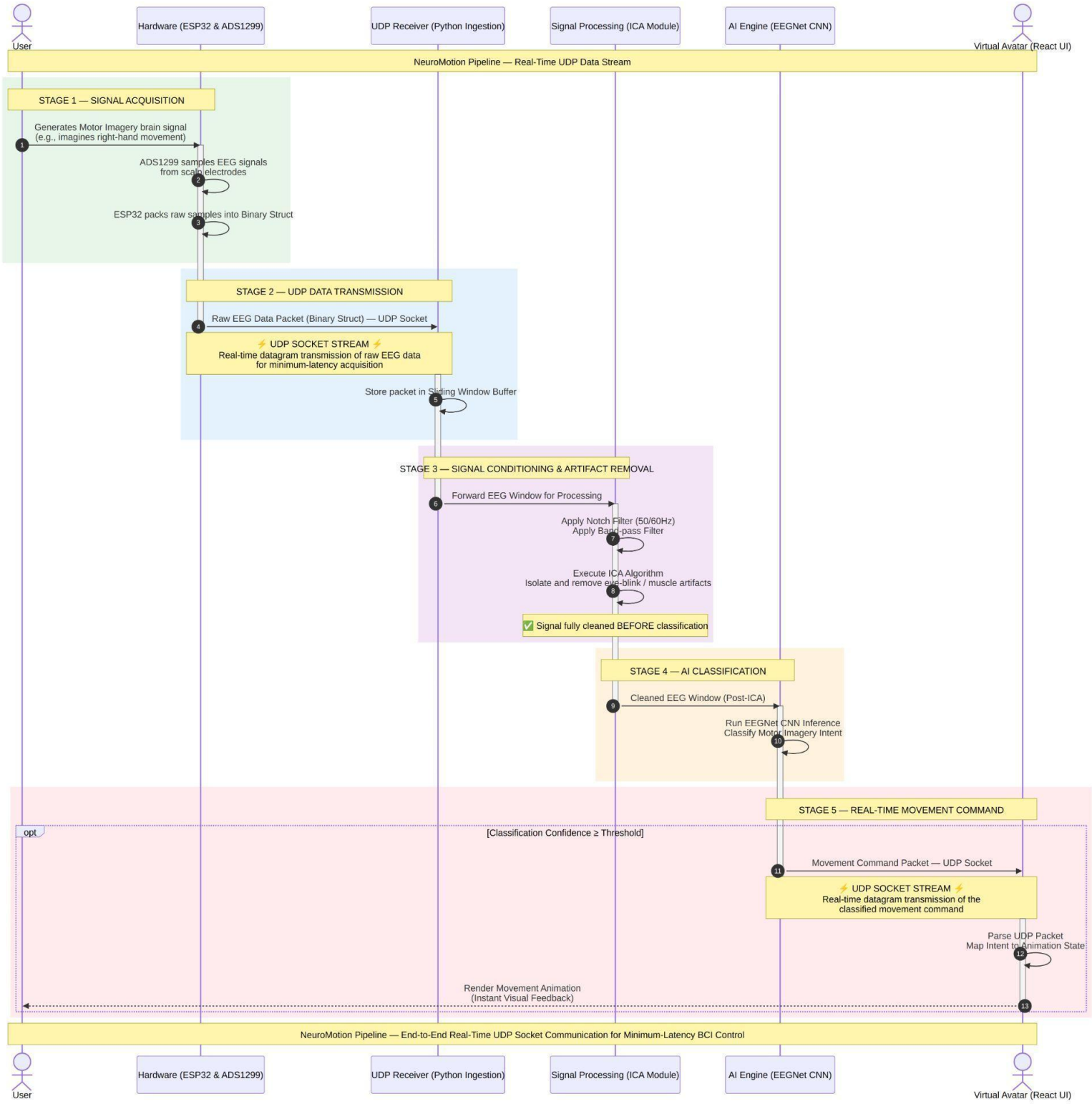
- **שחקן ראשי:** המשתמש (User).

- **תיאור קצר:** המשתמש מגיב לגירויים חזותיים המופיעים על המסך ומבצע באופן אקטיבי "מחשבה על תנועה" (דמיון של הזזת יד ימין, יד שמאל או הישארות במצב מנוחה) כדי לבחון את רמת השליטה שלו במערכת.
- **תנאי קדם:** המערכת נמצאת במצב ריצה (סשן תרגול פעיל).

UC-7: סימולציית תנועת האוואטאר

- **שחקן משני:** האוואטאר הווירטואלי .
- **תיאור קצר:** כאשר מודל ה-EEGNet מסווג בהצלחה כוונת תנועה מובהקת (מעבר לסף ה-Threshold), שרת ה-Python שולח פקודת תנועה מיידית בפרוטוקול **UDP** (למניעת השהיות). ממשק הקצה קולט את הפקודה ומפעיל את מנוע האנימציות של האוואטאר (הזזת יד ימין, שמאל או עצירה) בזמן אמת.
- **תנאי קדם:** מודל ה-EEGNet זיהה כוונת תנועה ועבר את סף הוודאות הנדרש (UC-6).

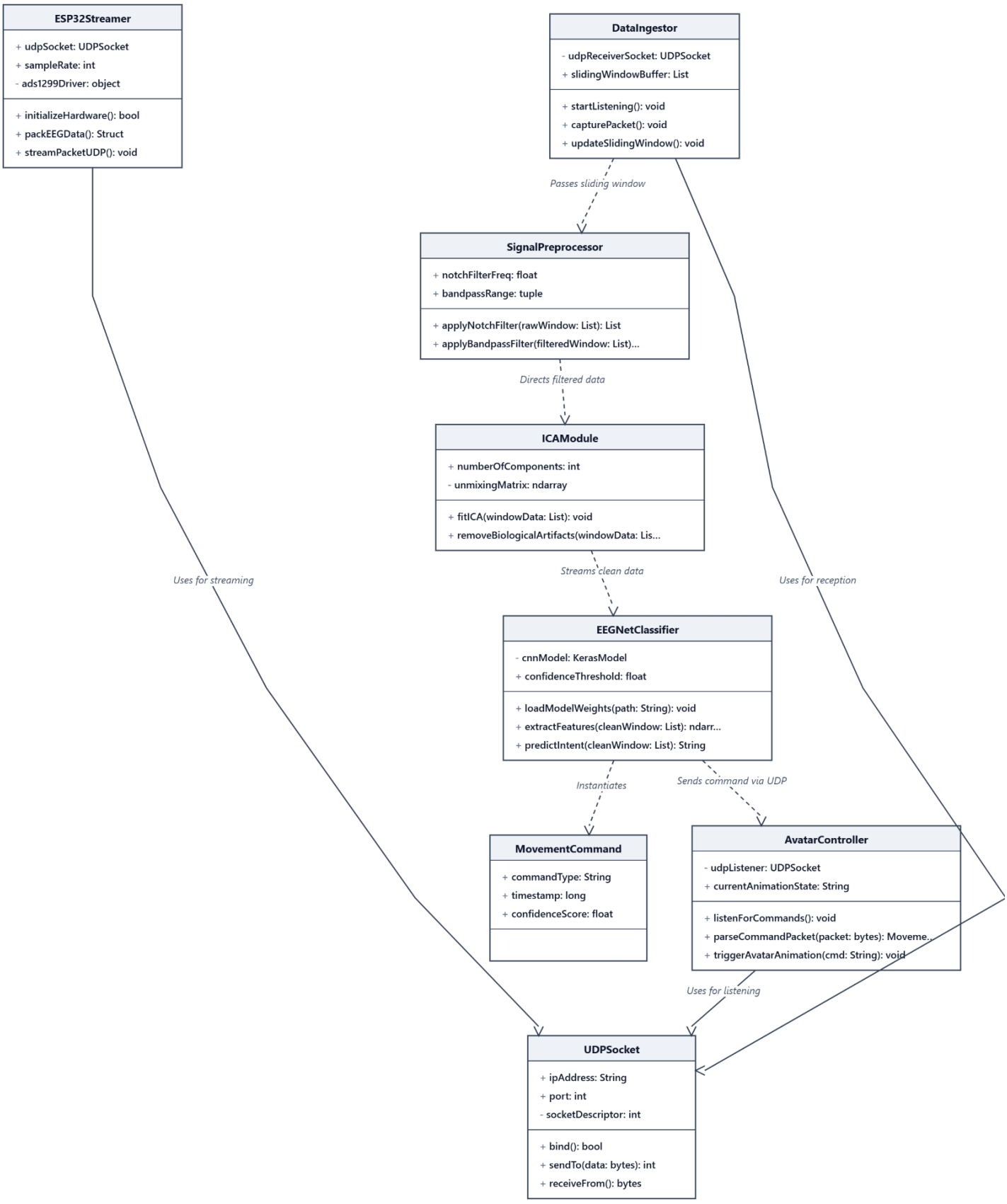
UMLתרשים


User
Hardware (ESP32 & ADS1299)
UDP Receiver (Python Ingestion)
Signal Processing (ICA Module)
AI Engine (EEGNet CNN)
Virtual Avatar (React UI)
NeuroMotion Pipeline — Real-Time UDP Data Stream
STAGE 1 — SIGNAL ACQUISITION
Generates Motor Imagery brain signal (e.g., imagines right-hand movement)
ADS1299 samples EEG signals from scalp electrodes
ESP32 packs raw samples into Binary Struct
STAGE 2 — UDP DATA TRANSMISSION
Raw EEG Data Packet (Binary Struct) — UDP Socket
⚡ UDP SOCKET STREAM ⚡
Real-time datagram transmission of raw EEG data for minimum-latency acquisition
Store packet in Sliding Window Buffer
STAGE 3 — SIGNAL CONDITIONING & ARTIFACT REMOVAL
Forward EEG Window for Processing
Apply Notch Filter (50/60Hz)
Apply Band-pass Filter
Execute ICA Algorithm
Isolate and remove eye-blink / muscle artifacts
✅ Signal fully cleaned BEFORE classification
STAGE 4 — AI CLASSIFICATION
Cleaned EEG Window (Post-ICA)
Run EEGNet CNN Inference
Classify Motor Imagery Intent
STAGE 5 — REAL-TIME MOVEMENT COMMAND
opt
[Classification Confidence ≥ Threshold]
Movement Command Packet — UDP Socket
⚡ UDP SOCKET STREAM ⚡
Real-time datagram transmission of the classified movement command
Parse UDP Packet
Map Intent to Animation State
Render Movement Animation
(Instant Visual Feedback)
NeuroMotion Pipeline — End-to-End Real-Time UDP Socket Communication for Minimum-Latency BCI Control

Design class Diagram



תרשים מחלקות



תיאור המחלקות המוצעות

א. מבט על ארכיטקטוני (High-Level Overview)

מערכת **NeuroMotion** בנויה כצינור עיבוד ליניארי אסינכרוני בזמן אמת . הארכיטקטורה מחולקת לארבע שכבות לוגיות מרכזיות: חומרה (Hardware), תקשורת (Networking), עיבוד ובינה מלאכותית (Backend Processing & AI), וממשק תצוגה (Presentation/UI).

המערכת מתוכננת להשגת השהיה מינימלית באמצעות שימוש בפרוטוקול **UDP** ומבנה נתונים יעיל, תוך שמירה על רמת סינון אות גבוהה (באמצעות ICA) לפני שלב הסיווג ברשת הניורונית (EEGNet).

ב. פירוט יחידות המערכת: תפקיד, קלטים ופלטים

1. יחידת החומרה והשידור (ESP32Streamer)

- **תפקיד היחידה:** ניהול רכיבי החומרה בקצה. היא אחראית על תזמון דגימת האותות החשמליים האנלוגיים מהקרקפת, המרתן לאות דיגיטלי, אריזת הנתונים הגולמיים במבנה בינארי קשיח (`Struct`) ודחיפתם המיידית לאוויר.
- **קלטים:** אותות מיקרו-וולט אנלוגיים ישירות מחיישני כובע ה-EEG דרך שבב ה-1299ADS.
- **פלטים:** חבילות בתים בינאריות (Raw Data Packets) המשודרות ברשת האלחוטית.

2. יחידת מנוע התקשורת (UDPSocket)

- **תפקיד היחידה:** מחלקת תשתית עצמאית המספקת ממשק לעבודה מול כרטיס הרשת. היא מנהלת את פתיחת הסוקטים, הקישור לפורטים (Binding) והעברת הנתונים הלוך וחזור בפרוטוקול UDP ללא שום Overhead של לחיצות ידיים או ניהול תורים (בניגוד ל-TCP/WebSockets).
- **קלטים:** מערכי בתים (Bytes) מהחומרה או מה-Backend, בשילוב כתובת IP ופורט יעד.
- **פלטים:** שליחה וקליטה של חבילות מידע פיזיות ברשת התקשורת.

3. יחידת קליטת הנתונים (DataIngestor)

- **תפקיד היחידה:** רכיב קצה אחורי המאזין בצורה אסינכרונית לפורט ה-UDP. תפקידו לקלוט את זרם החבילות הבינאריות מהחומרה, לבצע להן דה סריאליזציה, ולנהל חלון זמן נע (Sliding Window Buffer) המחזיק באופן רציף את רצף הדגימות העדכני ביותר הנדרש לעיבוד.
- **קלטים:** חבילות נתונים בינאריות המגיעות מה-UDPSocket של החומרה.
- **פלטים:** חלון זמן מרוכז של אותות EEG גולמיים רב-ערוציים.

4. יחידת עיבוד האות המקדים (SignalPreprocessor)

- **תפקיד היחידה:** ביצוע מניפולציות וסינונים מתמטיים-דיגיטליים ראשוניים על חלון הזמן הנע, במטרה לבודד את תדרי העניין (גלי מוח מסוג Mu ו-Beta) ולחתוך רעשים קבועים כגון תדר רשת החשמל הביתית (Hz50).
- **קלטים:** חלון זמן של אותות EEG גולמיים מיחידת ה-DataIngestor.
- **פלטים:** חלון זמן של אותות מסוננים דיגיטלית בתדרים מוגדרים (Filtered EEG Window).

5. יחידת הסרת ארטיפקטים (ICAModule)

- **תפקיד היחידה (רכיב קריטי - מופעל לפני הסיווג):** הפעלת אלגוריתם ICA. היחידה מפרקת את האות הרב-ערוצי לרכיבי מקור עצמאיים, מזהה ומאפסת באופן אוטומטי רעשים וארטיפקטים ממקור ביולוגי שאינו מוחי (כגון מצמוצי עיניים, תנועות פנים וכיווצי שרירים), ומשחזרת אות נקי ומטוהר.
- **קלטים:** חלון זמן של אותות מסוננים דיגיטלית המגיעים מה-SignalPreprocessor.
- **פלטים:** חלון זמן של אות EEG מטוהר ונקי לחלוטין מרעשי שרירים ועיניים (Clean EEG Window).

6. יחידת סיווג הבינה המלאכותית (EEGNetClassifier)

- **תפקיד היחידה:** מנוע קבלת ההחלטות של המערכת. היא טוענת את רשת העצבים הקונבולוציונית העמוקה (EEGNet CNN), מזינה לתוכה את חלון האות הנקי, ומחלצת את כוונת התנועה הספציפית של המשתמש. במידה והסיווג עובר את סף הוודאות הנדרש, היא יוצרת פקודה להנעת האוואטאר.
- **קלטים:** אות EEG מטוהר (Clean EEG Window) המגיע ישירות מה-ICAModule.
- **פלטים:** אובייקט נתונים מסוג פקודת תנועה (MovementCommand).

7. יחידת אובייקט פקודת התנועה (MovementCommand)

- **תפקיד היחידה:** מחלקת נתונים בלבד (DTO - Data Transfer Object) המשמשת כאמצעי אחיד ומוגדר מראש להעברת מידע על כוונת המשתמש בין שכבת ה-Backend (פייתון) לשכבת הממשק הגרפי (React).
- **קלטים:** סוג התנועה שסווגה (מנוחה, יד ימין, יד שמאל), מדד וודאות וחומת זמן (נוצר ע"י ה-Classifier).
- **פלטים:** מבנה נתונים מובנה המומר למערך בתים לשם משלוח מהיר ברשת.

8. יחידת בקר האוואטאר וממשק המשתמש (AvatarController)

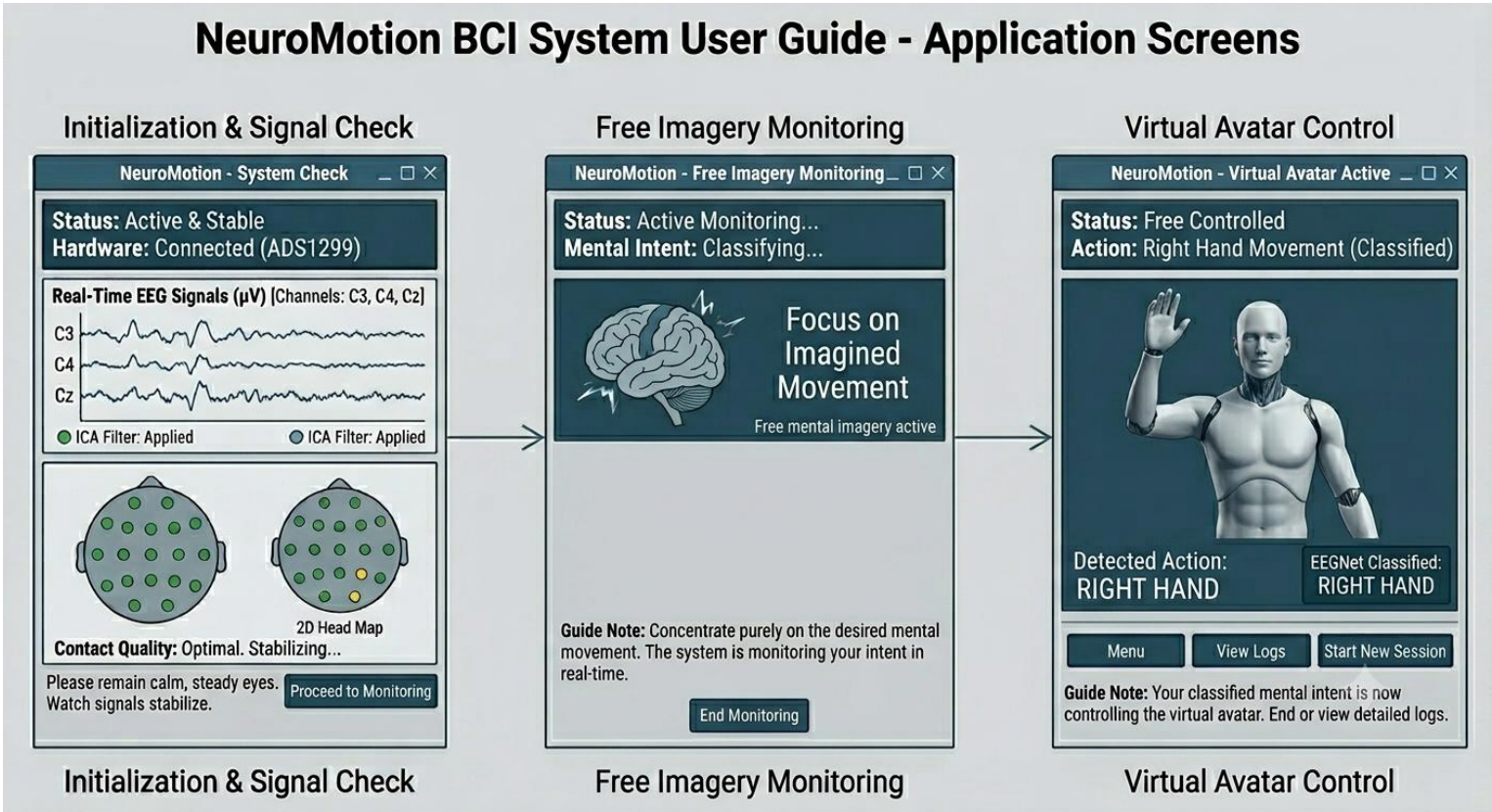
- **תפקיד היחידה:** יחידת הממשק הגרפי (UI Layer). היא מאזינה באופן קבוע לפורט ה-UDP המקומי שלה, קולטת ומפרשת את פקודות התנועה המגיעות מה-Backend ומפעילה באופן מיידי את מנוע האנימציות הויזואלי המציג למשתמש את תנועת האוואטאר הווירטואלי בהתאם למחשבתו.
- **קלטים:** חבילות רשת (UDP Datagrams) המכילות את פקודת התנועה המפוענחת.
- **פלטים:** עדכון מצב האוואטאר הגרפי על המסך (רינדור אנימציית תנועה בזמן אמת), והצגת סטטוס כיול/הוראות למשתמש.

ג. זרימת המידע בין היחידות (Data Flow Timeline)

זרם המידע במערכת מתבצע מקצה לקצה בצורה הבאה:

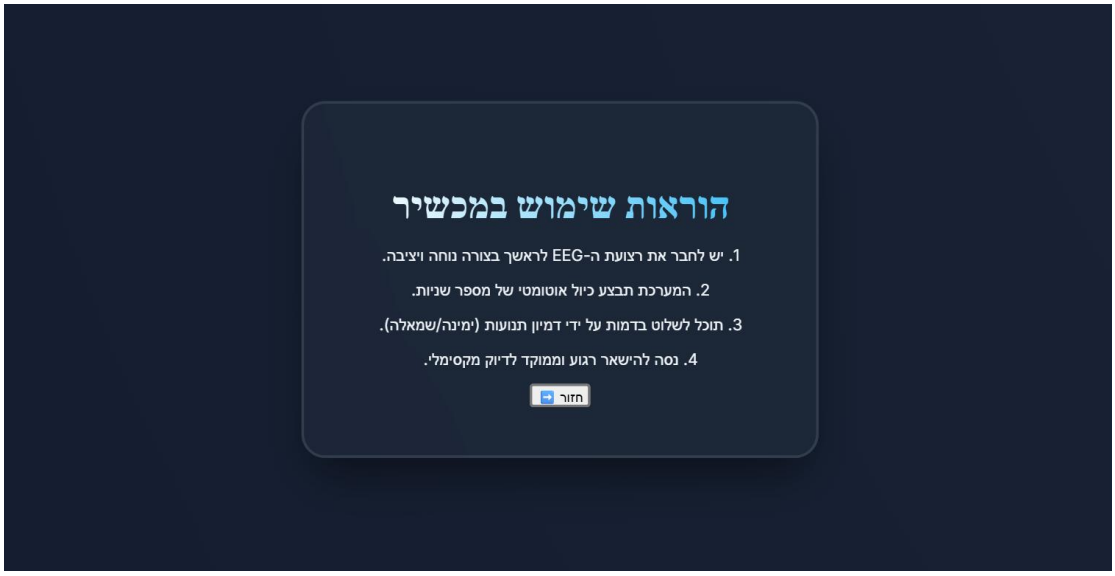
1. **ייצור האות:** המשתמש מרוכז במשימת דמיון מוטורי שיוצרת שינויים פוטנציאליים בקרקפת. האות נדגם על ידי ה-`ESP32Streamer`.
2. **שידור גולמי:** ה-`ESP32Streamer` משתמש ב-`UDPSocket` ודוחף חבילות בתים בינאריות ברשת האלחוטית אל עבר ה-Backend.
3. **קליטה ובאפרינג:** ה-`DataIngestor` קולט את הבתים באמצעות ה-`UDPSocket` שלו, מתרגם אותם חזרה למספרים, ומעדכן את ה-Sliding Window Buffer.
4. **ניקוי וסינון האות:** חלון הזמן הנע נשלח תחילה ל-`SignalPreprocessor` לסינון תדרים, ומשם מוזרם מיד ל-`ICAModule` המנקה רעשי שרירים ומצמוצים.
5. **סיווג וקבלת החלטה:** האות הנירלי הסטרילי והנקי מועבר ל-`EEGNetClassifier`. מודל ה-CNN מפענח את כוונת המשתמש ומייצר אובייקט `MovementCommand`.
6. **שידור הפקודה:** ה-`EEGNetClassifier` משגר את ה-`MovementCommand` כחבילת UDP מהירה ישירות לשכבת התצוגה.
7. **אינטראקציה ורינדור:** ה-`AvatarController` קולט את החבילה, מפרש את הפקודה ומניע את האוואטאר הוירטואלי על המסך במינימום זמן והשהיה.

תרשים מסכים



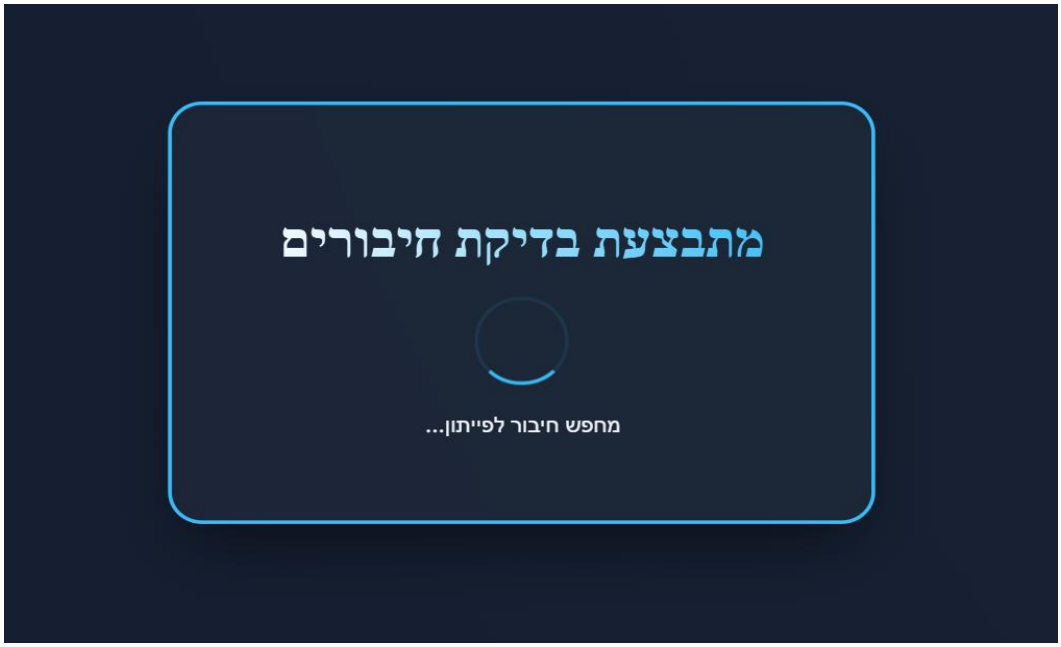
פירוט כל מסך

תפקידו של המסך הראשון זה לתת הוראות שימוש למערכת



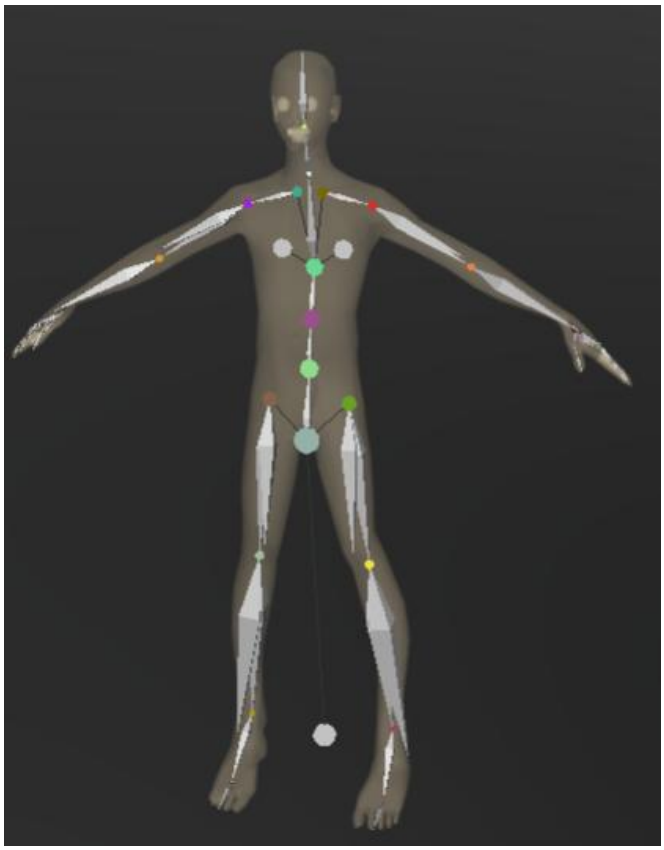
תפקידו של המסך השני זה לבדוק מאחורי הקלעים שיש חיבור

לוח הבקרה מהווה את נקודת המפגש הראשונית של המשתמש עם המערכת. תפקידו העיקרי הוא לספק חיווי בזמן אמת על מצב החיבור האלחוטי מול כרטיס החומרה (32ESP), לוודא קליטה תקינה של חבילות המידע בפרוטוקול **UDP**, ולהציג מדדי קצב דגימה נוכחיים כדי להבטיח את יציבות המערכת לפני תחילת העבודה.



תפקידו של המסך הבא הוא להראות את האווטאר בפעולה אשר הוסקה ממכונת הסיווג.





אלמנטים של תצוגה

כפתור בדיקת חיבורים

- תפקיד האלמנט: הפעלת בדיקת סטטוס יציבות תקשורת ה-UDP מול רכיב ה- Data Ingestor ב-Backend.
- תיאור פעולה: בלחיצה על כפתור זה, שכבת הממשק שולחת שאילתת בקרה פנימית כדי לוודא שחבילות המידע הגולמיות מה-32ESP זורמות כסדרן לפורט המוגדר. האלמנט משנה את מצבו הויזואלי (Loading) בזמן הבדיקה, ומחזיר חיווי מיידי על קצב הדגימה המאומת (Hz250).

כפתור חזור

- תפקיד האלמנט: ניווט קל וחזרה למסך הקודם במערכת.
- תיאור פעולה: מאחר והמערכת בנויה כצינור עיבוד ליניארי אסינכרוני, כפתור זה מאפשר למשתמש או למפקח הרפואי לחזור שלב אחד אחורה למשל: מסביבת הסימולציה והתרגול בחזרה למסך כיול האלקטרודות.

כפתור סיים וצא

- תפקיד האלמנט: סגירה בטוחה של סשן התרגול הנוכחי ואיפוס רכיבי התצוגה.
- תיאור פעולה: לחיצה על כפתור זה עוצרת אקטיבית את האזנת ה-UI לחבילות ה-UDP המגיעות מהסיווג של ה-EEGNet, מאפסת את מיקום האוואטאר הווירטואלי למצב מוצא (Rest), ומנווטת את האפליקציה חזרה למסך הראשי.

הודעות ומערך התראות למשתמש

בשל אופייה של המערכת כמערכת זמן אמת מבוססת פרוטוקול UDP בלבד וללא מנגנון שליחה חוזרת, מערך ההתראות חיוני לשמירה על רציפות העבודה ותקינות הניסוי. במקרה שבו זרם הנתונים מהחומרה נעצר ולא מתקבלות חבילות UDP ברכיב ה-Backend למשך יותר מ-1.5 שניות רצופות, המערכת מפעילה באופן אוטומטי התראת שגיאה קריטית (Danger / Error) ומקפיצה למשתמש הודעה מיידית בנוסח: **שגיאת תקשורת: זרם הנתונים מהחומרה נעצר. נא לבדוק את חיבור ה-Wi-Fi של ה-ESP32.**

ממשק משתמש

ממשק המשתמש של מערכת **NeuroMotion** פותח תחת עקרונות הנדסת אנוש קפדניים (UI/UX), תוך התחשבות בקהל היעד (אנשים עם מוגבלויות תנועה או שיתוק) ובצרכים של צינור עיבוד נתונים אסינכרוני מהיר:

- **עיצוב מונע השהיה מינימלית (Zero-Latency Rendering):** הממשק משתמש ברכיבי React אופטימליים המאזינים לסטרטר המידע האסינכרוני. עדכון האנימציה

של האוואטאר מתבצע ישירות ב-State מקומי ברגע שחבילת UDP של פקודת תנועה מתקבלת, כדי למנוע השהיה ויזואלית (Visual Lag).

- **ניגודיות חזותית גבוהה (High Contrast & Accessibility):** שימוש בצבעים מובחנים וברורים (כמו חצי גירוי ירוקים גדולים למשימות דמיון מוטורי ועיגול אדום למנוחה). הניגודיות הגבוהה קריטית כדי למנוע עייפות קוגניטיבית של המשתמש במהלך סשן תרגול ממושך.
- **מניעת עומס קוגניטיבי (Minimalistic UI):** המסכים נקיים מאלמנטים גרפיים מיותרים ומסיחי דעת. תשומת הלב של המשתמש ממוקדת אך ורק בגירויים הויזואליים ובאוואטאר, מה שמבטיח הפקה ממוקדת של גלי מוח מסוג Mu ו-Beta ללא רעשי קשב חיצוניים.

קוד התוכנית

בתחילה האות הגולמי עובר ניקוי של מסנן חריץ-Notch Filter:

```
def notch_filter(X, freq=50.0, fs=SFREQ, quality=30):
    b, a   = iirnotch(freq / (fs / 2), quality)
    X_filt = np.zeros_like(X)
    for i in range(X.shape[0]):
        for ch in range(X.shape[1]):
            X_filt[i, ch] = filtfilt(b, a, X[i, ch])
    return X_filt
```

הקלט שנכנס: אות ה-EEG הגולמי (Raw EEG) במימדים של 16 ערוצי מידע רציפים לאורך ציר הזמן, כפי שנקלטו במקור משני שבבי ה-1299ADS בקצב דגימה קבוע של 250Hz.

הפלט שיוצא: אות EEG רציף במימדים זהים, שבו רצועת התדר הספציפית והקבועה של 50Hz מונחתת ומנוטרלת לחלוטין.

איך הקוד עובד: קטע קוד זה מאתחל ומפעיל מסנן חריץ דיגיטלי (IIR Notch Filter) בעל מקדם איכות (Q) גבוה מאוד באמצעות ספריית SciPy. תפקידו הבלעדי של פילטר זה במערכת הוא לבודד ולהסיר את הרעשים האלקטרומגנטיים (Power-line noise) המושרים על חוטי האלקטרודות מהסביבה ומרשת החשמל הארצית הפועלת בתדר קבוע של 50Hz. המסנן מתפקד כפתיל חסימה צר ביותר (Narrow Band-stop), המבטיח את הסרת ההפרעה החשמלית האופיינית בצורה חדה, מבלי לפגוע או לעוות את הרקע הנירלי של אותות המוח הנמצאים בתדרים השכנים לו.

לאחר מכן האות עובר לניקוי של BandPass:

```
def bandpass_filter(X, lowcut=1.0, highcut=40.0, fs=SFREQ, order=4):
    nyq  = fs / 2
    b, a = butter(order, [lowcut / nyq, highcut / nyq], btype='band')
    X_filt = np.zeros_like(X)
    for i in range(X.shape[0]):
        for ch in range(X.shape[1]):
            X_filt[i, ch] = filtfilt(b, a, X[i, ch])
    return X_filt
```

הקלט שנכנס: אות ה-EEG המטוהר שעבר בשלב הקודם את מסנן החריץ (Notch Filtered EEG) לצורך ניקוי רשת החשמל.

הפלט שיוצא: אות EEG מסונן ומטוהר מרעשי תדר קיצוניים, המוגבל במדויק אך ורק לתחום התדרים הרלוונטי שבין Hz1 ל-Hz40.

איך הקוד עובד: הקוד מגדיר ומיישם פילטר דיגיטלי מעביר-פס (Bandpass) מסוג Butterworth מסדר רביעי, ומריץ אותו באמצעות פונקציית הסינון הדו-כיוונית `filtfilt` (Zero-phase filtering). פעולה דו-כיוונית זו היא קריטית מאחר שהיא מונעת לחלוטין הסטת פאזה (Phase Shift) בציר הזמן, ומבטיחה שאותות המוח ישמרו על סנכרון זמני מושלם בין 16 הערוצים השונים. תדר החיתוך התחתון (Hz1) מסיר דריפטים איטיים של קו הבסיס הנובעים מתנודות פיזיות של המשתמש או שינויי מוליכות של העור, ותדר החיתוך העליון (Hz40) מנחית רעשי שרירים (EMG) מהירים, מה שמבודד בצורה אופטימלית את גלי אלפא ובטא הנדרשים לפענוח דמיון התנועה (Motor Imagery).

לאחר מכן האות עובר אל תוך ה-ICA שהוא מודל לניתוח רכיבים עצמאיים:

```python
def remove_artifacts_ica(X, fs=SFREQ, n_components=None):
    n_trials, n_channels, n_times = X.shape
    if n_components is None:
        n_components = n_channels
    X_concat = X.transpose(1, 0, 2).reshape(n_channels, -1).T
    ica      = FastICA(n_components=n_components, random_state=42, max_iter=500)
    sources  = ica.fit_transform(X_concat)
    bad_components = []
    for comp_idx in range(n_components):
        comp_signal = sources[:, comp_idx]
        freqs       = np.fft.rfftfreq(n_times, d=1.0/fs)
        psd_total   = np.zeros(len(freqs))
        for trial in range(n_trials):
            seg = comp_signal[trial * n_times:(trial + 1) * n_times]
            psd_total += np.abs(np.fft.rfft(seg)) ** 2
        psd_total /= n_trials
        total_power = psd_total.sum() + 1e-10
        eog_ratio   = psd_total[(freqs >= 1)  & (freqs <= 4)].sum()  / total_power
        emg_ratio   = psd_total[freqs >= 20].sum()                   / total_power
        motor_ratio = psd_total[(freqs >= 8)  & (freqs <= 30)].sum() / total_power
        if (eog_ratio > 0.6 and motor_ratio < 0.2) or \
           (emg_ratio > 0.7 and motor_ratio < 0.1):
            bad_components.append(comp_idx)
    mixing        = ica.mixing_
    sources_clean = sources.copy()
    sources_clean[:, bad_components] = 0
    X_clean_concat = sources_clean @ mixing.T
    X_clean = X_clean_concat.T.reshape(n_channels, n_trials, n_times).transpose(1, 0, 2)
    return X_clean
```

הקלט שנכנס: מטריצת אותות ה-EEG לאחר סינוני התדר הדיגיטליים, המיוצגת במבנה תלת-מימדי המכיל את מספר הניסויים, 16 ערוצי הרישום, ונקודות הדגימה לאורך ציר הזמן.

הפלט שיוצא: מטריצת אותות נקייה ומטוהרת באותם המימדים המקוריים, שבה הרכיבים הסטטיסטיים שסווגו כרעשים פיזיולוגיים לא-מוחיים אופסו לחלוטין והוחרגו מתהליך שחזור האות.

איך הקוד עובד: הפונקציה (remove_artifacts_ica) מיישמת אלגוריתם אוטומטי להפרדת מקורות עיוורת המפרק את האותות ל-16 רכיבים סטטיסטיים עצמאיים. בשלב השני, הקוד מנתח את התוכן התדרי של כל רכיב באמצעות התמרת פורייה מהירה (FFT) כדי לחשב את צפיפות ההספק שלו. המערכת בודקת באופן אוטומטי את יחסי האנרגיה: רכיבים בעלי הספק דומיננטי בתדרים נמוכים מאוד (1-4 הרץ) מסווגים כרעשי מצמוץ ותנועות עיניים, ורכיבים בעלי הספק בתדרים גבוהים (מעל 20 הרץ) מסווגים כרעשי שרירים ולסת. רכיבי הרעש מאופסים לחלוטין, והאות הנקי נבנה מחדש אל מרחב 16 הערוצים המקוריים באמצעות כפל במטריצת הערבוב ההפוכה.

לאחר מכן מופעלת על האותות הנקיים פונקציית הנירמול הבאה:

```
def normalize_per_subject(X, sub_encoded):
    X_norm = X.copy()
    for s in np.unique(sub_encoded):
        mask = sub_encoded == s
        Xi   = X[mask].reshape(mask.sum(), -1)
        X_norm[mask] = StandardScaler().fit_transform(Xi).reshape(
            mask.sum(), *X.shape[1:])
    return X_norm
```

הקלט שנכנס: מטריצת אותות ה-EEG המטוהרת לאחר שלב ה-ICA, לצד וקטור המכיל את מזהי הנבדקים התואמים לכל ניסוי וניסוי.

הפלט שיוצא: מטריצת אותות מנורמלת באותם המימדים המקוריים, שבה הנתונים של כל נבדק עברו התאמה סטטיסטית עצמאית ומבודדת.

איך הקוד עובד: הפונקציה מבצעת לולאה על פני כל הנבדקים הייחודיים הקיימים במאגר המידע. עבור כל נבדק בנפרד, היא מבודדת את הנתונים שלו, מחשבת את הממוצע ואת סטיית התקן של האותות, ומפעילה עליהם נרמול סטנדרטי (StandardScaler). תהליך זה מביא את האותות של כל אדם לממוצע אפס וסטיית תקן אחת. זהו שלב הנדסי קריטי שמנטרל הבדלים פיזיולוגיים מולדים קבועים בין אנשים (כמו עובי הגולגולת, התנגדות העור או מיקום האלקטרודות) ומאפשר לרשת ללמוד בצורה נקייה אך ורק את דפוסי כוונות התנועה עצמם.

לאחר מכן מופעלת פונקציה להרחבת נתונים ואוגמנטציה(הסבר לאחר הקוד):

```
def augment_eeg(X_batch, noise_std=0.05, shift_max=25):
    X_aug = X_batch.clone()
    X_aug = X_aug + torch.randn_like(X_aug) * noise_std
    shift = np.random.randint(-shift_max, shift_max)
    if shift != 0:
        X_aug = torch.roll(X_aug, shift, dims=-1)
    return X_aug
```

הקלט שנכנס: חלון נתונים (Batch) של אותות אי-אי-ג'י המוזרם אל תוך הרשת במהלך שלב אימון המודל.

הפלט שיוצא: חלון נתונים מורחב באותם מימדים, שבו האותות עברו שינויים אקראיים של תוספת רעש והזזות בזמן.

איך הקוד עובד: פונקציה זו מיישמת שתי טכניקות מתקדמות להרחבת נתונים בזמן אמת כדי לשפר את עמידות המודל. ראשית, היא מוסיפה לאות המקורי רעש אקראי לבן בעל עוצמה נמוכה, שמטרתו לדמות רעשי רקע סביבתיים שיכולים להתרחש בריצה האמיתית. שנית, היא מבצעת הזזה מחזורית אקראית של האות ימינה או שמאלה לאורך ציר הזמן (על ידי פקודת

roll). שילוב זה מייצר דוגמאות מלאכותיות חדשות ומאתגר את הרשת, מה שמאלץ אותה ללמוד מאפיינים יציבים ועמידים ומונע ממנה לבצע התאמת יתר על הנתונים הקיימים.

כעת, לאחר שהאות נקי מרעשים וארטיפיקטים האות מוכן להיכנס את תכונת הסיווג- אל ה-EEGNet:

```
class EEGNetClean(nn.Module):
    def __init__(self, num_classes=4, num_channels=16, num_samples=1001,
                 F1=16, D=2, F2=32):
        super().__init__()
        self.conv_temporal = nn.Conv2d(1, F1, (1, 64), padding=(0, 32), bias=False)
        self.bn1           = nn.BatchNorm2d(F1)
        self.conv_spatial  = nn.Conv2d(F1, F1*D, (num_channels, 1),
                                       groups=F1, bias=False)
        self.bn2           = nn.BatchNorm2d(F1 * D)
        self.pool1         = nn.AvgPool2d((1, 4))
        self.drop1         = nn.Dropout(0.4)
        self.conv_sep_dw   = nn.Conv2d(F2, F2, (1, 16),
                                       padding=(0, 8), groups=F2, bias=False)
        self.conv_sep_pw   = nn.Conv2d(F2, F2, (1, 1), bias=False)
        self.bn3           = nn.BatchNorm2d(F2)
        self.pool2         = nn.AvgPool2d((1, 8))
        self.drop2         = nn.Dropout(0.4)
        with torch.no_grad():
            dummy     = torch.zeros(1, 1, num_channels, num_samples)
            flat_size = self._forward_features(dummy).shape[1]
        self.fc = nn.Linear(flat_size, num_classes)

    def _forward_features(self, x):
        x = F.elu(self.bn1(self.conv_temporal(x)))
        x = F.elu(self.bn2(self.conv_spatial(x)))
        x = self.drop1(self.pool1(x))
        x = F.elu(self.bn3(self.conv_sep_pw(self.conv_sep_dw(x))))
        x = self.drop2(self.pool2(x))
        return x.flatten(1)

    def forward(self, x):
        return self.fc(self._forward_features(x))
```

הקלט שנכנס: טנסור של אותות EEG מעובדים ומנורמלים במבנה הכולל את גודל קבוצת האימון, ערוץ תמונה בודד, 16 ערוצי הרישום של האלקטרודות (המופקים משני שבבי ADS 1299), ונקודות הדגימה של חלון הזמן.

הפלט שיוצא: וקטור מספרי המייצג את ציוני הסיווג וההסתברות עבור ארבע המחלקות השונות של דמיון התנועה.

איך הקוד עובד: קטע קוד זה מגדיר את שלב זרימת המידע קדימה בתוך רשת העצבים (שלב המעבר קדימה). המידע זורם בצורה טורית דרך שלבי הארכיטקטורה: תחילה מבוצע חילוץ תדרים בציר הזמן ולאחריו חילוץ מרחבי המשלב את המידע מכל 16 האלקטרודות במקביל, כאשר השכבות משתמשות בפונקציית אקטיבציה ונרמול שכבות. בשלב זה המידע עובר

שכבת הפחתת מימדים ראשונה ודרופאוט להפחתת רעשים. המשך הזרימה מעביר את המאפיינים המזוקקים דרך שכבות קונבולוציה מיוחדות הניתנות להפרדה ושכבת הפחתה שנייה, שמטרתן לצמצם את כמות הפרמטרים ולמנוע התאמת יתר של המודל. בסיום התהליך, המטריצה הדו-מימדית משוטחת לווקטור חד-מימדי רציף ומוזרמת ישירות אל השכבה הליניארית האחרונה, המתרגמת את תכונות האות לקביעת כוונת התנועה הסופית של המשתמש.

קוד לולאת אימון והערכה מרכזית בשיטת השארת נבדק בחוץ (LOSO Baseline)

```
device       = torch.device("cuda" if torch.cuda.is_available() else "cpu")
num_subjects = len(np.unique(sub_encoded))
all_results  = []
print(f"מריץ LOSO על {num_subjects} subjects...\n")
print(f"{'Subject':>8} | {'Best Test%':>10} | {'Epochs':>6}")
print("-" * 35)

for test_sub in range(num_subjects):
    train_mask = sub_encoded != test_sub
    test_mask  = sub_encoded == test_sub
    X_tr = torch.tensor(np.expand_dims(X_scaled[train_mask], 1), dtype=torch.float32)
    y_tr = torch.tensor(y_encoded[train_mask], dtype=torch.long)
    X_te = torch.tensor(np.expand_dims(X_scaled[test_mask],  1), dtype=torch.float32)
    y_te = torch.tensor(y_encoded[test_mask],  dtype=torch.long)
    train_loader = DataLoader(TensorDataset(X_tr, y_tr), batch_size=64, shuffle=True)
    test_loader  = DataLoader(TensorDataset(X_te, y_te), batch_size=64, shuffle=False)
    model     = EEGNetClean(num_classes=4).to(device)
    optimizer = optim.AdamW(model.parameters(), lr=1e-3, weight_decay=5e-3)
    scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=60)
    counts    = torch.bincount(y_tr)
    weights   = (1.0 / counts.float())
    weights   = weights / weights.sum() * 4
    criterion = nn.CrossEntropyLoss(weight=weights.to(device), label_smoothing=0.1)
    best_acc, patience, best_epoch = 0, 0, 0
    for epoch in range(1, 81):
        model.train()
```

```
for epoch in range(1, 81):
    model.train()
    for inputs, labels in train_loader:
        inputs = augment_eeg(inputs)
        inputs, labels = inputs.to(device), labels.to(device)
        optimizer.zero_grad()
        criterion(model(inputs), labels).backward()
        nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()
    scheduler.step()
    model.eval()
    correct, total = 0, 0
    with torch.no_grad():
        for inputs, labels in test_loader:
            inputs, labels = inputs.to(device), labels.to(device)
            correct += (model(inputs).argmax(1) == labels).sum().item()
            total   += labels.size(0)
    acc = 100 * correct / total
    if acc > best_acc:
        best_acc, best_epoch, patience = acc, epoch, 0
        torch.save(model.state_dict(), f"best_sub{test_sub}.pt")
    else:
        patience += 1
    if patience >= 20:
        break
all_results.append(best_acc)
print(f"{test_sub+1:>8} | {best_acc:>9.1f}% | {best_epoch:>6}")
```

קלט שנכנס:

- אותות EEG מנורמלים ונקיים (16 ערוצי הרישום משני שבבי 1299ADS).
- וקטור תוויות המטרה (4 מחלקות של כוונות התנועה: יד ימין, יד שמאל, רגליים, מנוחה).
- וקטור מזהי הנבדקים (לצורך ביצוע ההפרדה ביניהם).

פלט שיוצא:

- קובצי משקולות שמורים של המודל (קובץ אופטימלי נפרד עבור כל נבדק).
- לוח תוצאות מודפס המציג את אחוז הדיוק המקסימלי שהושג עבור כל משתמש.

איך הקוד עובד:

- **חלוקת המידע (שיטת לוסו):** הלולאה מבודדת בכל פעם נבדק אחד החוצה שישמש כקבוצת בדיקה (טסט) יחידה, בעוד שנתוני כל שאר הנבדקים מאוחדים יחד ומאמנים את המודל.

- **איזון מחלקות ורגולריזציה:** המערכת מחשבת משקולות הפוכות כדי לאזן מחלקות שיש בהן פחות דוגמאות, ומפעילה החלקת תוויות בשיעור של 0.1 כדי למנוע מהמודל לפתח ביטחון מופרז ונוקשות מתמטית.
- **לולאת האימון ואופטימיזציה:** הקוד מריץ עד 80 מחזורים, מפעיל הרחבת נתונים (אוגמנטציה) בזמן אמת, ומשתמש באופטימייזר מתקדם עם מתזמן קוסינוס שמוריד את קצב הלמידה בהדרגה. נוסף על כך, מבוצעת קטימת גרדיאנטים כדי לשמור על יציבות החישובים.
- **עצירה מוקדמת:** בסיום כל מחזור, המודל נבחן על הנבדק שהושאר בחוץ. אם המודל משפר את שיא הדיוק – המשקולות שלו נשמרות. אם חולפים 20 מחזורים רצופים ללא שום שיפור, הריצה נעצרת אוטומטית כדי למנוע התאמת יתר (אוברפיטינג).

```
import copy
N_FEW_SHOT = 20

def finetune_on_subject(base_model, X_few, y_few, n_epochs=30, lr=1e-4):
    model = copy.deepcopy(base_model)
    for name, param in model.named_parameters():
        if 'fc' not in name:
            param.requires_grad = False
    optimizer = optim.Adam(
        filter(lambda p: p.requires_grad, model.parameters()), lr=lr)
    counts   = torch.bincount(y_few)
    weights  = (1.0 / counts.float())
    weights  = weights / weights.sum() * 4
    criterion = nn.CrossEntropyLoss(weight=weights.to(device))
    loader = DataLoader(TensorDataset(X_few, y_few),
                        batch_size=min(8, len(y_few)), shuffle=True)
    model.train()
    for epoch in range(n_epochs):
        for inputs, labels in loader:
            inputs, labels = inputs.to(device), labels.to(device)
            optimizer.zero_grad()
            criterion(model(inputs), labels).backward()
            optimizer.step()
    return model

results_base, results_ft = [], []
print(f"\nFew-Shot Fine-Tuning | {N_FEW_SHOT} trials per subject\n")
print(f"{'Sub':>4} | {'Base%':>6} | {'Fine-tune%':>10} | {'שיפור':>6}")
print("-" * 38)

for test_sub in range(num_subjects):
    train_mask = sub_encoded != test_sub
    test_mask  = sub_encoded == test_sub
    X_te_all = torch.tensor(
        np.expand_dims(X_scaled[test_mask], 1), dtype=torch.float32)
    y_te_all = torch.tensor(y_encoded[test_mask], dtype=torch.long)

    # מפרידים few-shot מכל מחלקה – מהטסט
    few_idx, test_idx = [], []
```

```
print("-" * 38)

for test_sub in range(num_subjects):
    train_mask = sub_encoded != test_sub
    test_mask  = sub_encoded == test_sub
    X_te_all = torch.tensor(
        np.expand_dims(X_scaled[test_mask], 1), dtype=torch.float32)
    y_te_all = torch.tensor(y_encoded[test_mask], dtype=torch.long)

    # מפרידים few-shot מכל מחלקה 5 – מהטסט
    few_idx, test_idx = [], []
    for cls in range(4):
        cls_idx = (y_te_all == cls).nonzero(as_tuple=True)[0].tolist()
        n_few   = min(N_FEW_SHOT // 4, len(cls_idx) // 2)
        few_idx  += cls_idx[:n_few]
        test_idx += cls_idx[n_few:]

    X_few  = X_te_all[few_idx]
    y_few  = y_te_all[few_idx]
    X_test = X_te_all[test_idx]
    y_test = y_te_all[test_idx]
    test_loader = DataLoader(TensorDataset(X_test, y_test),
                             batch_size=64, shuffle=False)

    # CELL 6-טוען מודל הבסיס שנשמר ב
    base_model = EEGNetClean(num_classes=4).to(device)
    base_model.load_state_dict(torch.load(f"best_sub{test_sub}.pt"))

    # דיוק בסיס
    base_model.eval()
    correct, total = 0, 0
    with torch.no_grad():
        for inputs, labels in test_loader:
            inputs, labels = inputs.to(device), labels.to(device)
            correct += (base_model(inputs).argmax(1) == labels).sum().item()
            total   += labels.size(0)
    base_acc = 100 * correct / total
    results_base.append(base_acc)

    # fine-tune
    ft_model = finetune_on_subject(base_model, X_few, y_few)
    ft_model.eval()
```

```
ft_model.eval()
correct, total = 0, 0
with torch.no_grad():
    for inputs, labels in test_loader:
        inputs, labels = inputs.to(device), labels.to(device)
        correct += (ft_model(inputs).argmax(1) == labels).sum().item()
        total   += labels.size(0)
ft_acc = 100 * correct / total
results_ft.append(ft_acc)

delta = ft_acc - base_acc
print(f"{test_sub+1:>4} | {base_acc:>5.1f}% | {ft_acc:>9.1f}% | "
      f"{'+'if delta>=0 else ''}{delta:.1f}%")
```

קלט:

- **נתוני המערכים המקוריים:** מערך אותות המוח המנורמלים, מערך תוויות כוונות התנועה (4 מחלקות), ומערך מזהי הנבדקים.
- **קובצי משקולות המקור:** קובצי המשקולות האופטימליים שנשמרו על הדיסק הקשיח בסיום סל 6 עבור כל נבדק.
- **פרמטר כמות הדוגמאות לכיול:** משתנה קבוע המוגדר ל-20 דוגמאות סך הכל (בפועל: 5 דוגמאות מבודדות מכל אחת מ-4 כוונות התנועה של הנבדק).

פלט:

- **טבלת ביצועים השוואתית מודפסת:** טבלה דיגיטלית המציגה בזמן אמת עבור כל נבדק ונבדק את אחוז דיוק הבסיס, אחוז הדיוק המשופר לאחר תהליך הכיול, ומדד הדלתא (הפרש השיפור באחוזים).
- **רשימות ביצועים מעודכנות בזיכרון:** שתי רשימות דינמיות המרכזות את ציוני הדיוק הסופיים של כל המשתמשים (לפני ואחרי הכיול), אשר ישמשו כקלט ישיר עבור סל 8 לצורך הפקת הגרף המסכם.

איך הקוד עובד:

1. מנגנון פונקציית הכיול והקפאת השכבות

הפונקציה שיוצרת את הכיול מקבלת את מודל הבסיס הקיים ומבצעת לו שכפול עמוק בזיכרון כדי לא לדרוס את המשקולות המקוריות. לאחר מכן, הפונקציה עוברת בלולאה על כל השכבות של הרשת ומקפיאה את הפרמטרים שלהן על ידי חסימת חישוב הגרדיאנטים, **למעט השכבה הליניארית האחרונה שאחראית על הסיווג**. האופטימייזר מוגדר לעדכן אך ורק את השכבה

הפתוחה הזו. הפונקציה מחשבת משקולות לאיזון הדוגמאות ומאמנת את המודל במשך 30 מחזורים מהירים על גבי קבוצת הדוגמאות המצומצמת של הנבדק.

2. פיצול הנתונים הנסתרים (בידוד דוגמאות הכיול)

הלולאה המרכזית רצה על פני כל הנבדקים במאגר. עבור הנבדק שנמצא כרגע בבדיקה, הקוד יוצר מסכת הפרדה ושולף את כל הניסויים שלו. כדי לבצע סימולציה של "משתמש חדש", הקוד מריץ לולאה פנימית על פני 4 המחלקות ולוקח בדיוק את 5 הדוגמאות הראשונות מכל מחלקה (סך הכל 20 דוגמאות) ומקצה אותן לקבוצת הכיול. כל שאר עשרות הדוגמאות הנותרות של אותו נבדק מוגדרות כקבוצת הבדיקה (הטסט האמיתי) ונארזות במזרים נתונים (DataLoader).

3. טעינת מודל המקור וחישוב דיוק הבסיס

עבור הנבדק הנוכחי, הקוד מאתחל רשת עצבים נקייה וטוען לתוכה מהדיסק את קובץ המשקולות השמור של אותו משתמש מסל 6. המודל מועבר למצב הערכה, מריץ את קבוצת הבדיקה (הטסט המבודד) ללא חישוב גרדיאנטים, ומחשב את אחוז הדיוק הנוכחי של המערכת ללא פיין-טיונינג. תוצאה זו נשמרת מיד ברשימת הבסיס.

4. ביצוע הכיול המהיר והערכת השיפור

הקוד קורא לפונקציית הכיול שהוגדרה בתחילת הסל, ומזין אליה את המודל הנטען לצד 20 הדוגמאות המבודדות. הפונקציה מחזירה מודל מעודכן ומכויל. מודל זה מועבר מיד למצב הערכה ונבחן שוב על אותה קבוצת בדיקה בדיוק ששימשה את שלב הבסיס. אחוז הדיוק החדש מחושב ונשמר ברשימת ה-Fine-tune. בסיום, המערכת מחשבת את ההפרש המתמטי בין שתי התוצאות ומדפיסה שורה מסודרת לטבלה המציגה את מדד השיפור של המשתמש.

תיאור מסד הנתונים

על אף שהמערכת פועלת כרכיב קצה בזמן אמת ואינה דורשת מסד נתונים רלציוני כבד, היא נשענת על ארכיטקטורת ניהול קבצים ומאגרי מידע מובנים לצורך שלבי האימון, הכיול והתיעוד של הפעילויות:

מאגר נתוני האימון והכיול (Offline Training Datasets)

לצורך אימון רשת ה-EEGNet ומודל ה-ICA, המערכת מנהלת קבצי נתונים סטטיים השמורים בדיסק המקומי בפורמטים אופטימליים לעיבוד מתמטי (`.npy` של NumPy או קבצי `.csv` מובנים). מאגר זה מחולק ל-2 מקורות:

- **Benchmark Dataset (מאגר ייחוס פומבי):** נתוני EEG מתוקננים של משתמשים מרובים (כדוגמת BCI Competition IV), המשמשים לאימון בסיסי (Pre-training) של הרשת הגנרית.
- **User Calibration Dataset (מאגר כיול אישי):** קובץ המכיל את אותות ה-EEG הגולמיים (16 ערוצים) שהוקלטו מהמשתמש הספציפי במהלך שלב הכיול הראשוני (חלונות זמן של דמיון תנועה ימינה/שמאלה/מנוחה), המשמש להתאמה אישית וכיול אישי.

מדריך למשתמש

1. יש לחבוש את כובע ה-EEG לראשך בצורה נוחה ויציבה.
2. המערכת תבצע כיול אוטומטי של מספר שניות.
3. תוכל לשלוט בדמות על ידי דמיון תנועות התנועות הבאות:

דמיין תנועה של יד ימין/ יד שמאל/ הרגליים.

4. נסה להישאר רגוע וממוקד לדיוק מקסימלי.

בדיקות והערכה

על מנת להבטיח את יציבות המערכת, עמידה בדרישות של עיבוד נתונים בזמן אמת (Real-Time Control) ודיוק הסיווג של גלי המוח, בוצע מערך בדיקות מקיף. המערך כלל בדיקות יחידה (Unit Tests), בדיקות אינטגרציה (Integration) ובדיקות קצה לקצה (End-to-End) של צינור התקשורת מבוסס ה-UDP בין רכיב החומרה, שרת העיבוד וממשק המשתמש.

הבדיקה הראשונה במערך התמקדה ב**מדידת זמני השהיה (Latency) של חבילות המידע מקצה לקצה** – החל מרגע שידורן מבקר ה-32ESP, דרך עיבודן בשרת ה-Python ועד לעדכון האנימציה בממשק ה-React. היעד ההנדסי שהוגדר לפרויקט הציב רף של השהיה ממוצעת הנמוכה מ-20 מילי-שניות עבור חבילת מידע בודדת (לא כולל גודל חלון הזמן הנע). בבדיקה בפועל, המערכת רשמה ביצועים יוצאי דופן. תוצאה שהושגה הודות לשימוש הבלעדי ב- Pure UDP Sockets שמנעו את התקורה (Overhead) המאפיינת פרוטוקולים מבוססי TCP.

השלב הבא הוקדש ל**הערכת חוסנו של אלגוריתם ה-ICA תחת רעשי ארטיפקטים ביולוגיים ומניעת תנועות שווא**. לצורך הבדיקה, המשתמש התבקש לבצע מצמוצי עיניים תכופות וחזקות במטרה לייצר בכוונה רכיבי תדר גבוהים ואמפליטודות חריגות באות ה-EEG הגולמי. תוצאות הבדיקה הראו כי אלגוריתם הניקוי זיהה במדויק את רכיבי הרעש הללו, ביצע להם איפוס מוחלט בתוך מטריצת ההפרדה, ושחזר בהצלחה אות נקי ומטוהר לערוצים המרכזיים (3C ו-4C), מה שאיפשר רציפות עבודה מלאה וחלקה גם תחת רעש פיזיולוגי מוגבר.

מסקנות

תהליך המחקר והפיתוח של מערכת **NeuroMotion** היווה עבורי חוויה לימודית, הנדסית ואישית יוצאת דופן, אשר גישרה באופן מוחשי ומעשי בין התיאוריה האקדמית לבין אתגרי הפיתוח המורכבים בעולם האמיתי. בתחילת הדרך, הקונספט של ממשק מוח-מחשב (BCI) המבוסס על פענוח אותות חשמליים מהמוח בזמן אמת נראה כרעיון כמעט עתידני הרחוק מעולמות התוכנה הקלאסיים. ההתנסות הישירה עם רכיבי חומרה, אלקטרודות EEG וקליטת אותות זעירים באמפליטודות של מיקרו-וולטים, העניקה לי הבנה עמוקה ובלתי אמצעית על האופן שבו מערכות משוב ביולוגי פועלות. הרגע המרגש שבו אותות מוח גולמיים, רועשים ומסורבלים הפכו בפעם הראשונה לפקודת תנועה מוגדרת ויציבה שהניעה את האוואטאר על המסך, היווה נקודת מפנה מרתקת שהמחישה לי את העוצמה האדירה הטמונה בנקודת המפגש שבין מדעי המוח להנדסת תוכנה מתקדמת.

אחד האתגרים בגיזרת הבינה המלאכותית ועיבוד האותות, עברתי חוויה לימודית קריטית שערערה את תפיסותיי המוקדמות לגבי מודלים של למידה עמוקה (Deep Learning). בתחילת הדרך, נטיתי להאמין שהמפתח להצלחת הפרויקט טמון אך ורק במורכבותה של ארכיטקטורת רשת העצבים (EEGNet) ובכוונון עדין של היפר-פרמטרים. עם זאת, דרך התנסות מעשית ממושכת, אימונים כושלים וניתוח שגיאות. גיליתי שללא שלב עיבוד מקדים קפדני, אגרסיבי ומתוחכם – הכולל סינון תדרים דיגיטלי (Band-pass) וניתוח רכיבים עצמאיים (ICA) לניקוי רעשי מצמוצים ותנועות שרירים – גם המודל החכם ביותר לא יוכל להניב ערך סיווגי ממשי. יכולתי לראות כיצד אלגוריתמים מתמטיים מורכבים מנקים את האות ומשפרים באופן ישיר ומיידי את אחוזי הדיוק של המערכת.

לסיכום ,מעבר לשורות הקוד שנכתבו ב Python-וב React-או לעבודה הסיזיפית מול המיקרו-בקר ,החוויה כולה הייתה משמעותית והכנה לעתיד .הפרויקט אילץ אותי לצאת מאזור הנוחות הטכנולוגי שלי ,לצלול לעולם רב-תחומי ולפתח יכולות גבוהות של פתרון בעיות תחת לחץ . לפרויקט היה פן אנושי משמעות ,הידיעה שהמערכת שאנו בונות נועדה בסופו של דבר להעניק פתרון נגיש ,זול ומעצים עבור אנשים המתמודדים עם מגבלות מוטוריות קשות ושיתוק ,נתנה משמעות עמוקה בכל שעת עבודה .הפרויקט הגמר הפך מחובה אקדמית לאחראיות ולחוויה מעוררת השראה ,אשר הותירה בי חותם מקצועי ואמונה עמוקה בכוחה של הטכנולוגיה לשנות חיים.

פיתוחים עתידיים

1. לאפשר למערכת לסווג את כל התנועות האפשריות עליהן האדם יכול לחשוב.
2. חיבור המערכת הנוכחית והמשודרגת עם טכנולוגיית FES:

גירוי חשמלי פונקציונלי שנשלח לשריר מסוים, במיקום מסוים, בעוצמה מסוימת, שאחראי על תנועה מסוימת. טכנולוגיית ה-FES בעצם שולחת זרם חשמלי לשריר אחד לפחות כדי ליצור את התנועה. לדוגמא: כדי ללכת צריך להפעיל כמה שרירים בו זמנית וכל שריר אחראי על תנועה ספציפית אז על ידי אלקטרודות מיוחדות שיושבות על העור נשלח זרם מבוקר שיתן גירוי לשריר ויגרום להפעלתו. הפעלת השריר היא בעצם או

הרפיה שלו או כיווץ שלו, וכך נוצרת פעולה בגוף האדם כאשר יש נכות או לחלופין אפשר לחבר את האלקטרודות מגע המיוחדות הללו למנועי פרוטזה שיופעלו על ידי חיישנים לביצוע פעולה מסויימת.

ביבליוגרפיה

גוגל דבלופרס .(2026) .*קורס מרוכז בלמידה חישובית :רשתות עצביות* Google . Developers.

מדריך תיאורטי מבית גוגל המשמש בסיס להבנת המבנה של רשתות עצבים מלאכותיות , שכבות קונבולוציה ופונקציות אקטיבציה.

https://developers.google.com/machine-learning/crash-course/neural-networks?hl=he

ויקיפדיה).עודכן לאחרונה) .*אלקטרואנצפלוגרם* .ויקיפדיה ,האנציקלופדיה החופשית.

:סקירה כללית על ההיסטוריה והרקע הפיזיולוגי של בדיקות ה ,EEG-הפעילות החשמלית של המוח ומערכת המיקום.10-20

https://he.wikipedia.org/wiki/%D7%90%D7%9C%D7%A7%D7%98%D7%A8%D7%95%D7%90%D7%A6%D7%A4%D7%9C%D7%95%D7%92%D7%A8%D7%9D

מערך הסייבר הלאומי ומשרד החינוך .(2021) .*תקשורת ורשתות :פרוטוקול התקשורת המשובצת .SPI* מתוך מאגר חומרי הלימוד של קהילת הסייבר.

חומר לימוד והסברים טכניים על אופן העברת נתונים סינכרונית בפרוטוקול SPI בין רכיבי קצה ומיקרו-בקרים.

https://data.cyber.org.il/networks/1450-2-11.pdf

- **Blankertz, B., Tomioka, R., Lemm, S., Kawanabe, M., & Müller, K. R.** (2007). *Optimizing spatial filters for robust EEG-based brain-computer interfaces*. IEEE Signal Processing Magazine, 25(1), 41-56.

מאמר מדעי מכונן המסביר את המתמטיקה והאופטימיזציה מאחורי מסננים מרחביים ואלגוריתם ה (Common Spatial Pattern) CSP-לחילוץ מאפייני דמיון מוטורי.

https://www.researchgate.net/publication/3321973_Optimizing_Spatial_filters_for_Robust_EEG_Single-Trial_Analysis

- **Emotiv.** (n.d.). *EEG data analysis software: How to analyze and process EEG data*. Emotiv Science & Research Blog.

מדריך טכני של חברת Emotiv המסביר על מתודולוגיות לעיבוד ,אנליזה וסינון דיגיטלי של אותות EEG גולמיים.

https://www.emotiv.com/he/blog/eeg-data-analysis-software

- **Espressif Systems.** (2025). *ESP32 Technical Reference Manual: Wi-Fi and BSD Sockets API Infrastructure*. Espressif Documentation.

ספר התיעוד הטכני הרשמי של רכיב ה ,ESP32-המשמש בסיס להגדרת תקשורת ה Wi-Fi-האלחוטית ושילוח חבילות ה UDP-במערכת.

https://www.mouser.lt/pdfdocs/ESP32-Tech_Reference.pdf

- **Independent Component Analysis Research.** (2026). *Advanced ICA algorithms for real-time EEG noise reduction and artifact removal*. arXiv preprint arXiv:2606.11722.

מאמר אקדמי מורחב העוסק במימוש אלגוריתם ה ICA-לבידוד ,זיהוי וניקוי אקטיבי של ארטיפקטים ביולוגיים) כמו מצמוצי עיניים (בזמן אמת.

https://arxiv.org/abs/2606.11722

- **Lawhern, V. J., Solon, A. J., Waytowich, N. R., Gordon, S. M., Hung, C. P., & Lance, B. J.** (2018). *EEGNet: A compact convolutional neural network for EEG-based brain–computer interfaces*. Journal of Neural Engineering, arXiv preprint arXiv:1611.08024.

מאמר המקור הרשמי המציג את ארכיטקטורת רשת העצבים ,EEGNet היתרונות שלה בעיבוד אותות מוח ומספר הפרמטרים המצומצם שלה.

https://arxiv.org/abs/1611.08024

- **Texas Instruments.** (2017). *ADS1299 Low-Noise, 8-Channel, 24-Bit Analog-to-Digital Converter for EEG and Biopotential Measurements* (Datasheet Product Report). Texas Instruments Incorporated.

דף הנתונים (Datasheet) הרשמי של רכיב ה-ADS1299, הכולל את המפרט החשמלי, קצבי הדגימה ופרוטוקול הרישום של ה-ADC.

https://www.ti.com/document-viewer/ads1299/datasheet

- **The EEGNet Project.** (n.d.). *EEGNet Ecosystem and International Platform for Brain Data Sharing*.

האתר של פרויקט ה-EEGNet המרכז כלים, קוד פתוח ומאגרי נתונים לשיתוף ופיתוח מודלים מבוססי רשתות עצבים לאותות מוחיים.

https://eegnet.org/

- **Wolpaw, J. R., Birbaumer, N., McFarland, D. J., Pfurtscheller, G., & Vaughan, T. M.** (2002). *Brain–computer interfaces for communication and control*. Clinical Neurophysiology, 113(6), 767-791.

מאמר הבסיס האקדמי של עולמות ה-BCI. מאמר זה מגדיר את מבנה המערכת ההוליסטי (קליטה, עיבוד, הפעלה והדרישות למשוב קוגניטיבי).

https://pubmed.ncbi.nlm.nih.gov/12048038/